AI for Software Engineers Course

Core AI + Machine Learning Foundations (Engineer-Oriented)

https://github.com/havelsan/YZ_EGITIM

Mehmet PEKMEZCİ

Breakthrough

- ***AlexNET - 2012***
  - *Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton*
  - *8 Layer CNN Architecture with 60M parameters,*
  - *Used GPU to calculate the results faster.*
  - *More than 10.8% above the runner-up in ImageNet Large Scale Visual Recognition Challenge (ILSVRC)*

Caution

- ***LLMs Corrupt Your Documents When You Delegate - 2026***
  - *Philippe Laban Tobias Schnabel Jennifer Neville*
  - *Make changes, and change back to its original state*
  - *Best result was 90%*
  - *You can not count on any AI model totally.*

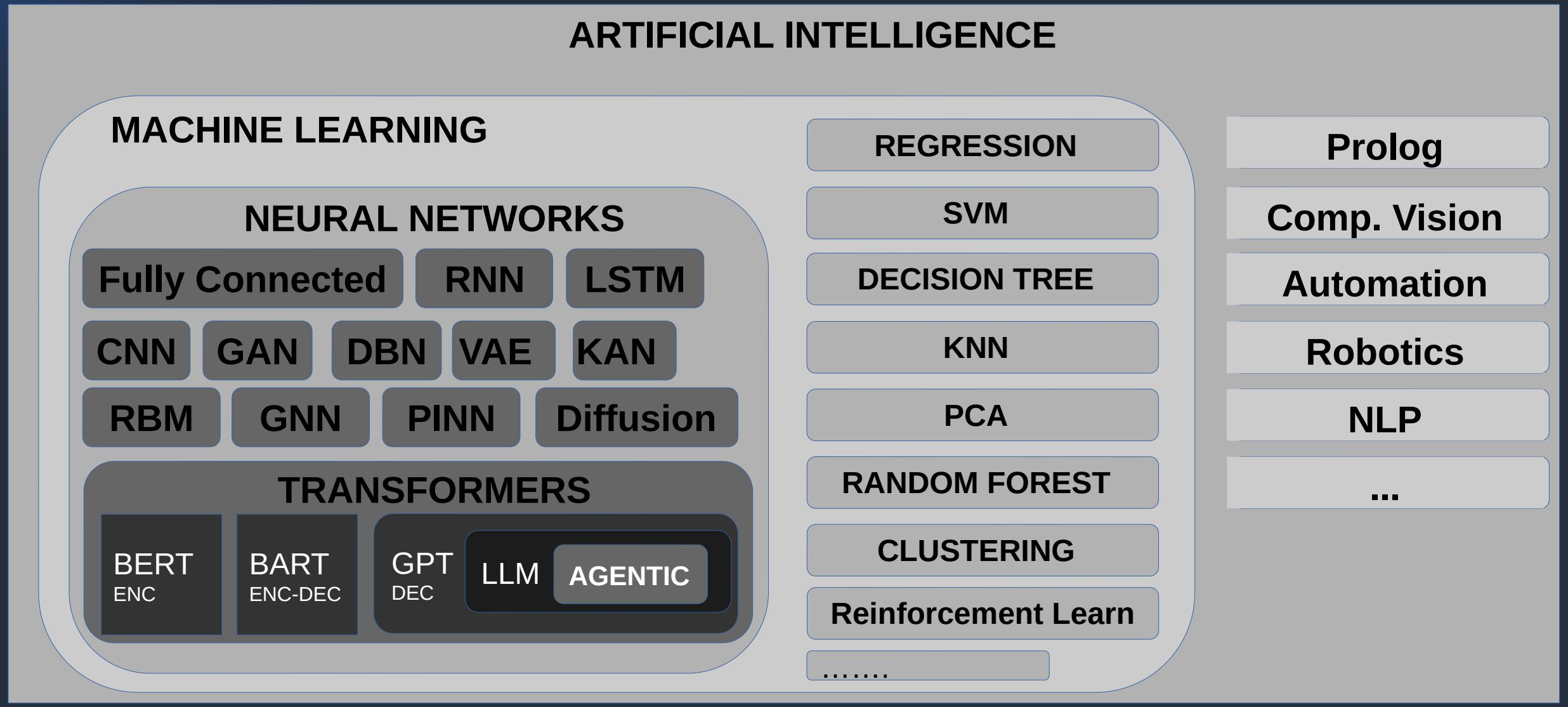
Core AI + Machine Learning Foundations (Engineer-Oriented)

1. *AI landscape for engineers*
2. *How ML systems actually work*
3. *Core ML types (practical framing)*
4. *Neural networks intuition*
5. *Hands-on (important for engineers)*

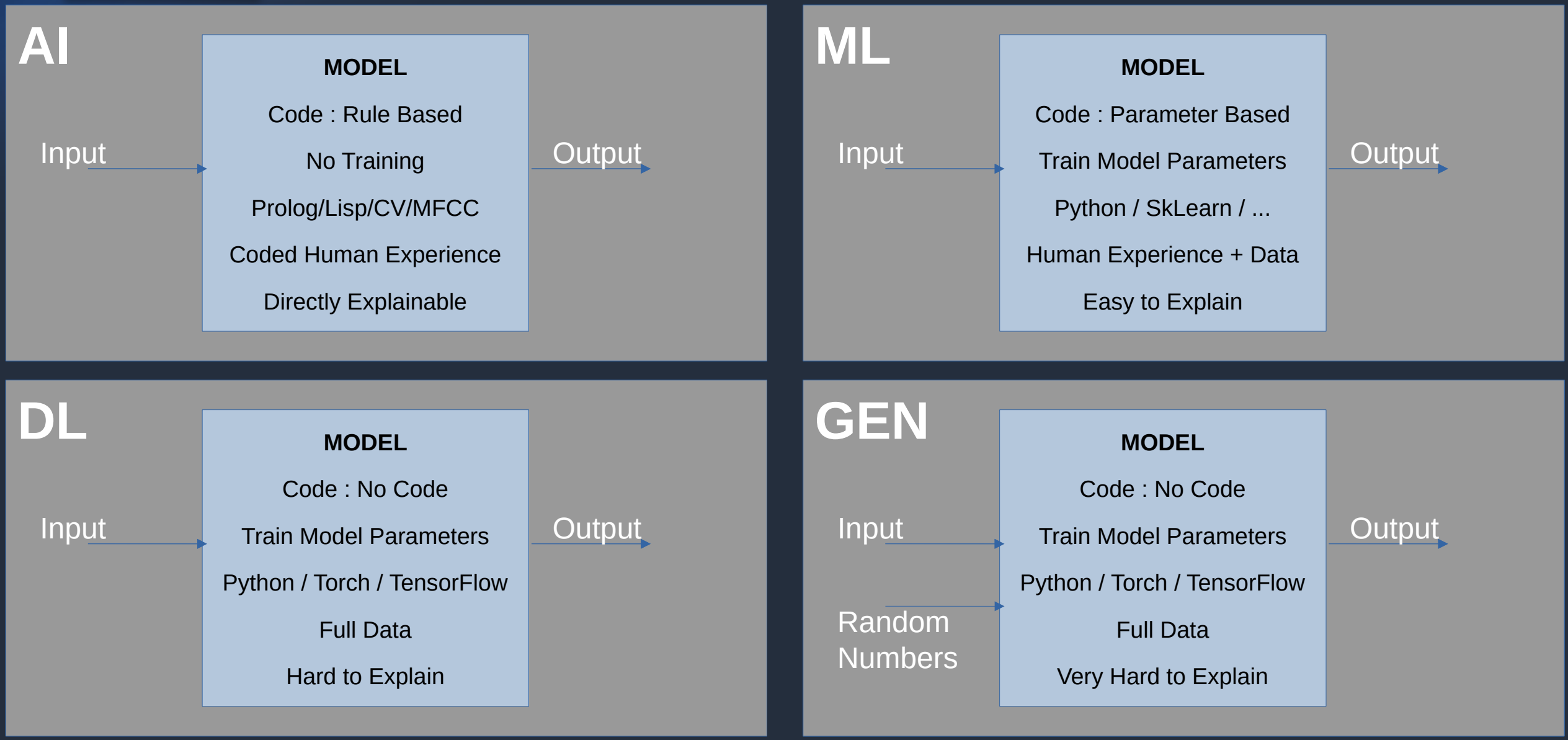
1 - AI Landscape for Engineers

1. *AI vs ML vs Deep Learning vs Generative AI*
2. *Where classical software ends and ML systems begin*
3. *Why ML is “data + optimization + models”*
4. *Roles*

1.1 - AI vs ML vs Deep Learning vs Generative AI



1.1 - AI vs ML vs Deep Learning vs Generative AI



1.2 - Where classical software ends and ML systems begin

Counter Example : Fibonacci Numbers : $a_n=a_{n-1}+a_{n-2}$ $a_{100000}=?$

Math : Binet's Formula (Generating Functions) **(De Moivre 1730)**

$$F(n) = \frac{\phi^n - (1-\phi)^n}{\sqrt{5}} \qquad \phi = \frac{1+\sqrt{5}}{2}$$

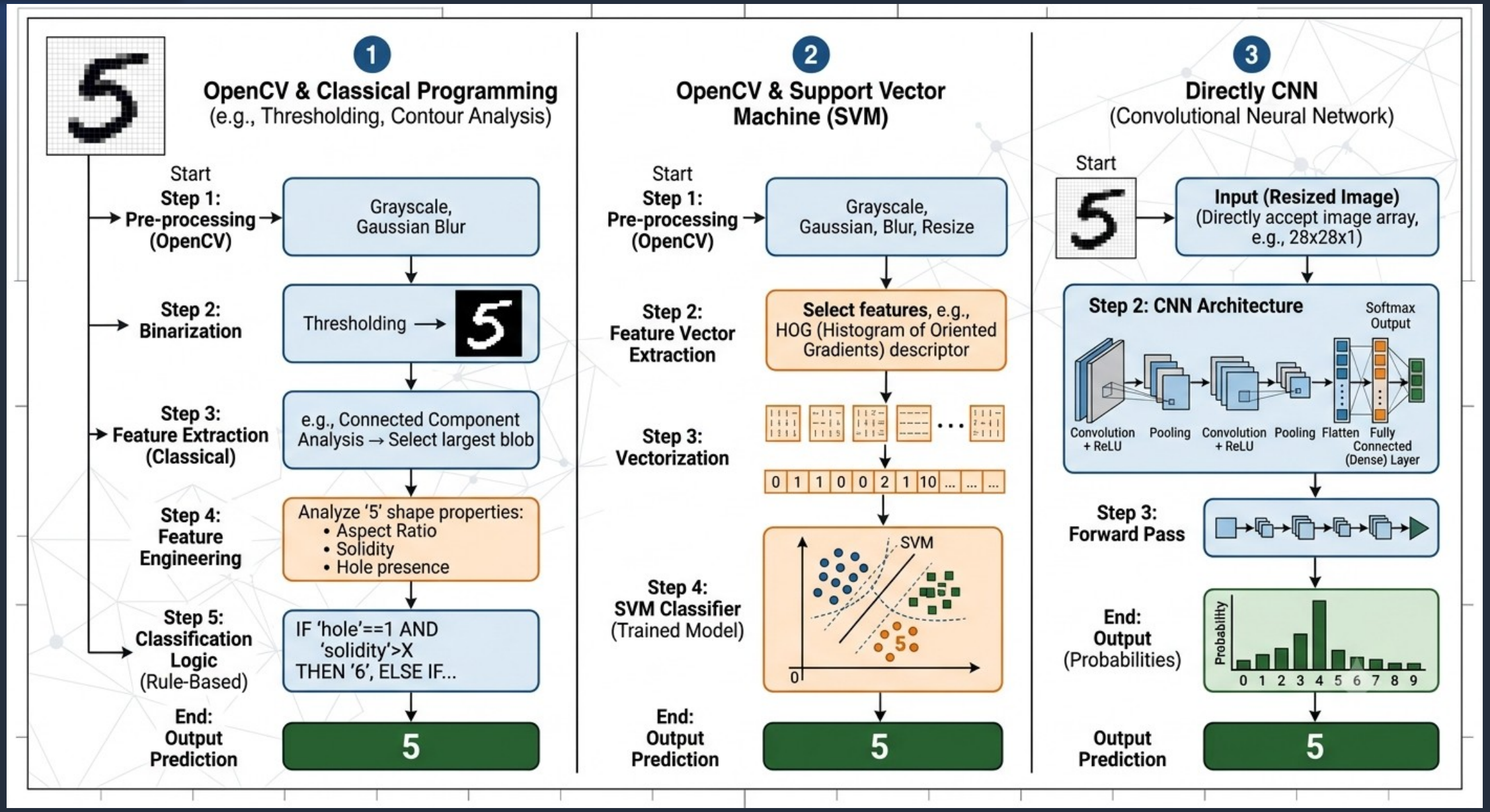
Algorithm :

```
F_n=0 ; F_n_1=2; F_n_2=1
n=5
for  in range(n):
  F_n =  F_n_1 + F_n_2
  F_n_2=F_n_1
  F_n_1=F_n
print(F_n)
```

O(n)

Machine Learning : Generate 1000 data points , train model using these data, try to guess a_{100000} (test the model).

1.2 - Where classical software ends and ML systems begin



1.2 - Where classical software ends and ML systems begin

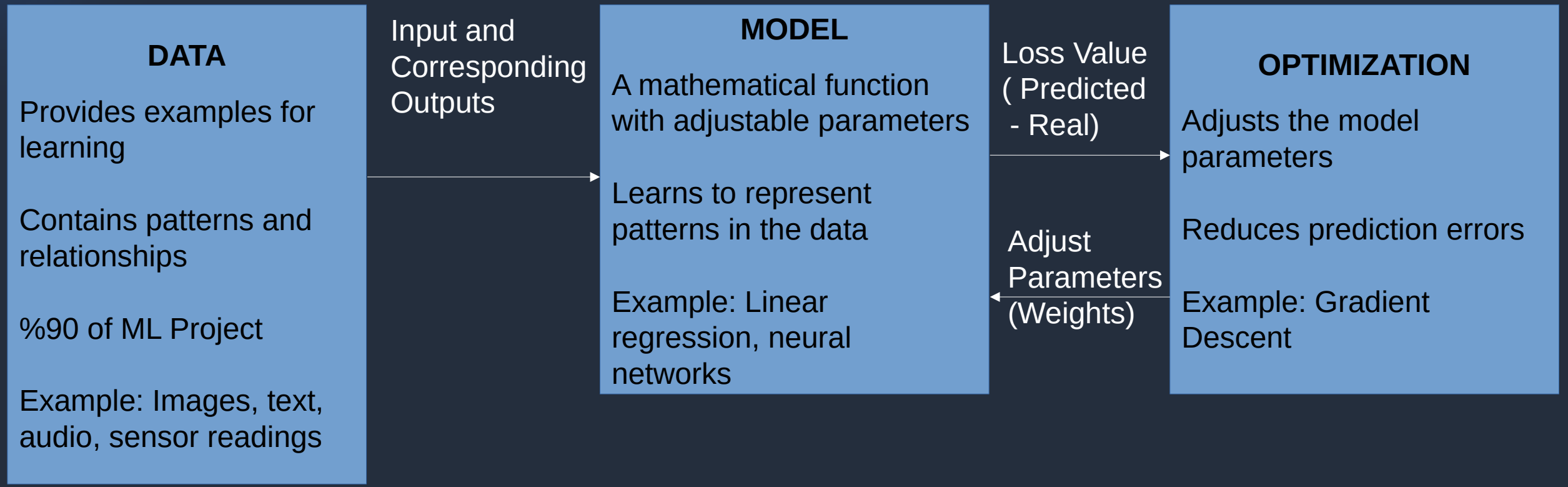
Solving problems with Classical Software :

- Analyze the problem mathematically
- Read previous works and try to generalize the previous solutions to cover your problem.
- **Test your solution on various input/output data of the problem.**
- **Analytical proof exists BUT can not be easily generalized !**

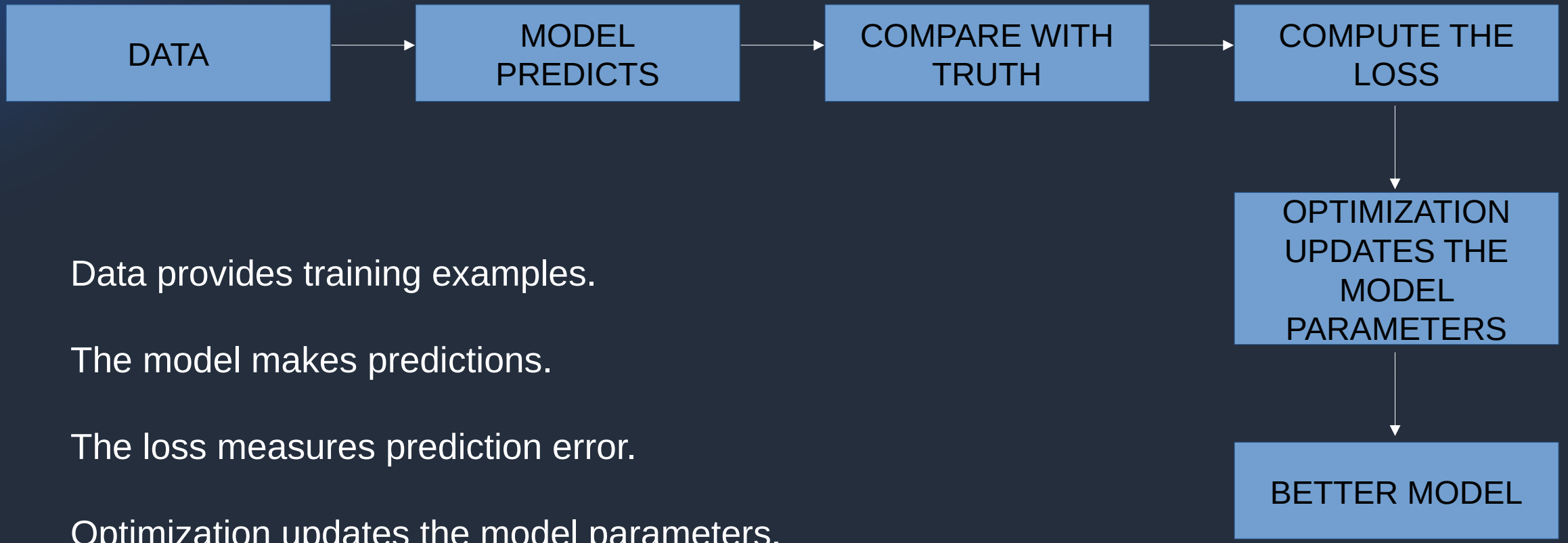
Solving problems with ML/DL :

- Collect real data for the problem.
- Choose the ML/DL Architecture to use. (SVM, CNN, Transformer, ….)
- Train, Validate and Test
- **Good generalization BUT no analytical proof and DEPENDS ON THE DATA QUALITY !**

1.3 – Why ML is “data + optimization + models”



1.3 – Why ML is “data + optimization + models”



Data provides training examples.

The model makes predictions.

The loss measures prediction error.

Optimization updates the model parameters.

The process repeats until the error is minimized

1.3 – Why ML is “data + optimization + models”

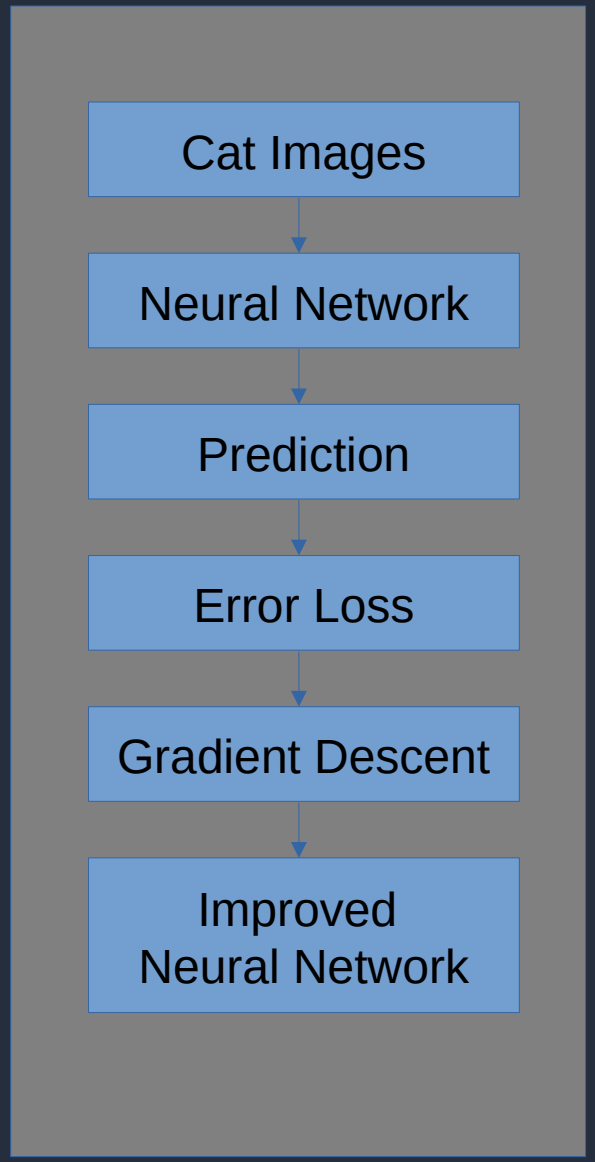
Without	What Happens
Data	No Information to learn from
Model	No function to represent the solution
Optimization	Model parameters never improve

Data tells the algorithm what to learn.

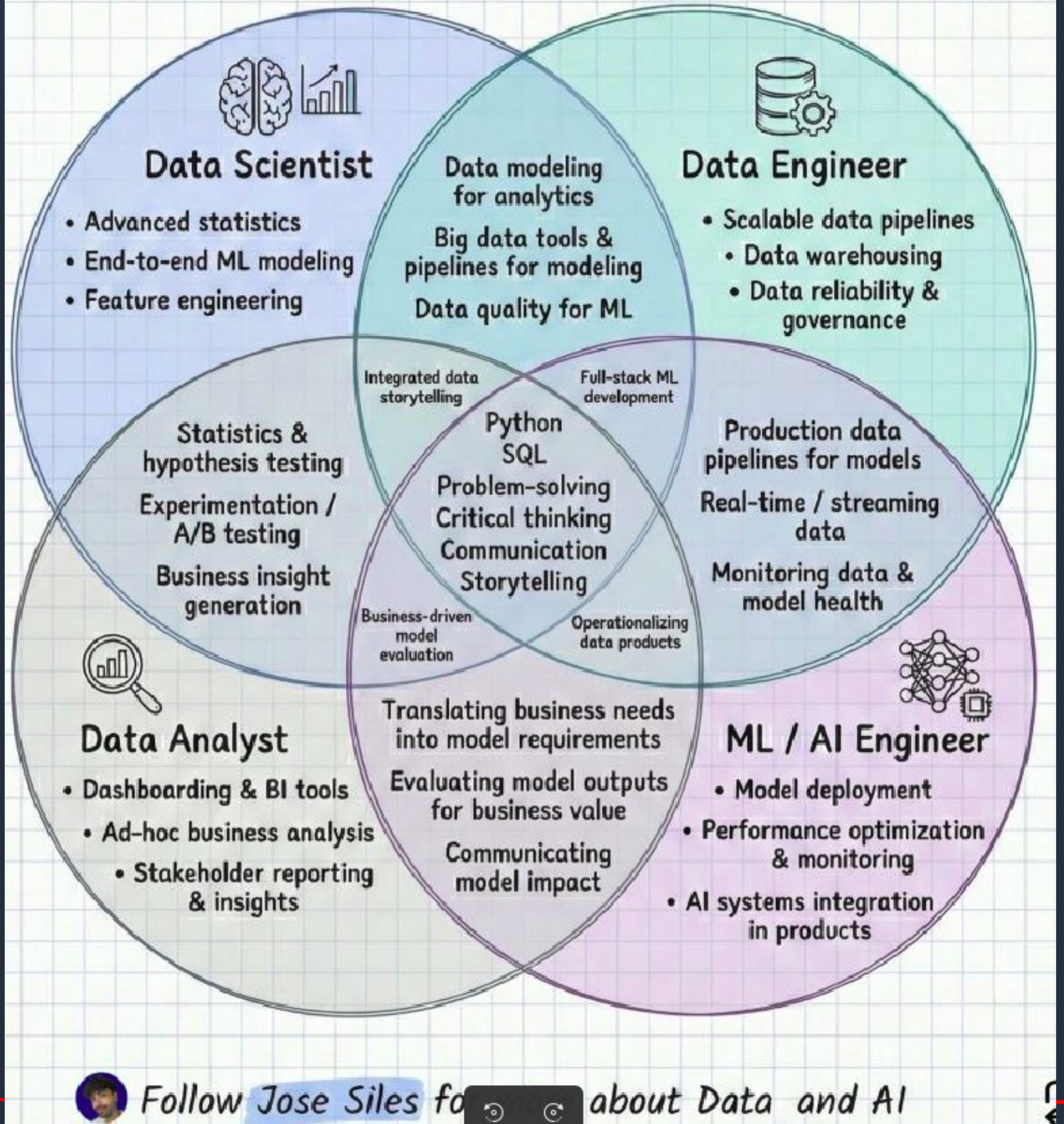
Model defines how knowledge is represented.

Optimization determines how the model improves.

EXAMPLE



1.4 – Roles


Data Scientist
• Advanced statistics
• End-to-end ML modeling
• Feature engineering
Data modeling for analytics
Big data tools & pipelines for modeling
Data quality for ML
Data Engineer
• Scalable data pipelines
• Data warehousing
• Data reliability & governance
Integrated data storytelling
Full-stack ML development
Statistics & hypothesis testing
Experimentation / A/B testing
Business insight generation
Python
SQL
Problem-solving
Critical thinking
Communication
Storytelling
Production data pipelines for models
Real-time / streaming data
Monitoring data & model health
Business-driven model evaluation
Operationalizing data products
Data Analyst
• Dashboarding & BI tools
• Ad-hoc business analysis
• Stakeholder reporting & insights
Translating business needs into model requirements
Evaluating model outputs for business value
Communicating model impact
ML / AI Engineer
• Model deployment
• Performance optimization & monitoring
• AI systems integration in products
Follow Jose Siles fo about Data and AI

2 - How ML Systems Actually Work

1. *Dataset → training → model → inference pipeline*
2. *Train/test split, overfitting (intuition, not math-heavy)*
3. *Loss function intuition*
4. *Feature vs representation learning*
5. *ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix*

2.1 - Dataset → training → model → inference pipeline

PROBLEM DEFINITION

DATA COLLECTION

DATA CLEANING

FEATURE EXTRACTION

TRAINING

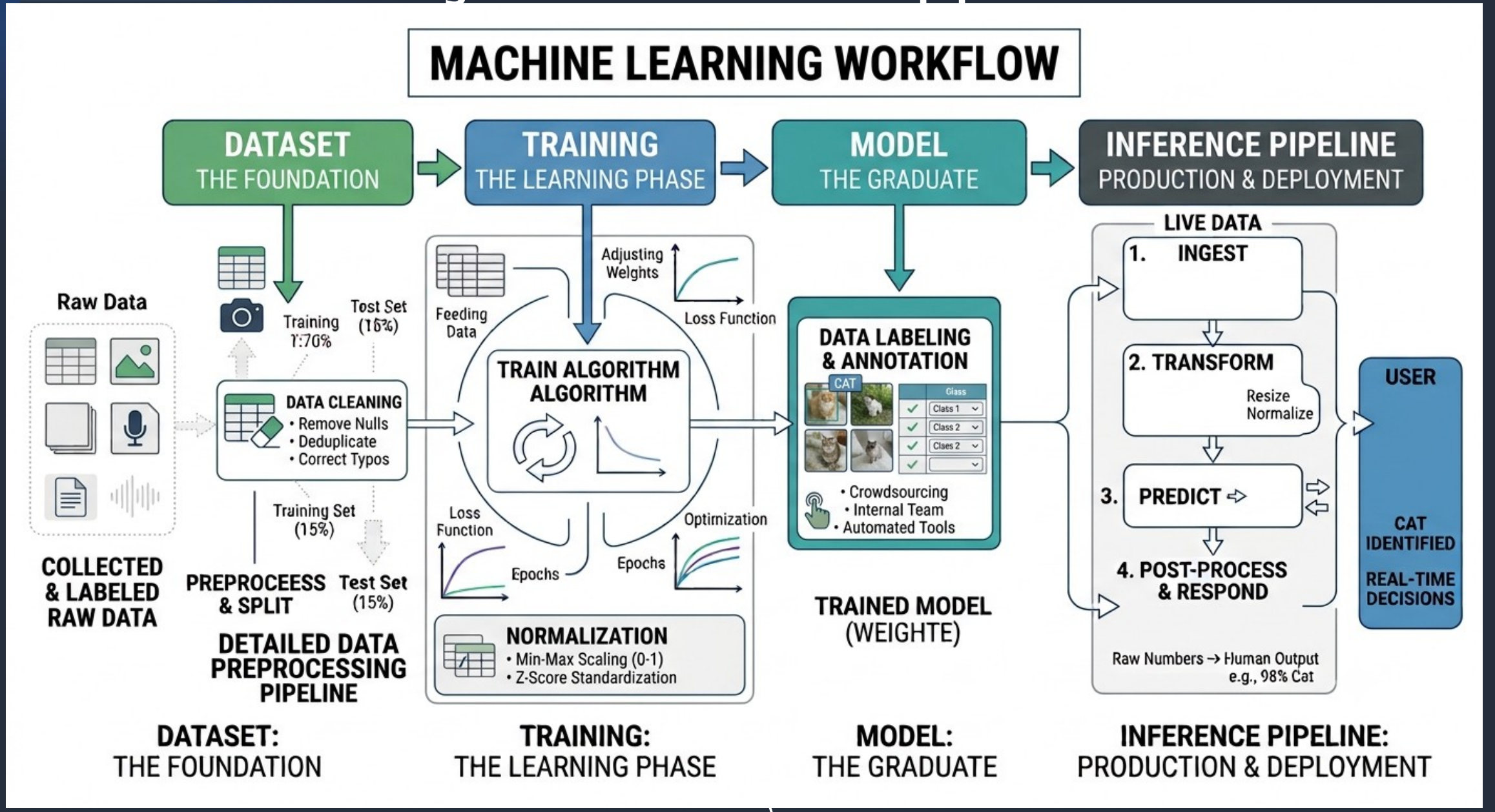
VALIDATION

TESTING

DATA VISUALIZATION

REPORT GENERATION

2.1 - Dataset → training → model → inference pipeline

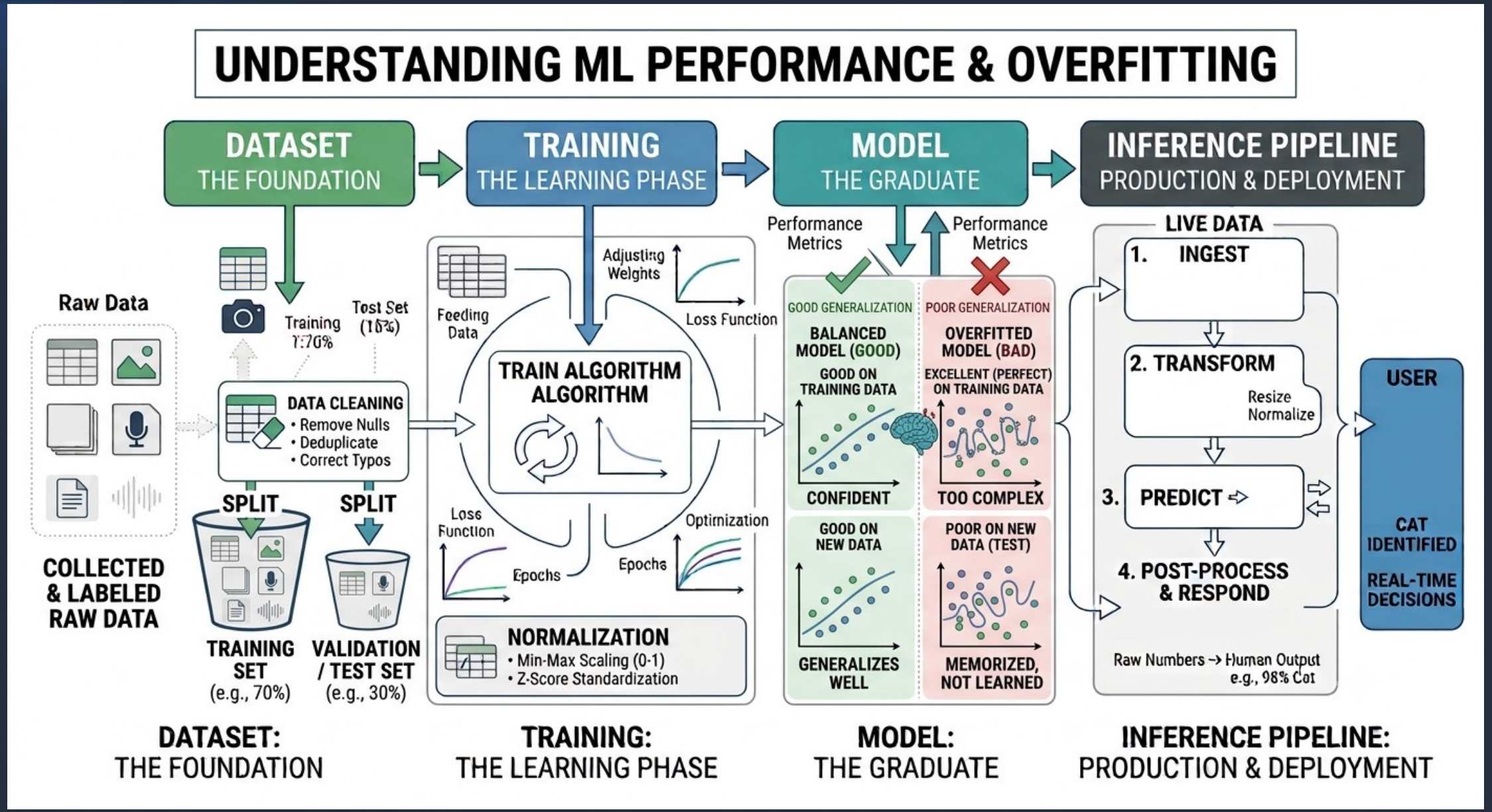


2.1 - Dataset → training → model → inference pipeline

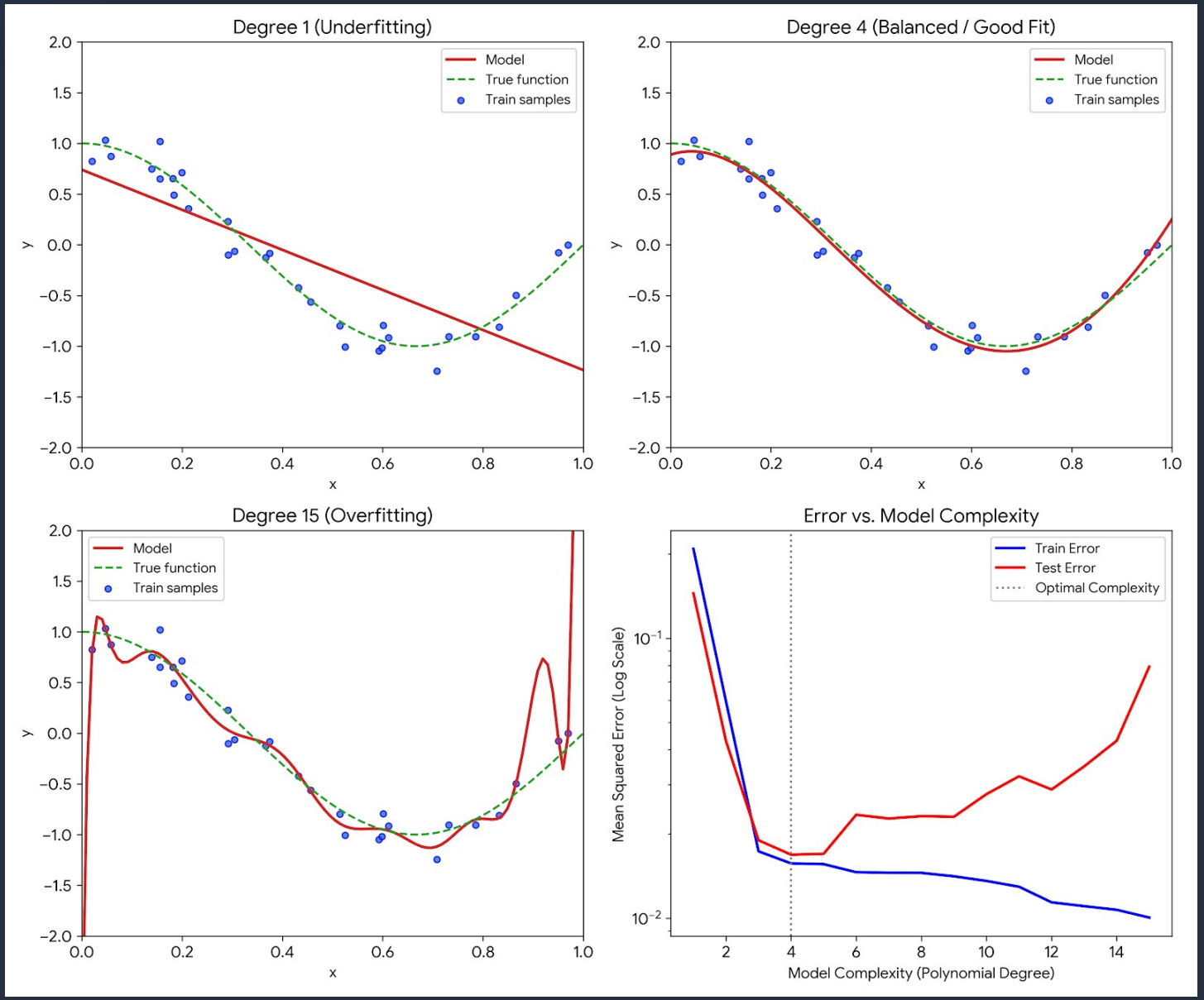
We need a lot of labeled data:

- We need to find **an automated way to collect data**
  - After building a semi automated sound recording system, I was able to collect thousands of sample data.
- We need the data to be **self labeled.**
  - Suppose that , using a monocular (single-camera) system, we try to extract the Bearing and Range of an object :
    - We record camera video,
    - We record AIS data of the objects around us, that correspond to the field of view of camera
    - We record FlightRadar etc. data also.
  - Thank to the AIS and FlightRadar data, we can say that our data is labeled (semi-labeled).

2.2 - Train/test split, overfitting (intuition, not math-heavy)



2.2 - Train/test split, overfitting (intuition, not math-heavy)



2.2 - Preventing overfitting

1. Regularization : Constraining the Model, L1, L2, Elastic, Dropout

2. Modifying the Data : Collect More Data, Augmentation, Feature Selection/Reduction

3. Adjusting Training Process : Early Stopping, Cross-Validation

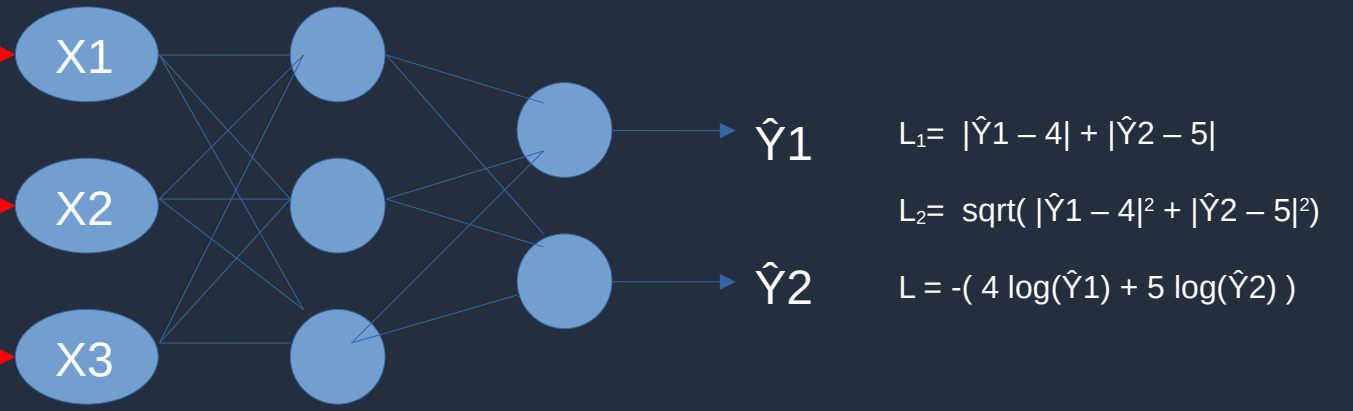
4. Architectural Control and Ensembling : Reduce Model Capacity, **Ensemble Methods (Multiple Small models instead of 1 big model)**

2.3 – Loss Function Intuition

X			Y	
X1	X2	X3	Y1	Y2
1	2	3	4	5
11	12	13	14	15
21	22	23	24	25
31	32	33	34	35
41	42	43	44	45

Loss Function = Distance Function = Metric

Loss = Error = Distance



$L_1 = |\hat{Y}1 - 4| + |\hat{Y}2 - 5|$

$L_2 = \sqrt{|\hat{Y}1 - 4|^2 + |\hat{Y}2 - 5|^2}$

$L = -(4 \log(\hat{Y}1) + 5 \log(\hat{Y}2))$

Some of Loss Functions n = output vector length (in this case 2):

Minkowski Distance : $L_p = \left(\sum_{i=0}^{n} |Y_i - \hat{Y}_i|^p\right)^{\frac{1}{p}}$, L_1 = Manhattan Distance , L_2 = Euclidean Distance

Cross Entropy : $L = -\sum_{i=0}^{n} Y_i . \log(\hat{Y}_i)$, Softmax , Log Loss , n=number of classes , output is one-hot encoded

2.3 – Loss Function Intuition

MAE : Mean Absolute Error

RMSE : Root Mean Squared Error (A type of Euclidean Distance)

Cosine Distance : Used in LLMs, comparing two vectors X.Y / |X| . |Y| = cos between X and Y vectors.

KL Divergence : VAE, GAN (Besides normal loss, the distance from distribution is added to loss)

Mahalonobis Distance : Distance to a group of data points (1-N)

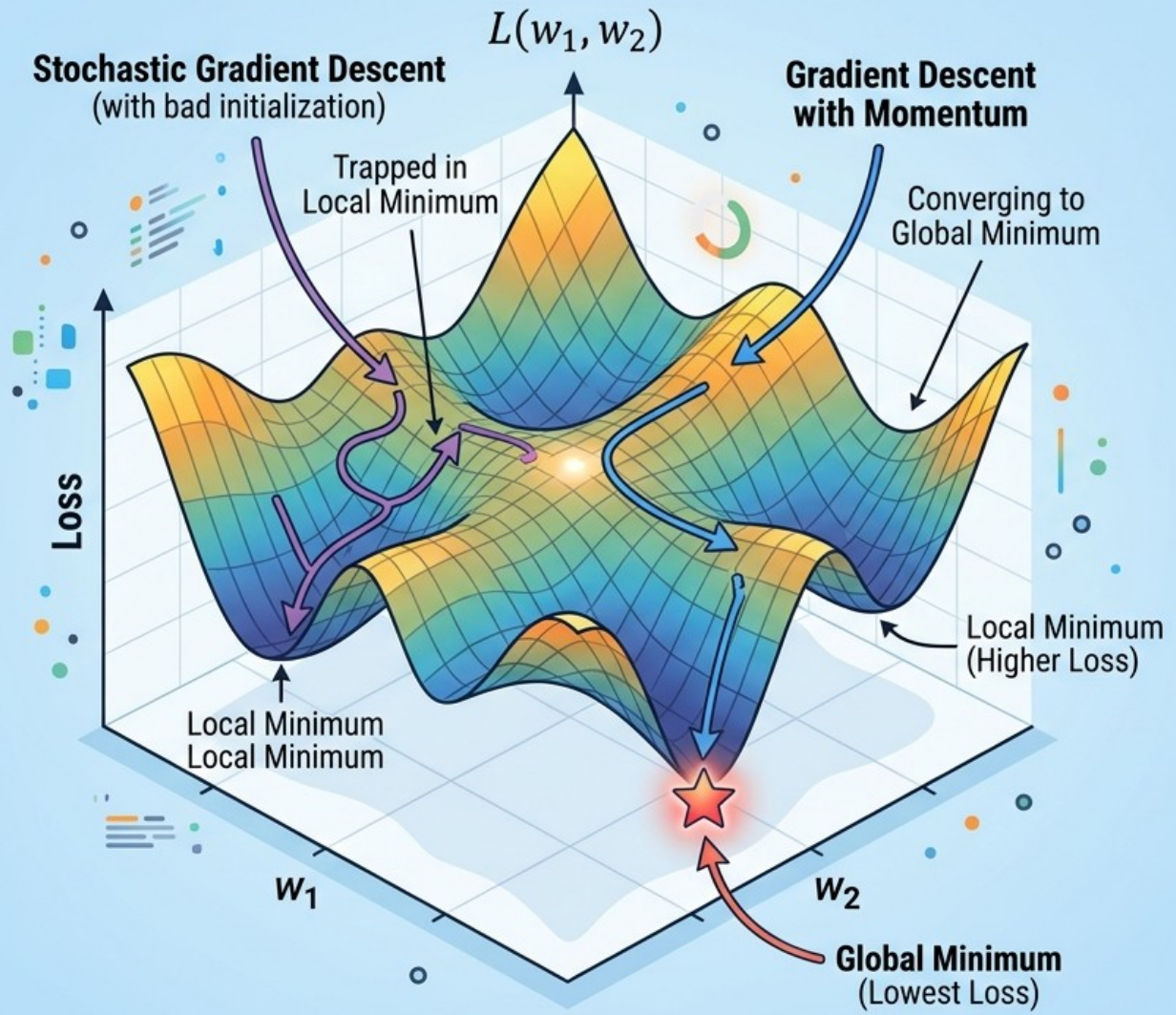
Hinge Loss : Used in SVMs

SSIM (Structural Similarity) : Used to compare images, sound, …

PINN : Loss functions in physics informed neural network is physical formula like wave equation

Dice, Perceptual, Total Variations, …… (Any Metric corresponding to problem nature)

2.3 – Loss Function Intuition



2D LOSS FUNCTION GLOBAL MINIMUM SEARCH

Objective:

Find the set of parameters (w_1, w_2) that minimizes the objective function $L(w_1, w_2)$ over its entire domain.

Challenges:

- **Local Minima** (multiple dips)
- **Saddle Points** (flat regions with different slopes)
- **Plateaus** (very flat areas)
- **Complex Non-Convex Landscapes**

Common Optimization Algorithms for Global Search:

Gradient Descent with Momentum:
Accelerates past local minima

Adam (Adaptive Moment Estimation):
Adapts learning rates, good for sparse gradients

Simulated Annealing / Genetic Algorithms:
Probabilistic and evolutionary approaches, suitable for complex landscapes

2.4 - Feature vs representation learning

Feature Learning (Extraction) : Learn useful predictors.

Representation Learning : Learn a useful encoding of data.

Aspect	Feature Learning	Representation Learning
Goal	Learn useful features from raw data	Learn a meaningful representation of the data
Scope	Narrower	Broader
Output	Features for a specific task	Embeddings or latent representations that capture data structure
Typical Use	Classification, regression	Transfer learning, generation, clustering, downstream tasks
Relationship	A subset of representation learning	Includes feature learning and more

2.4 - Feature vs representation learning

Traditional approach: Human designs features such as edges, corners, texture descriptors (e.g., SIFT or HOG).
Feature learning: A neural network learns filters that detect edges, textures, object parts, etc., during training.
Representation learning: Aims to learn a transformation of the data into a representation that captures important underlying structure.

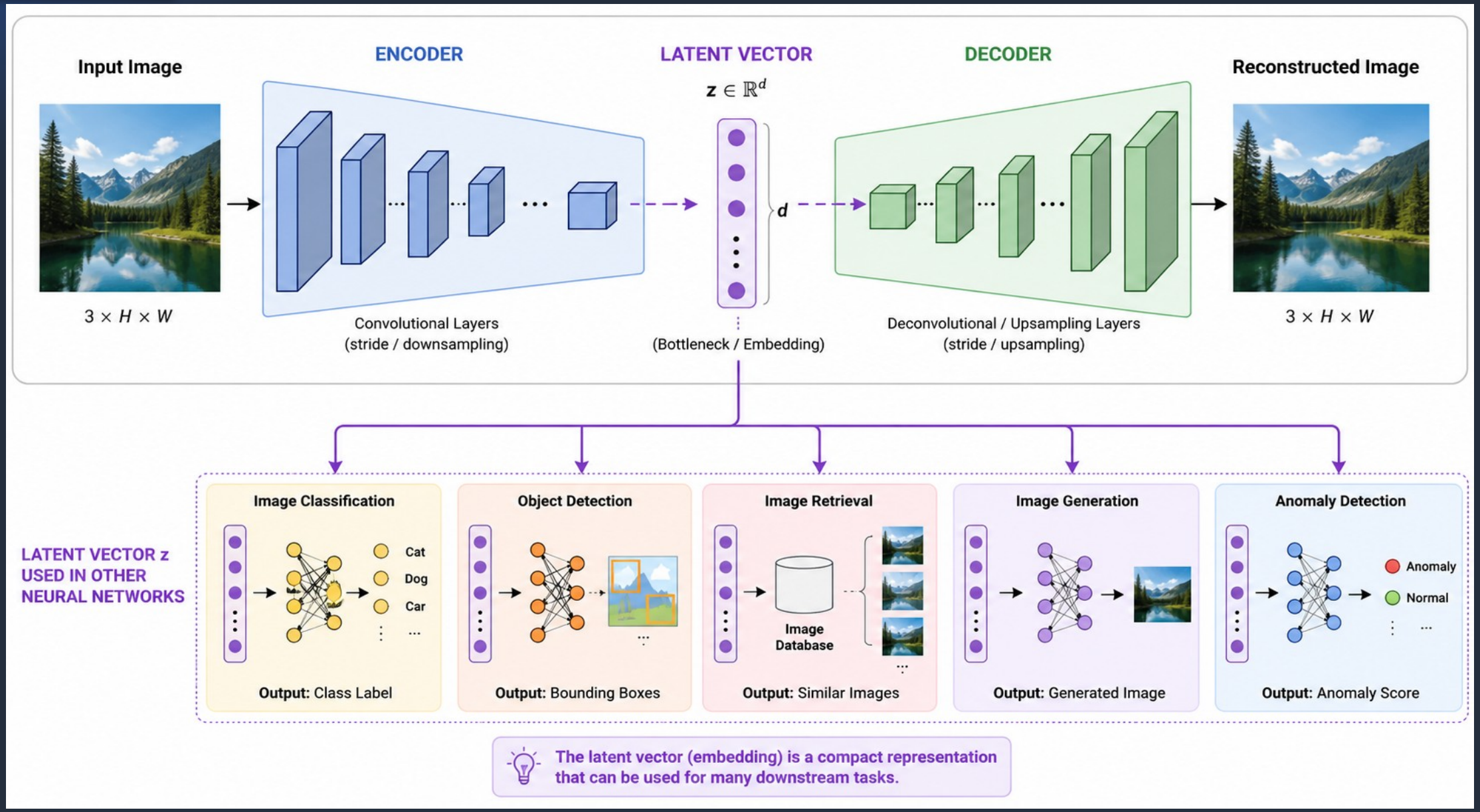
A good representation should ideally:

- Preserve relevant information.
- Discard irrelevant variation (noise).
- Be useful for multiple downstream tasks.
- Capture semantic relationships.

Examples include:

- Latent vectors from autoencoders
- Word embeddings (e.g., Word2Vec)
- Contextual embeddings from BERT
- Vision embeddings from contrastive learning models

2.4 - Feature vs representation learning

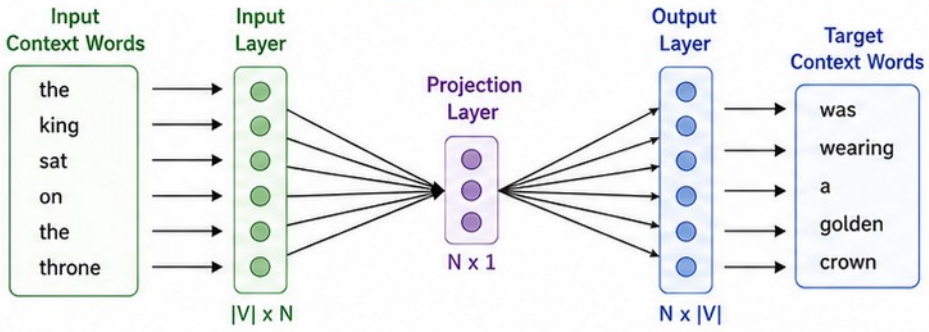


2.4 - Feature vs representation learning

Word2Vec

Words are mapped to dense vectors in a continuous vector space where semantically similar words are close to each other.

Word2Vec (Skip-gram)



Each word is represented as a dense vector (embedding)

Word	v_1	v_2	v_3	...	v_n (N dimensions)
queen	0.21	-0.13	0.34	...	0.12
woman	0.19	-0.24	0.31	...	0.07
man	-0.11	0.22	-0.28	...	-0.09
king	0.22	-0.11	0.35	...	0.10

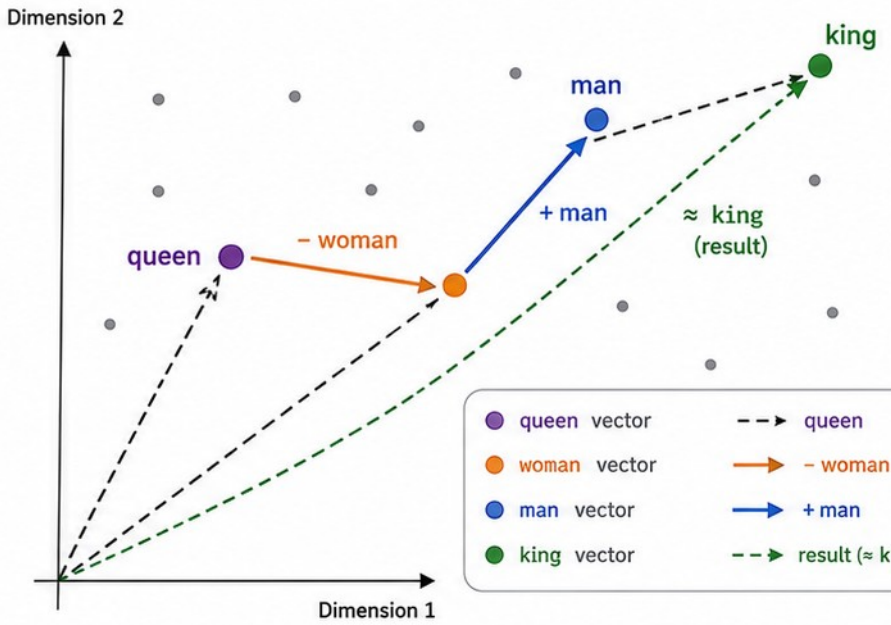
(N is typically 100–300)

Vector Arithmetic Example

queen − woman + man = king

In the vector space, the following relationship holds:

$\overrightarrow{king} \approx \overrightarrow{queen} - \overrightarrow{woman} + \overrightarrow{man}$



Key Idea

Word2Vec learns word embeddings so that words appearing in similar contexts have similar vectors.

This enables meaningful vector arithmetic: queen − woman + man = king.

2.5 - ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

Suppose we have a “machine learning model” that predicts if a track is friend or foe (IFF):

FOE=POSITIVE, FRIEND=NEGATIVE

- If the track is ACTUALLY FRIEND and the model PREDICTED as FRIEND : count as TRUE NEGATIVE
- If the track is ACTUALLY FRIEND and the model PREDICTED as FOE : count as FALSE POSITIVE
- If the track is ACTUALLY FOE and the model PREDICTED as FRIEND : count as FALSE NEGATIVE
- If the track is ACTUALLY FOE and the model PREDICTED as FOE : count as TRUE POSITIVE

Confusion Matrix For Binary Classification

	Predicted Positive	**Predicted Negative**
Actual Positive	Number of True Positives (**TP**)	Number of False Negatives (**FN**)
Actual Negative	Number of False Positives (**FP**)	Number of True Negatives (**TN**)

2.5 - ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positives (TP)	False Negatives (FN)
Actual Negative	False Positives (FP)	True Negatives (TN)

$$PRECISION = \frac{TP}{TP + FP}$$

$$RECALL = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 . PRECISION . RECALL}{PRECISION + RECALL}$$

$$ACCURACY = \frac{TP + FP}{ALL}$$

$$ALL = TP + FP + TN + FN$$

HIGH PRECISION : Model should not miss any FOE, but some of the FOES may be FRIEND actually.

HIGH RECALL : Model may miss some FOEs, but should be sure that no actual FRIEND may be reported as FOE.

HIGH ACCURACY : Model should not miss any FOE, and also should be sure that no actual FRIEND may be reported as FOE.

F1 :

- Completely ignores True Negatives (TN).
- This metric is used in lieu of ACCURACY, when the data is heavily imbalanced.
- Balance between catching positives and being right about them.
-

2.5 - ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

Confusion Matrix : A confusion matrix is a table used to evaluate the performance of a **classification model.**

Each X-Y-Cell Filled By :
How many of detected Class X was actually Class Y ?

Suppose our model will classify the vessels on the sea :

We suppose that it is sufficient to classify the vessels into 3 classes :

1. Naval Vessel
2. Cargo Vessel
3. Passenger Vessel

	Naval Vessel Predicted	Cargo Vessel Predicted	Passenger Vessel Predicted
Naval Vessel Actual	80	30	5
Cargo Vessel Actual	16	650	5
Passenger Vessel Actual	4	20	190

Suppose we made 1000 experiments, 100 were said to be Naval Vessel but actually only 80 of them were Naval, 16 of them were cargo, 4 of them were Passenger ship. For the cargo and the passenger vessels, the numbers are shown above.

2.5 - ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

CONFUSION MATRIX != CORRELATION MATRIX

A confusion matrix evaluates the performance of a classification model by comparing predicted classes to actual, ground-truth classes.

	Naval Vessel	Cargo Vessel	Passenger Vessel
Naval Vessel	80	30	5
Cargo Vessel	16	650	5
Passenger Vessel	4	20	190

A correlation matrix measures and visualizes the statistical relationships (how they move together) between numerical variables in an ACTUAL dataset.

	Vessel has gun	Vessel has radar	Vessel has rooms	Vessel has portholes
Vessel has gun	1.0	0.8	0.1	0.1
Vessel has radar	0.8	1.0	0.1	0.1
Vessel has rooms	0.1	0.1	1.0	0.9
Vessel has portholes	0.1	0.1	0.9	1.0

2.5 - ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

CONFUSION MATRIX IS USED IN THE MODEL EVALUATION PHASE

CORRELATION MATRIX IS USED IN THE DATA CLEANING/FEATURE EXTRACTION PHASES

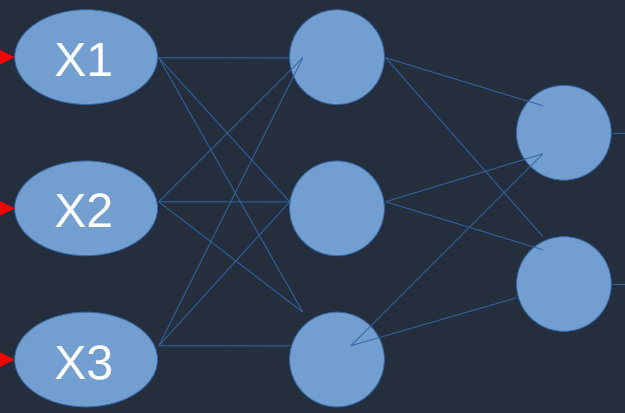
3 - Core ML Types (practical framing)

1. *Supervised learning (classification/regression)*
2. *Unsupervised learning (clustering, embeddings idea)*
3. *Reinforcement learning (high-level only)*
4. *Classical ML Techniques : Regression, KNN, SVM, Decision Tree, Random Forest, PCA, XGBoost, Ensemble Learning, K-Means clustering, DBScan clustering.*

3.1 – Supervised Learning (Classification/Regression)

! LABELED REAL DATA !

X (input)			Y (**labels**)	
X1	X2	X3	Y1	Y2
1	2	3	4	5
11	12	13	14	15
21	22	23	24	25
31	32	33	34	35
41	42	43	44	45



These two values are compared.

PREDICTIONS

Ŷ1

Ŷ2

LOSS = |Ŷ1 – 4| + |Ŷ2 – 5|

LEARNING = WEIGHT ADJUSTMENT

OPTIMIZER (E.G. SGD)

SUPERVISED LEARNING = DATA IS LABELED AND THE MODEL IS LEARNING THOSE LABELS

3.1 – Supervised Learning (Classification/Regression)

1. *Supervised Learning may be split into two types :*

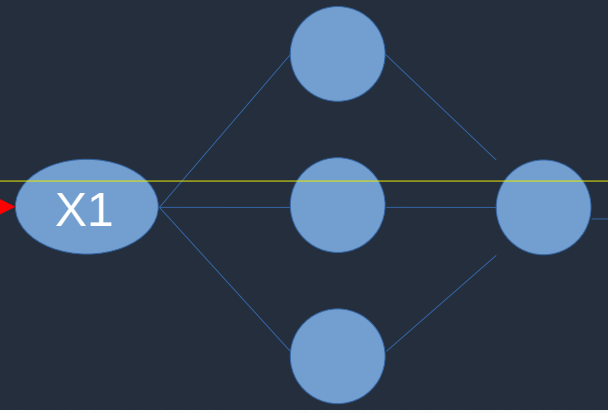
2. *Regression : Curve fitting. When the target variable is a continuous, numeric value, the task is regression. The goal is to predict a specific quantity.*

3. *Classification : When the target variable is a category or a discrete value, the task is classification. The goal is to predict which class or bucket an input belongs to. (Binary/Multi)*

3.1 – Supervised Learning - Regression

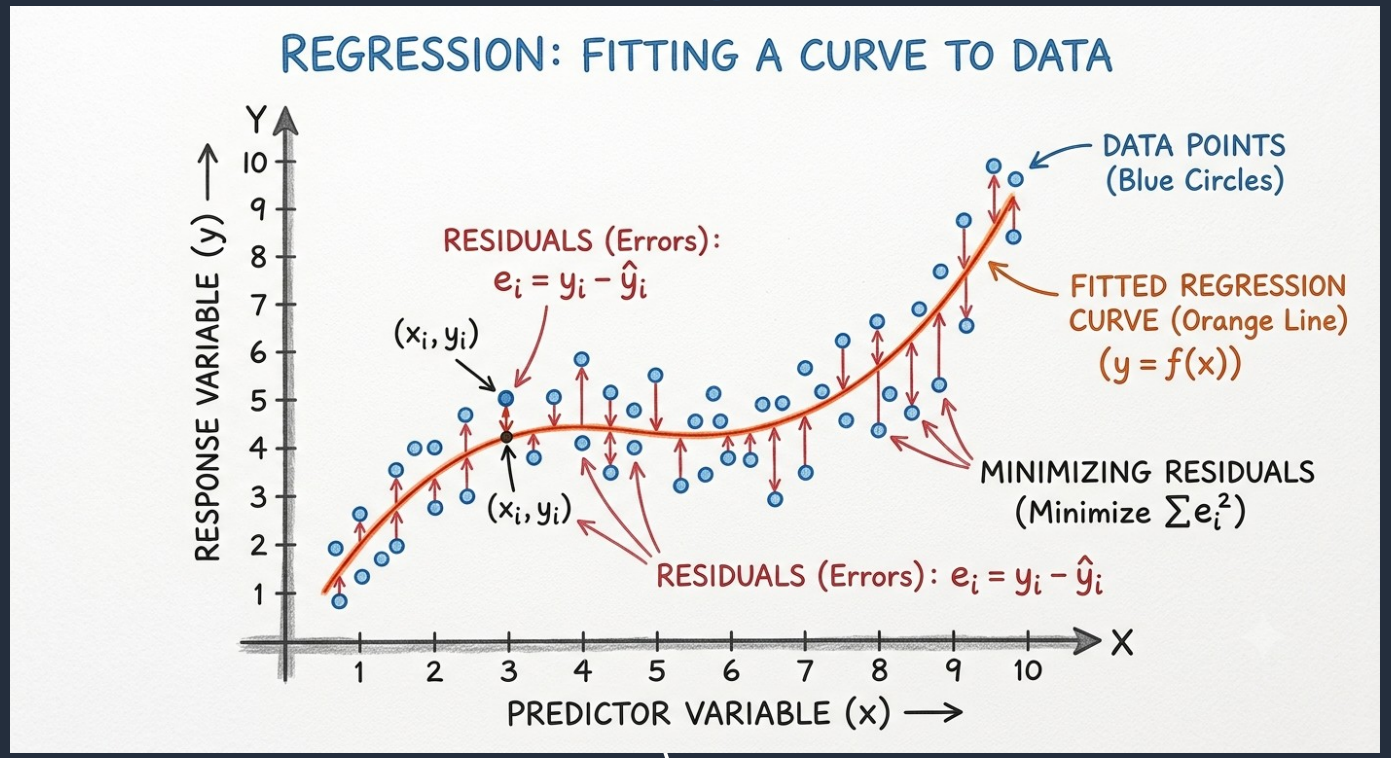
REAL DATA

X1	Y1
0.8	0.9
1	1.4
1.2	1.7
1.5	1.5
2	2.5



PREDICTIONS

$\hat{Y}1$ LOSS = $|\hat{Y}1 - 4|$



3.1 – Supervised Learning - Classification

One-hot-Encoding

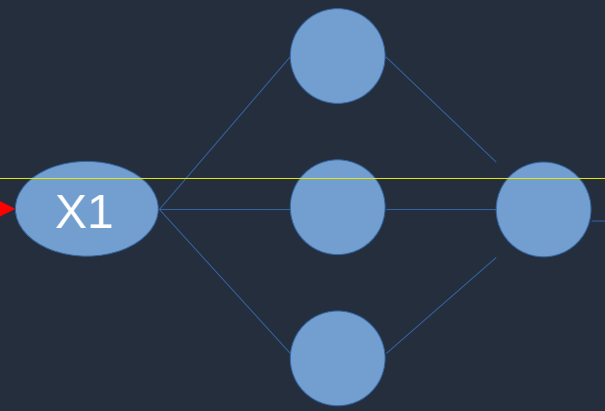
1	0	0

3 classes, vector length is 3, vector is composed of 1 and 0's.

Class-1 is 100
Class-2 is 010
Class-3 is 001

REAL DATA

X1	Y1
0.8	1
1	1
1.2	2
1.5	2
2	3

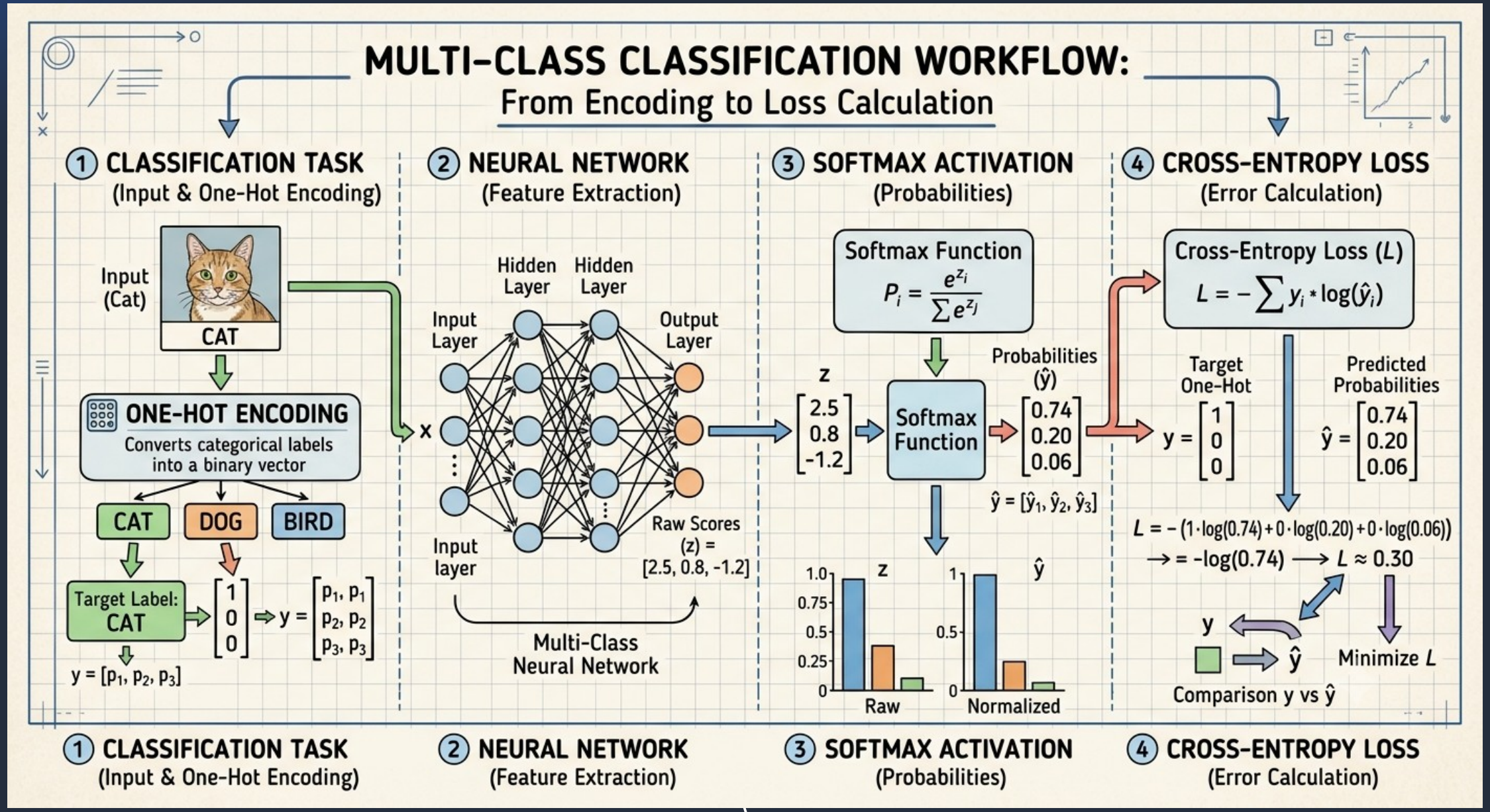


PREDICTIONS

Ŷ1
One-hot-Encoded
Softmax

LOSS = Cross Entropy
$= \Sigma\ Y1 \log(\hat{Y}1)$

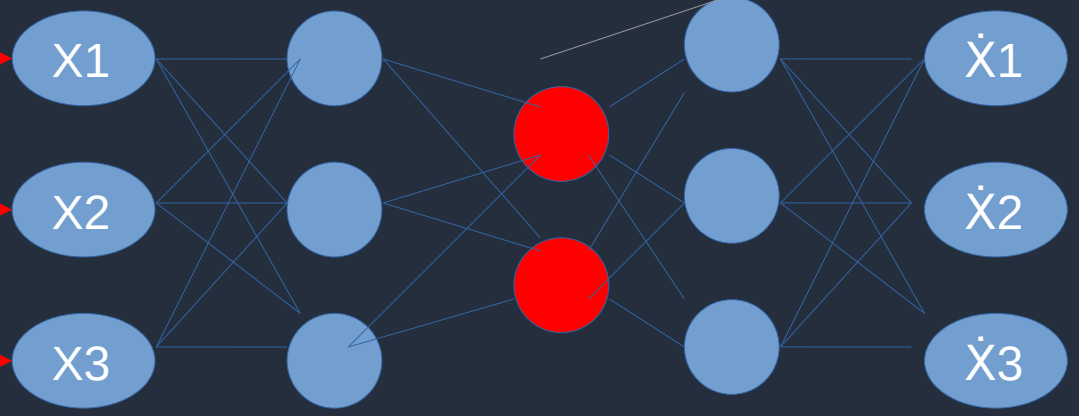
3.1 – Supervised Learning - Classification



3.2 – Unsupervised Learning (Embedding)

! NOT-LABELED REAL DATA !

X1	X2	X3
1	2	3
11	12	13
21	22	23
31	32	33
41	42	43



We use this latent vector to better represent data. E.g. Word Embeddings

LATENT VECTOR

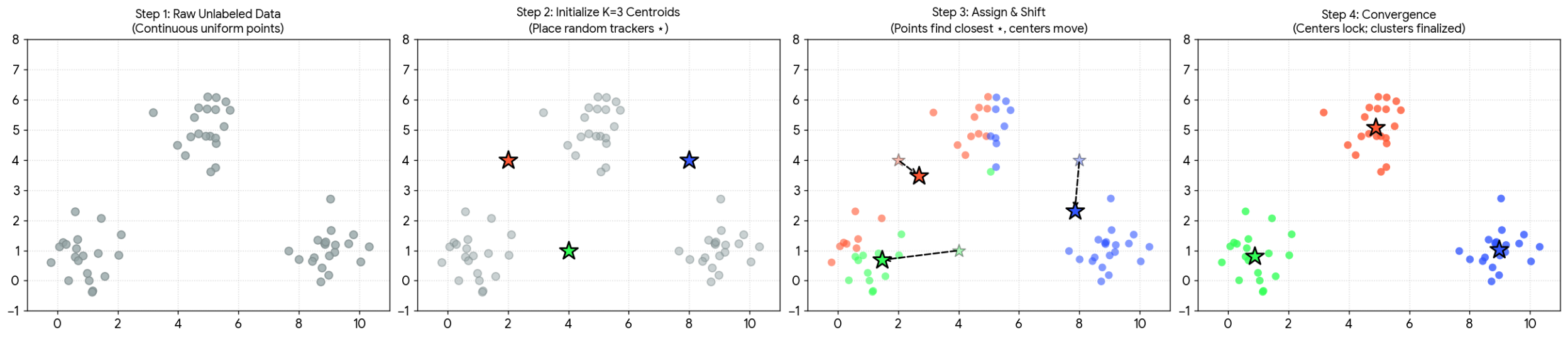
LOSS = $|\dot{X}1 - X1| + |\dot{X}2 - X2| + |\dot{X}3 - X3|$

LEARNING = WEIGHT ADJUSTMENT

OPTIMIZER (E.G. SGD)

UNSUPERVISED LEARNING = DATA IS NOT-LABELED AND THE MODEL IS LEARNING DATA FEATURES

3.2 – Unsupervised Learning (Clustering)



Step 1: Raw Unlabeled Data: The dataset starts entirely gray. The computer only tracks the coordinates (X_1, X_2) without knowing what category anything belongs to.
Step 2: Initialize K=3 Centroids: You select the value of K. The algorithm seeds initial tracker points, known as centroids (shown as colored stars), across the data area.
Step 3: Assign & Shift:

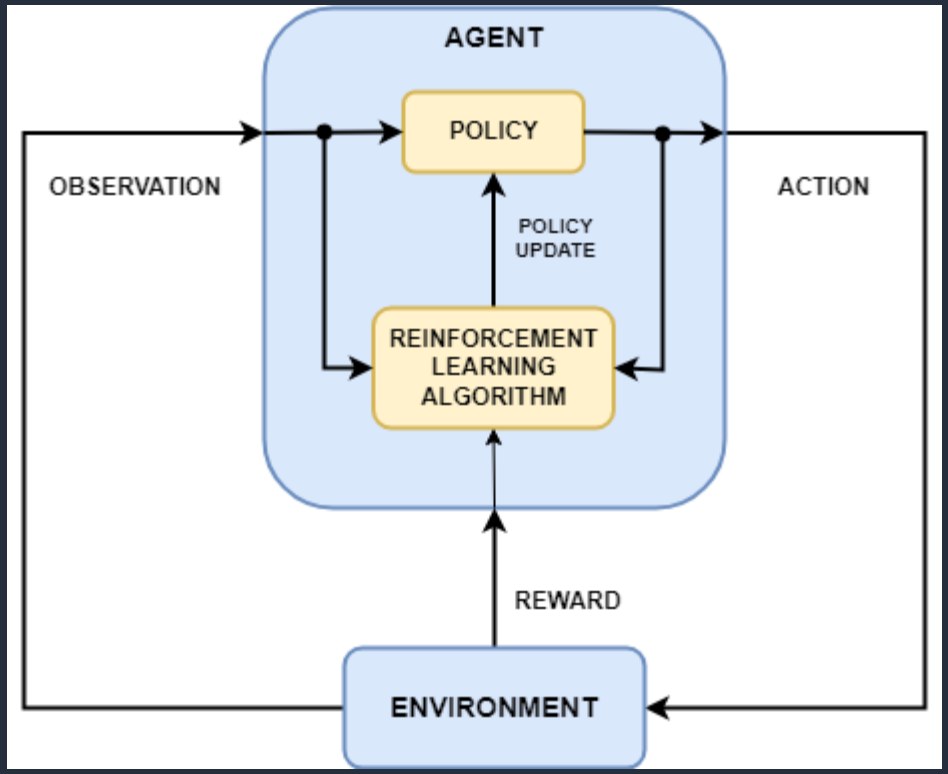
Assignment: Each gray data point identifies which colored star is closest to it and temporarily adopts that color group.

Shift: Each star calculates the geometric center (the mean coordinate) of its newly claimed colored dots and shifts directly along a vector trajectory (indicated by the dashed arrows).

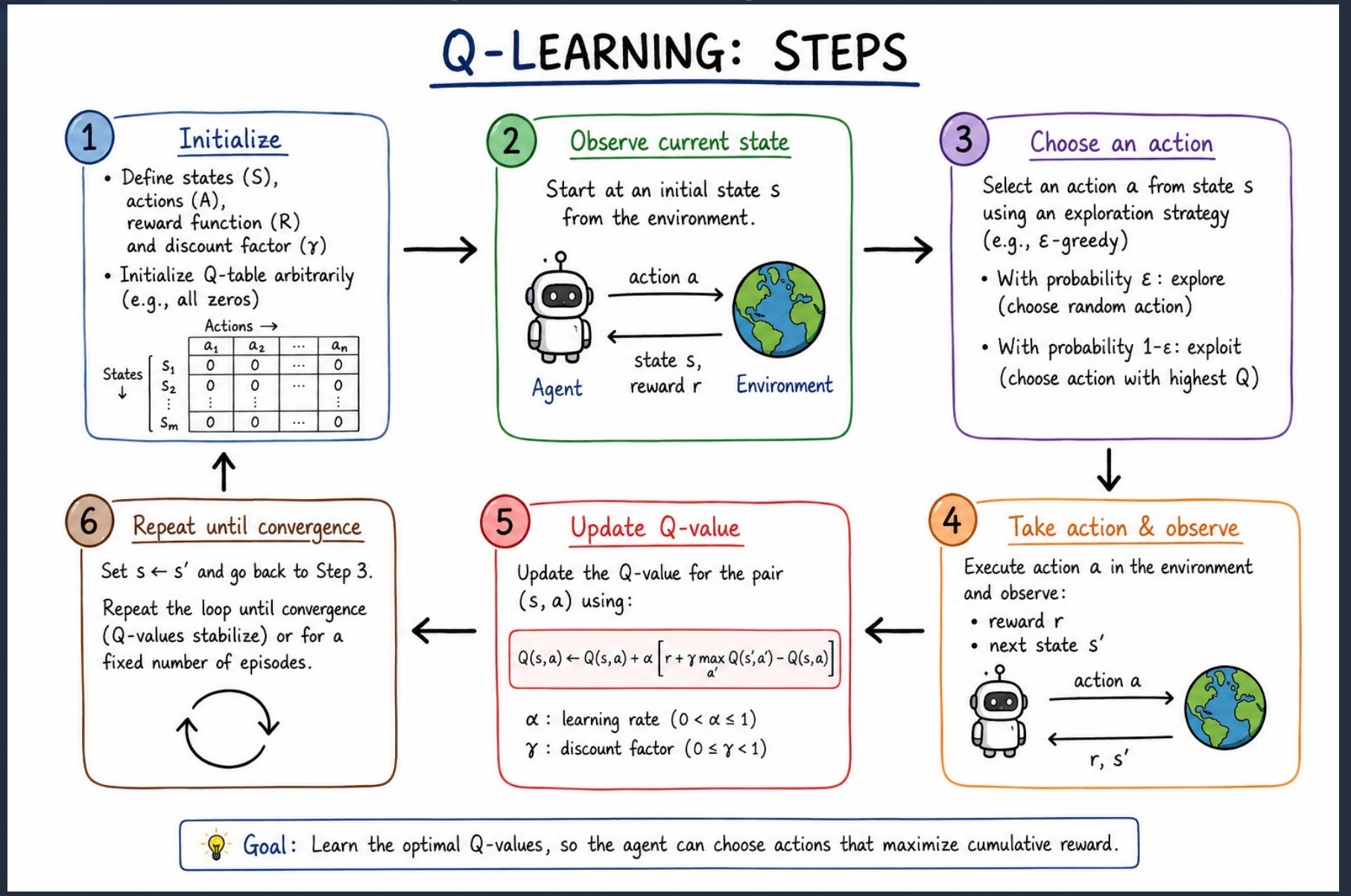
Step 4: Convergence: The assign-and-shift cycle loops automatically until the stars stop moving. The borders lock, yielding isolated, color-coded clusters.

3.3 – Reinforcement Learning

- Learns through interaction with an environment by trial and error.
- Agent selects actions based on the current state to maximize cumulative reward.
- Environment provides feedback in the form of rewards and next states.
- Balances exploration and exploitation to discover optimal strategies.
- Uses a policy or value function (e.g., Q-values) to improve decision-making over time.
- Applications include robotics, game playing, autonomous driving, recommendation systems, and resource optimization.
- https://www.youtube.com/shorts/ty7RamY3gZs (INVERTED PENDULUM)
- https://www.youtube.com/shorts/gNHdsElNYRg



3.3 – Reinforcement Learning - Q-learning



3.3 – Reinforcement Learning - Q-learning

Q-Learning: Learning from Experience

Q-Table
(Agent's Memory)

State	↑	↓	←	→
(0,0)	**2.4**	-0.5	-1.2	**5.8**
(1,1)	3.1	**7.2**	-2.0	4.5
(2,3)	6.3	8.1	**9.5**	-3.1

- High value (good action)
- Low value (bad action)

Bellman Update

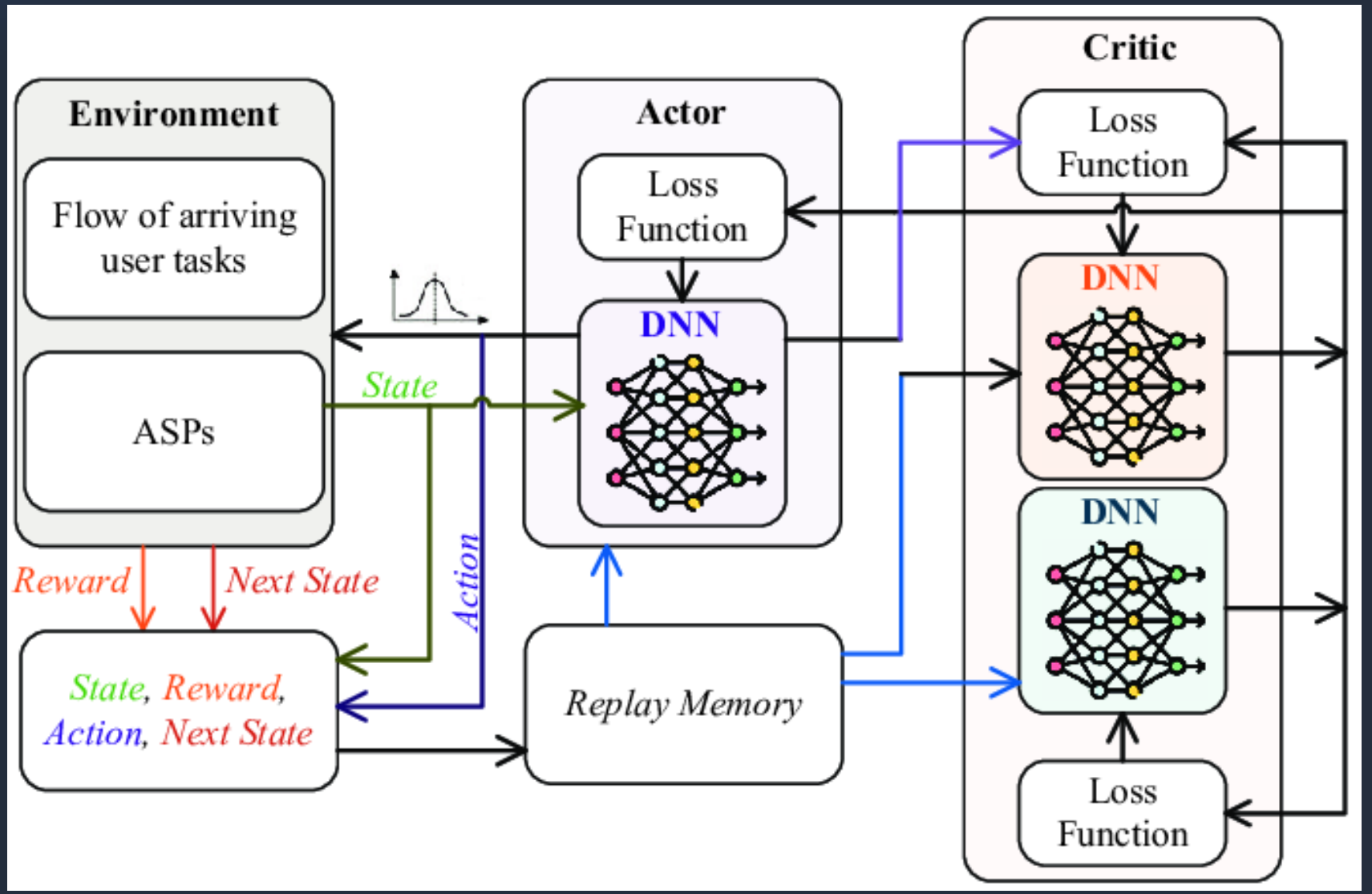
$$Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma \max Q(s',a') - Q(s,a)]$$

- **α** = Learning rate (0.1)
- **r** = Immediate reward
- **γ** = Discount factor (0.95)
- **max Q(s',a')** = Best future value

Updates Q-Table

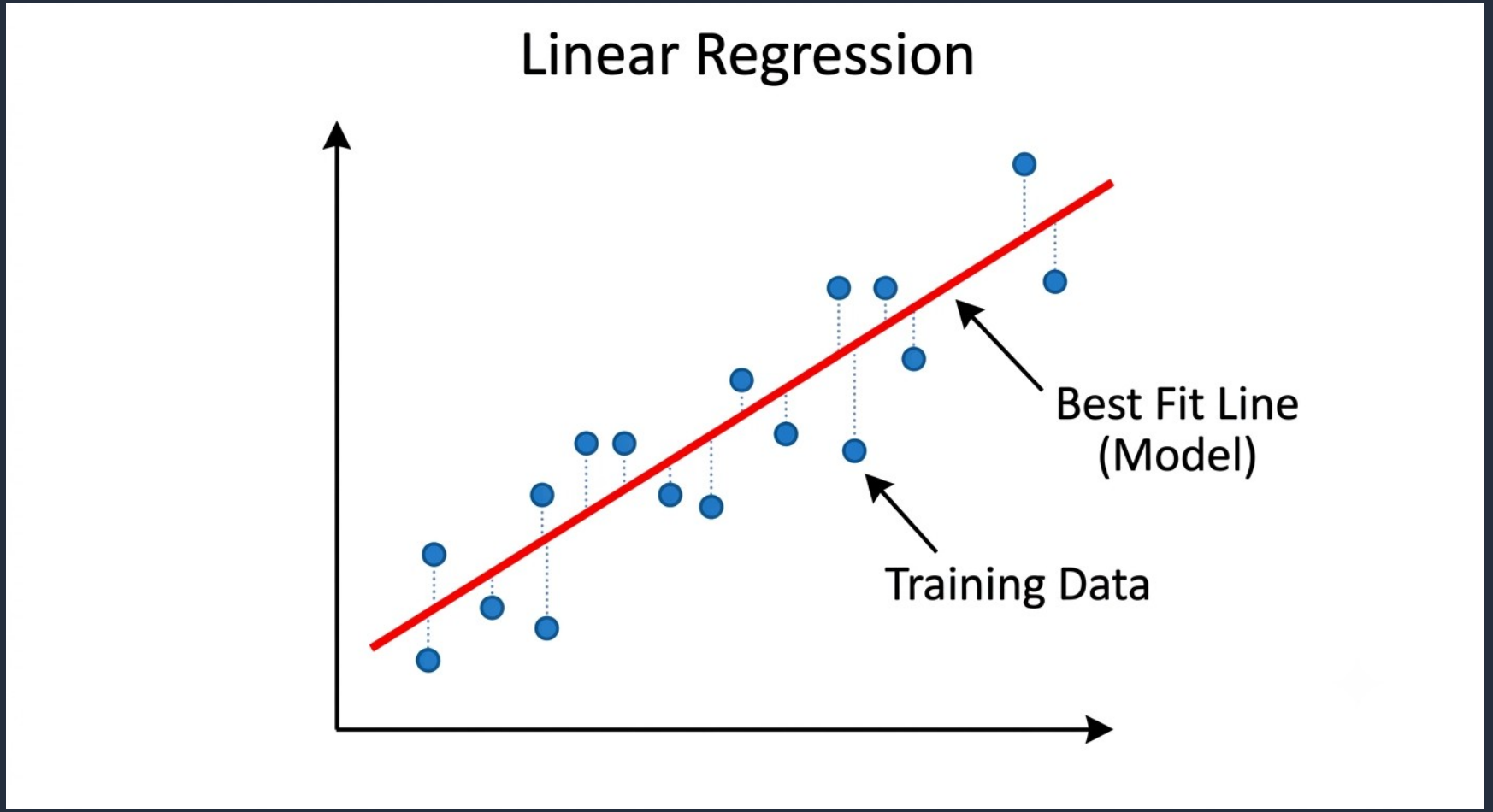
Epsilon-Greedy: ε = 10% explore (random) 90% exploit (best Q-value)

3.3 – Reinforcement Learning – Soft-Actor-Critic (SAC)



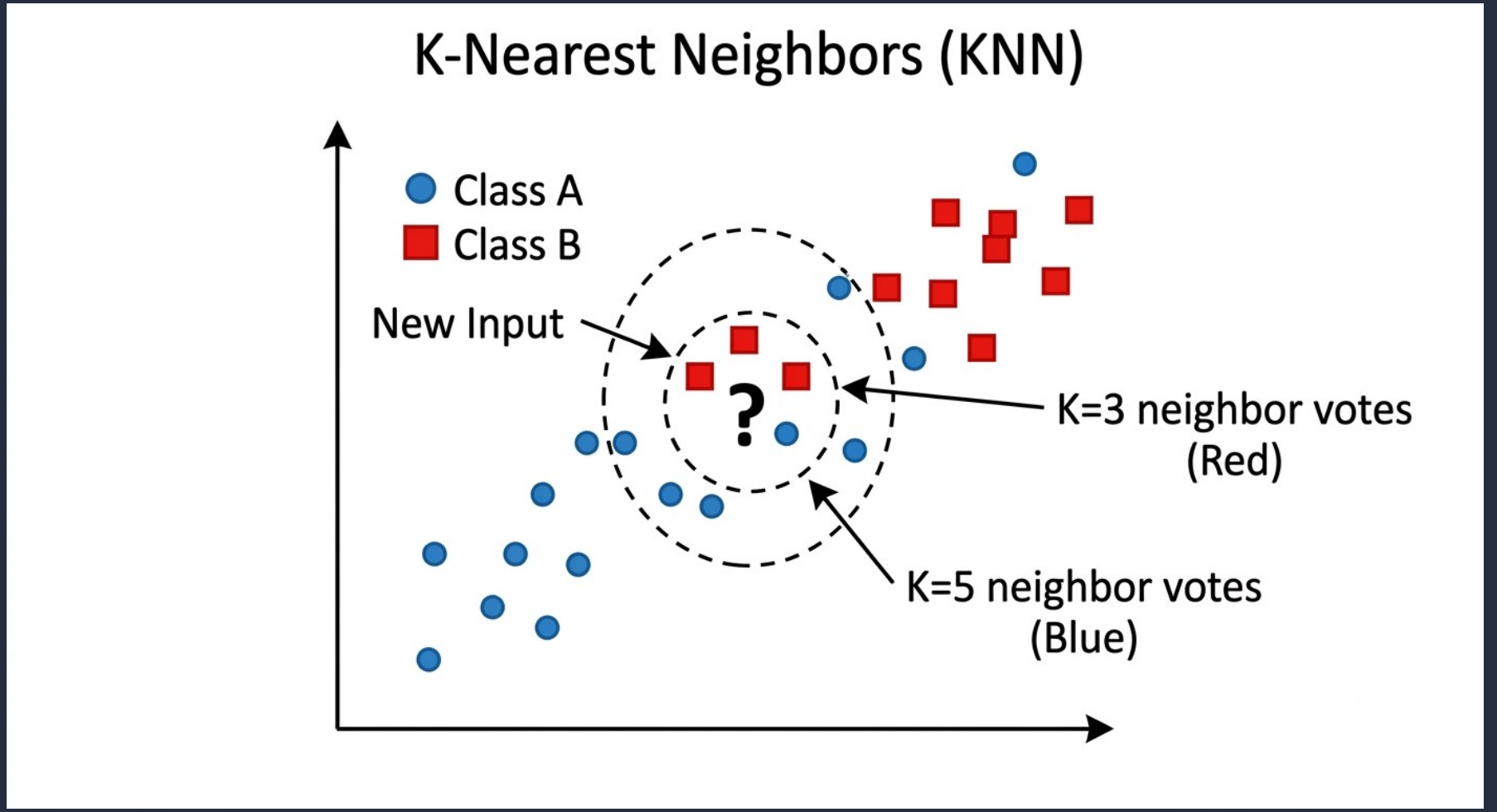
3.4 – Classical ML Techniques - Regression

- Curve Fitting
- Minimize the Errors (Residuals, dotted lines)



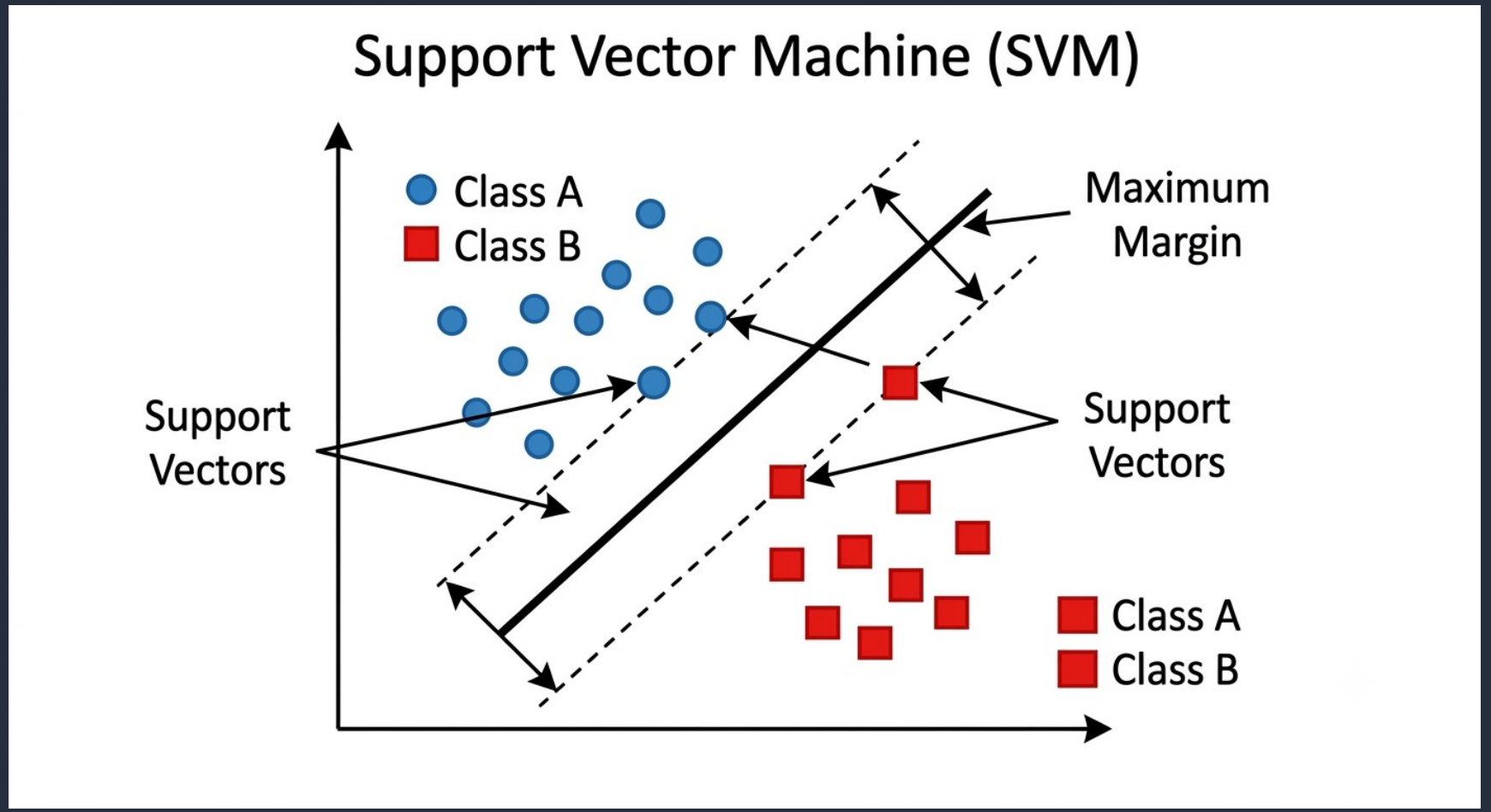
3.4 – Classical ML Techniques - KNN

- The inner dashed circle shows classification for K=3 (the three nearest neighbors): It finds 2 red and 1 blue, classifying the point as Red
- The outer dashed circle shows K=5: It finds 2 red and 3 blue, classifying the point as Blue. This highlights how sensitive KNN is to the choice of K



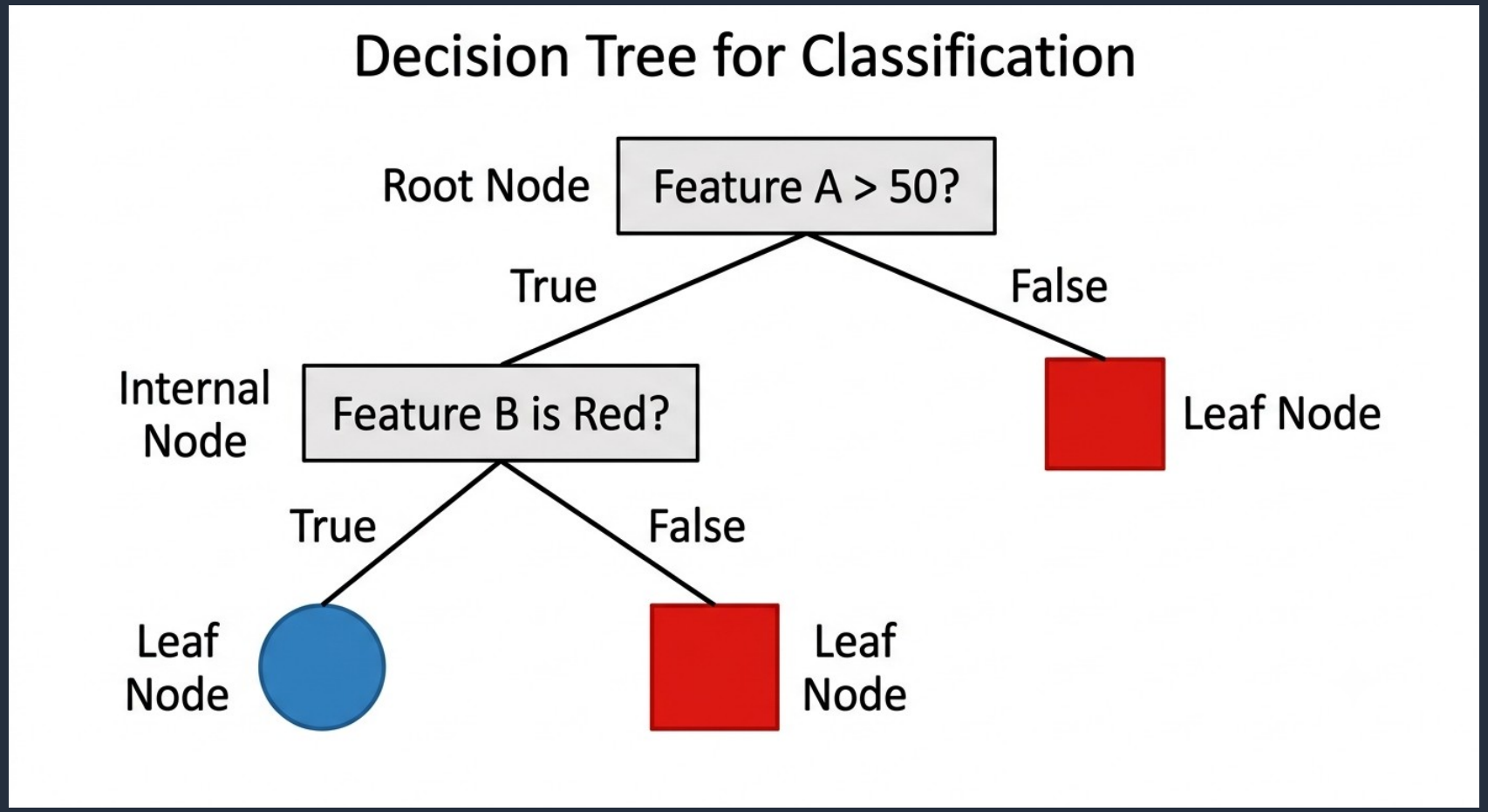
3.4 – Classical ML Techniques - SVM

- Two groups of points (blue vs. red) are separated by a solid black line (the decision boundary, or hyperplane).
- Two dashed parallel lines create a boundary 'margin.' The points that touch these dashed lines (labeled 'Support Vectors') are the critical data points that define the margin.
- SVM works by maximizing the width of this margin, providing the most robust separation.



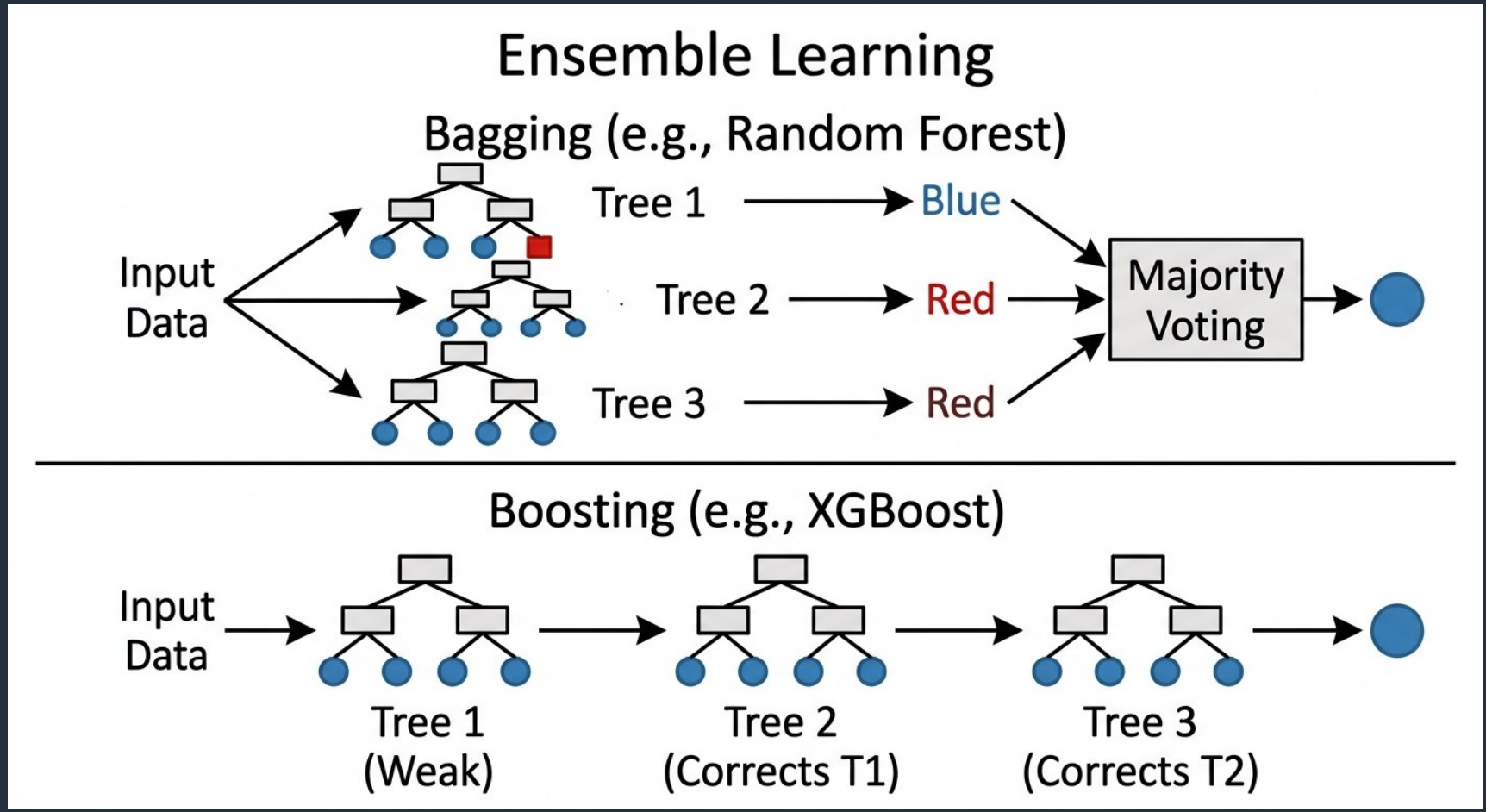
3.4 – Classical ML Techniques - Decision Trees

- The tree starts at a 'Root Node' making the first logical split (e.g., 'Feature A > 50?').
- If True, it follows the left branch; if False, the right.
- The data flows through internal 'nodes' (decisions) until it reaches a 'Leaf Node,' which provides the final classification (Red/Blue)



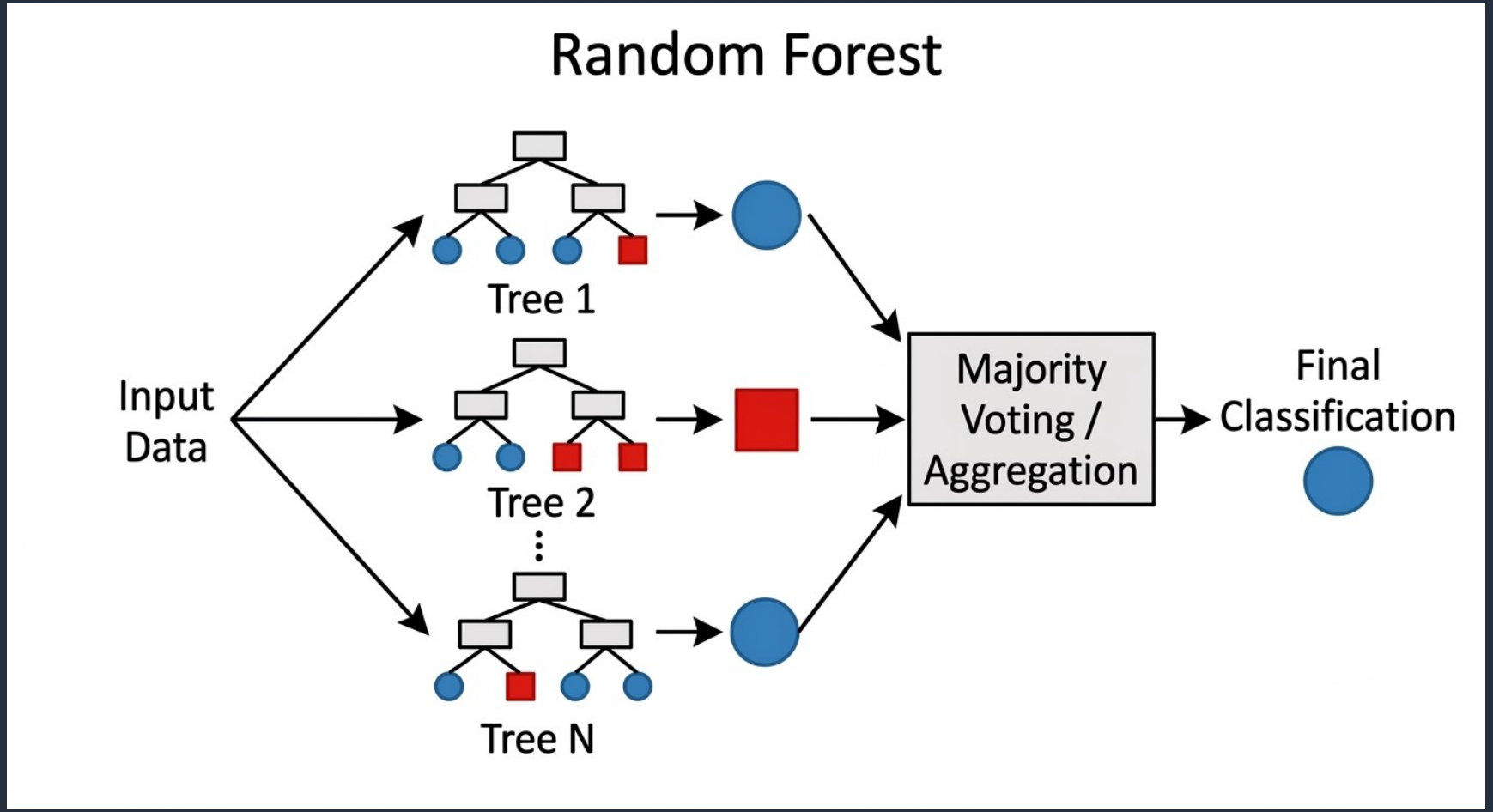
3.4 – Classical ML Techniques – Ensemble Learning

- Bagging (Top): Multiple models (e.g., small trees) run in parallel (Tree 1, 2, 3). They are independent, and their final predictions are combined via 'Majority Voting'.
- Boosting (Bottom): Models run sequentially (Tree 1 -> 2 -> 3). Tree 2 is trained to correct the errors of Tree 1, and Tree 3 corrects the combined errors of 1 and 2.



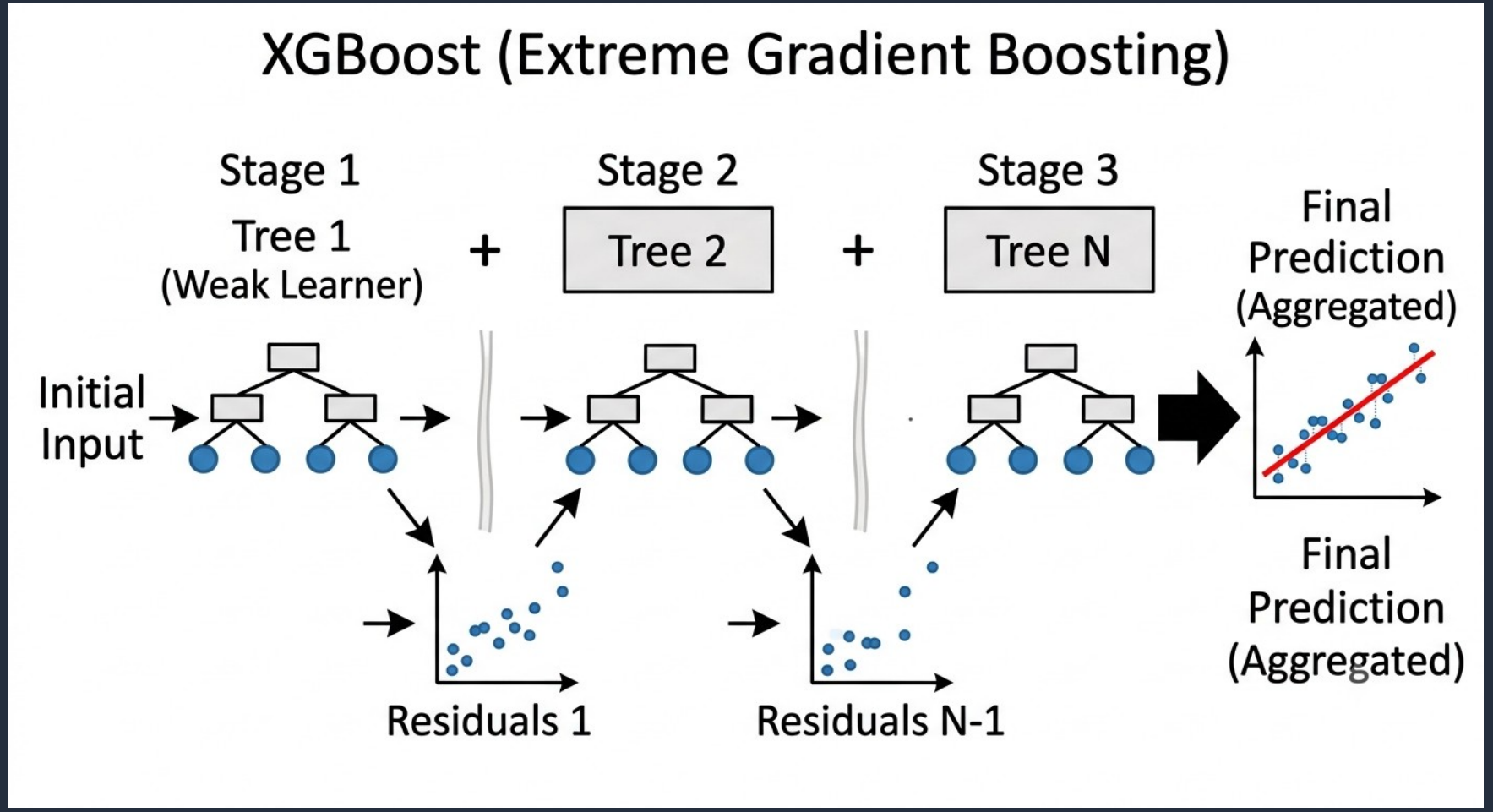
3.4 – Classical ML Techniques – Ensemble Learning - Random Forest

- Multiple small decision trees ('Tree 1' through 'Tree N') are trained independently on random subsets of the data.
- When new input data is received, each tree makes a prediction (e.g., Tree 1 predicts Blue, Tree 2 predicts Red, Tree N predicts Blue).
- These predictions feed into a central 'Majority Voting' box. The final model prediction is the result of the vote (in this case, Blue).



3.4 – Classical ML Techniques – Ensemble Learning - XGBoost

- Tree 1 is trained on the data and makes predictions. The 'Residuals 1' scatter plot shows the initial errors.
- Tree 2 is then trained specifically to predict those residuals. The additive nature is shown ('+') as the combination of Tree 1 and Tree 2 reduces the error (fewer residuals in 'Residuals 2').
- his process repeats for N trees, sequentially minimizing the error until a highly accurate 'Final Prediction' is achieved.

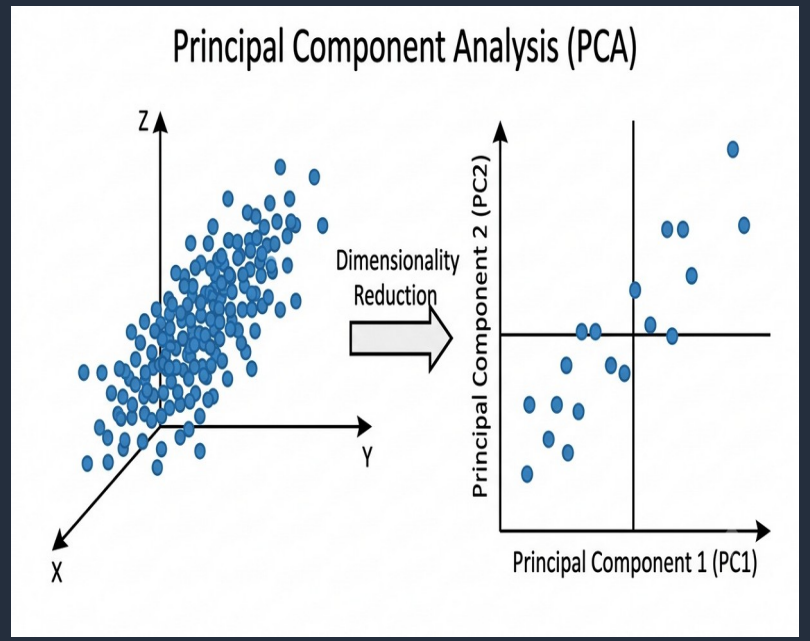


3.4 – Classical ML Techniques – Dimensionality Reduction - PCA

- Suppose that we have multi-dimensional X (input).
- We want to eliminate some of them(columns)
- We use Singular Value Decomposition linear algebra technique to select which columns (features, X[i]) represents best the data.

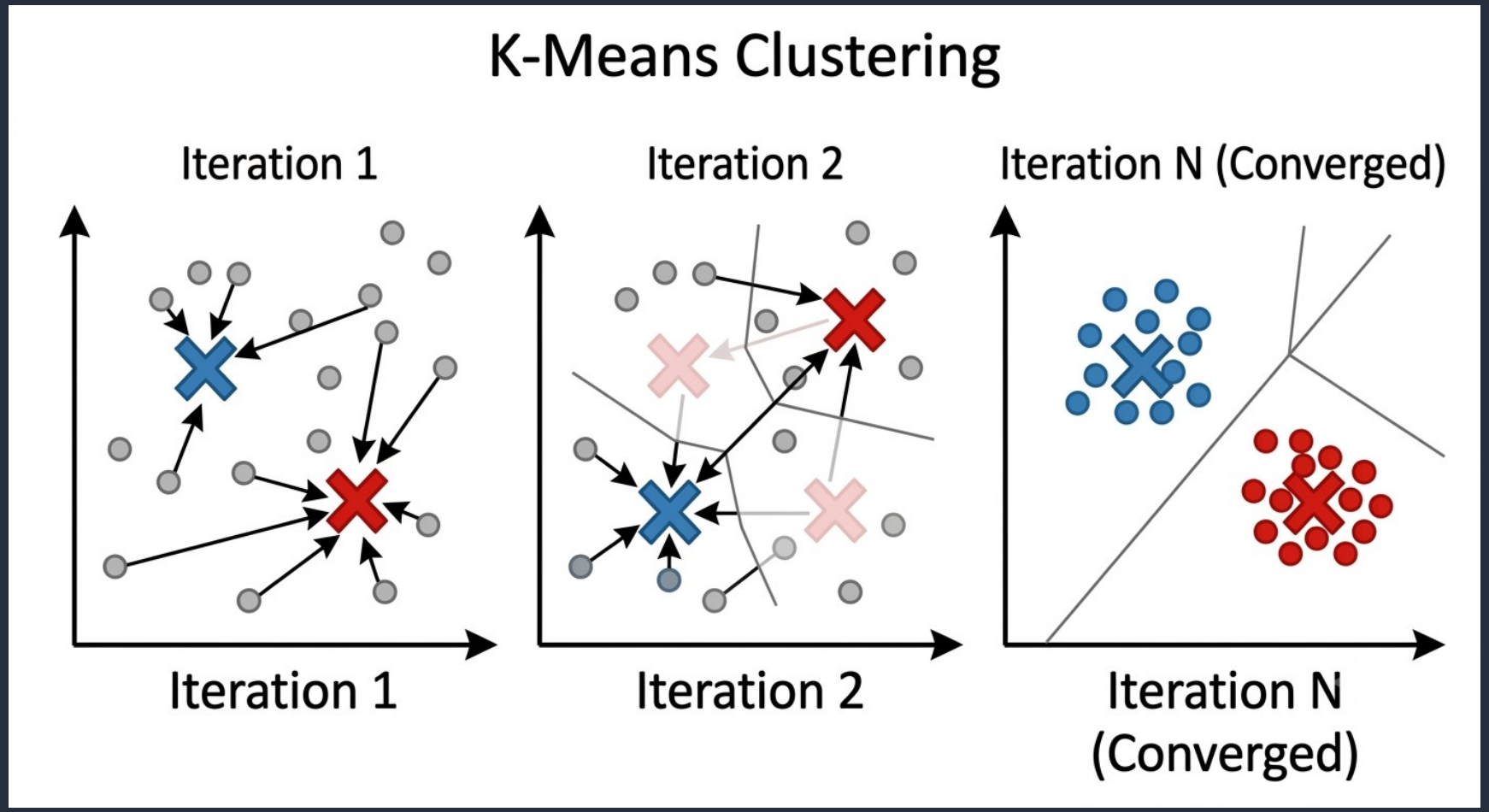
X (input)			Y **(labels)**	
X1	X2	X3	Y1	Y2
1	2	3	4	5
11	12	13	14	15
21	22	23	24	25
31	32	33	34	35
41	42	43	44	45

Select one or more columns that represent best the data.



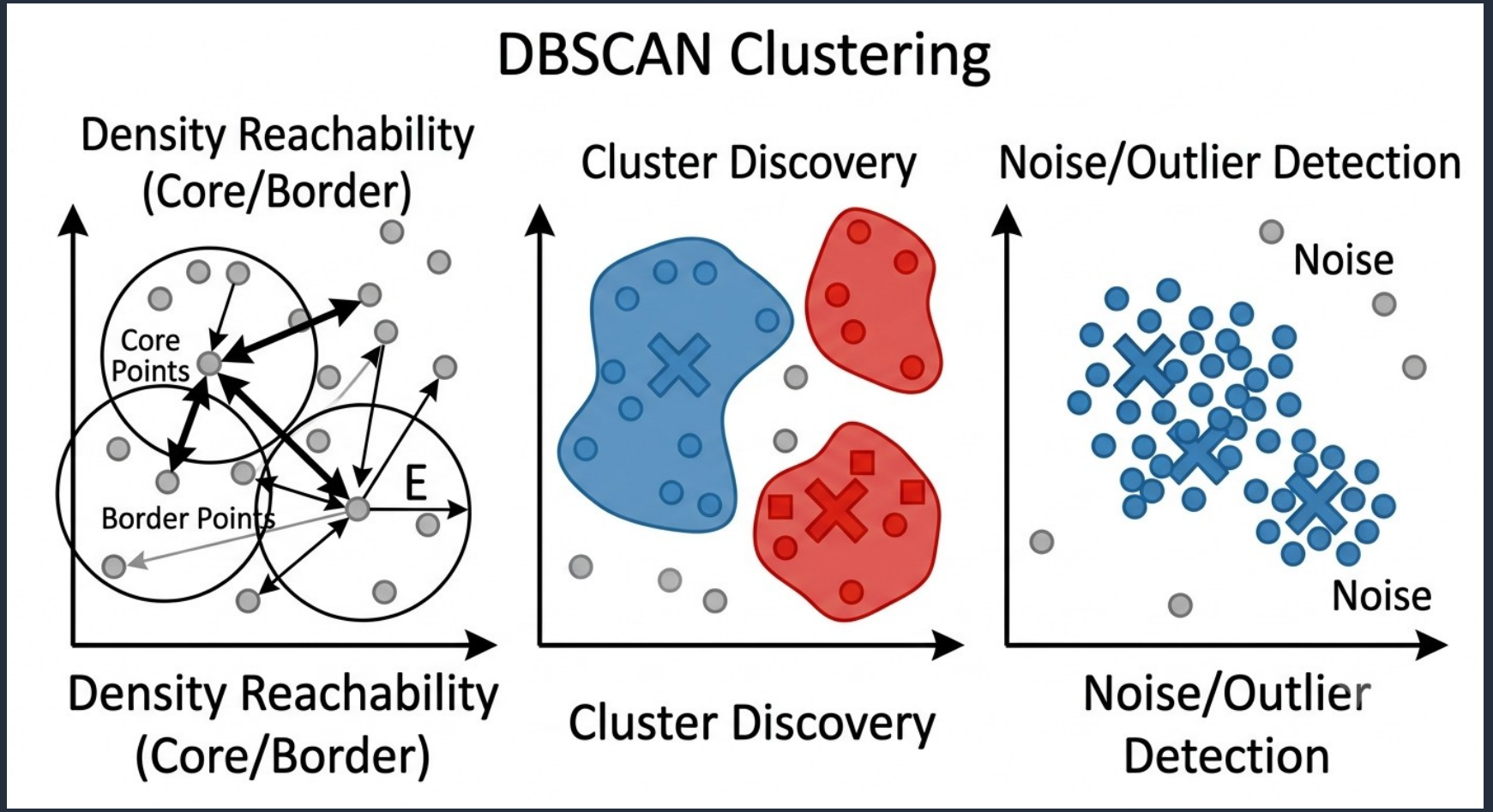
3.4 – Classical ML Techniques – Clustering - KMeans

- Iteration 1: K=2, Two centroids (X) are randomly placed among unlabeled (grey) data. Points are assigned to the closest X, forming initial, messy clusters (faint red/blue boundaries).
- Iteration 2: The centroids move to the center of their assigned points. The points are re-assigned, and the clusters start to define themselves.
- Iteration N (Converged): The centroids have stopped moving. The points are clearly separated into stable Red and Blue clusters (re-using the established class colors from Image 1), with optimized Voronoi-like boundaries..



3.4 – Classical ML Techniques – Clustering - KMeans

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm that finds clusters based on the density of data points. It is excellent at finding arbitrary shapes and identifying noise (outliers)..



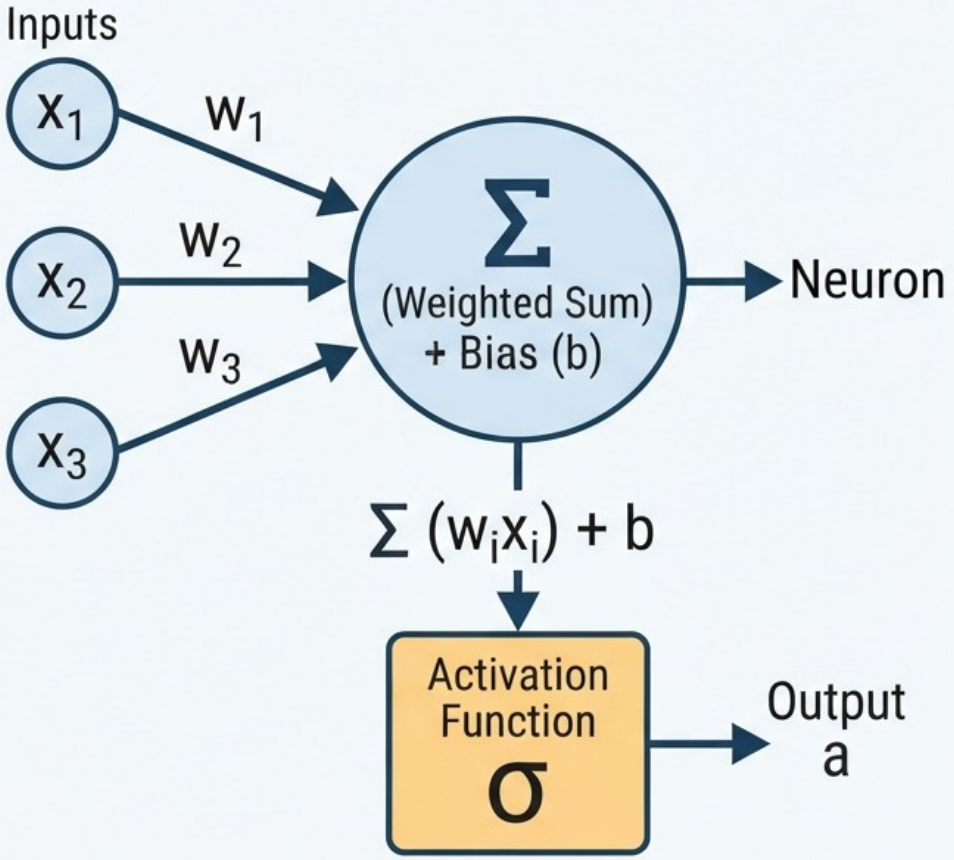
4 - Neural Networks Intuition

1. *Neurons, layers, activations (ReLU, softmax conceptually)*
2. *Why deep learning works better for perception/language*
3. *Classical DL Techniques : CNN, RNN, LSTM, GNN, RBM, VAE, GAN, KAN, Transformers, Diffusion*
4. *Bias/Vairance problem, Regularization and smooth learning*
5. *Preventing Overfitting (Memorization) : Early Stop, Dropout, Weight Regularization, Data Augmentation (**Very important), Simplify Model, Batch Normalization*
6. *Data Augmentation*
7. *Preventing Vanishing Gradient : Residuals (Skip Connections), Use RELU/LaekyRELU Activation Fn., Batch Normalization*
8. *Cross Entropy, Learning Rate Momentum*
9. *Transfer Learning*
10. *Problem Solving Using Neural Networks*

4.1 – Neurons, Layers, Activations

NEURAL NETWORK NEURON: STRUCTURE & MATHEMATICAL FOUNDATION

1. Neuron Structure & Function



2. Matrix Multiplication Representation

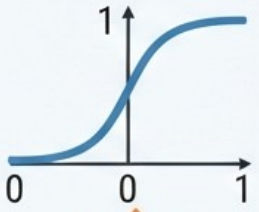
$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \times \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}^T + (b) = z$$

$$z = \mathbf{w}^T\mathbf{x} + b$$

3. Activation Functions

Sigmoid

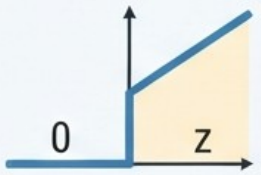
$$\sigma(z) = \frac{1}{1+e^{-z}}$$



z<0

ReLU

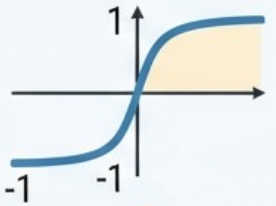
$$f(z) = \max(0,z)$$



z<0

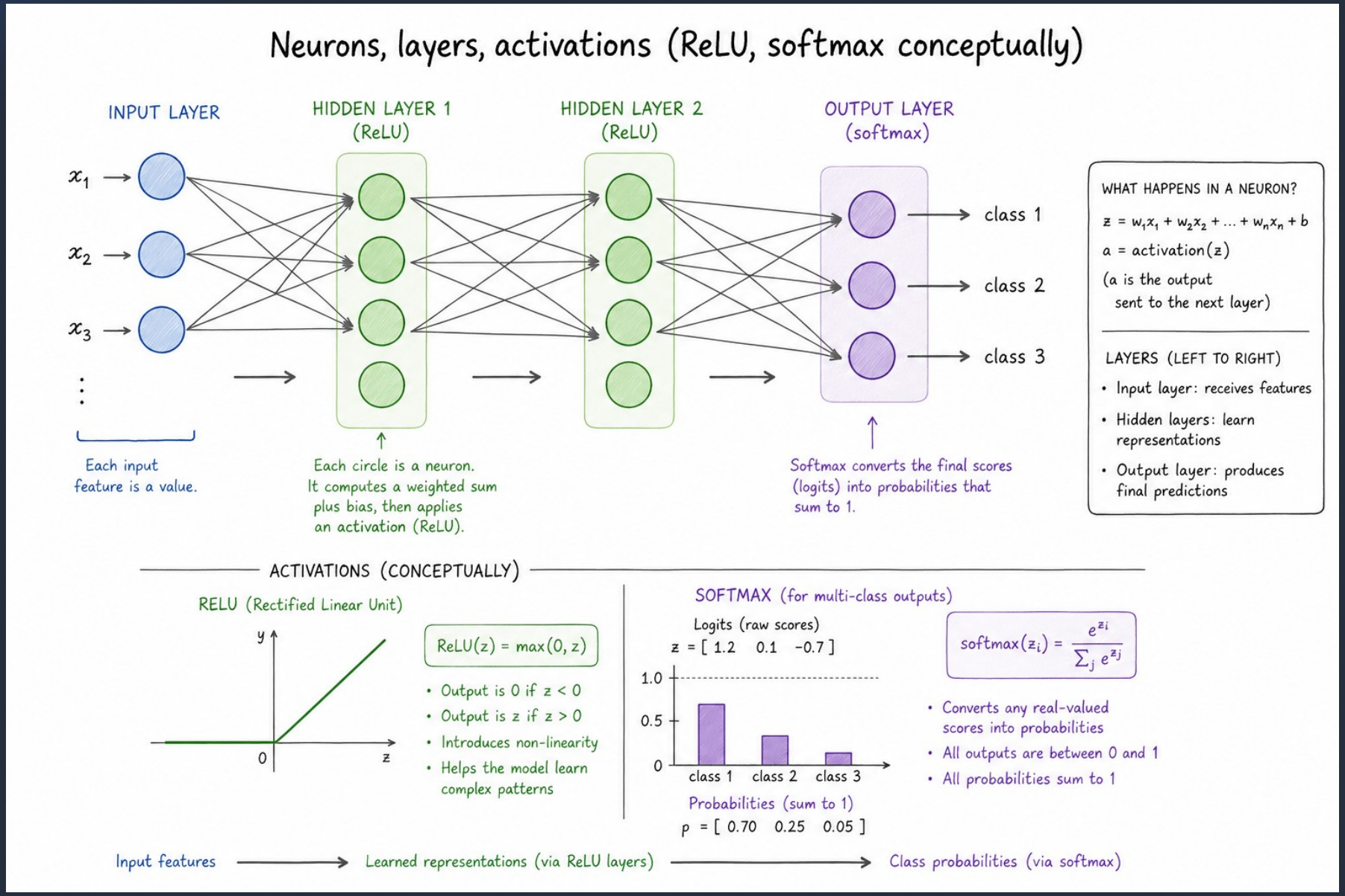
Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



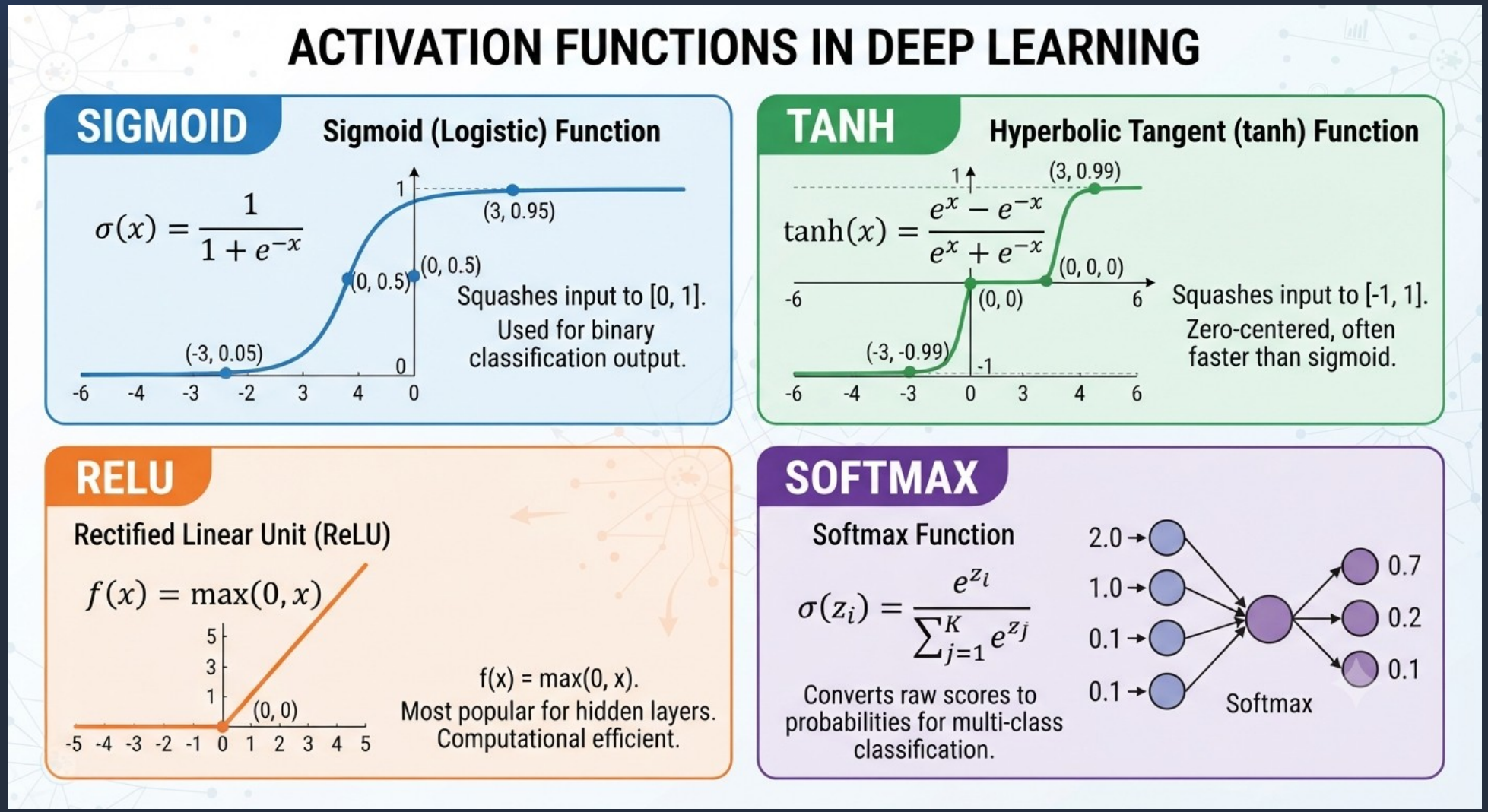
shape

4.1 – Neurons, Layers, Activations

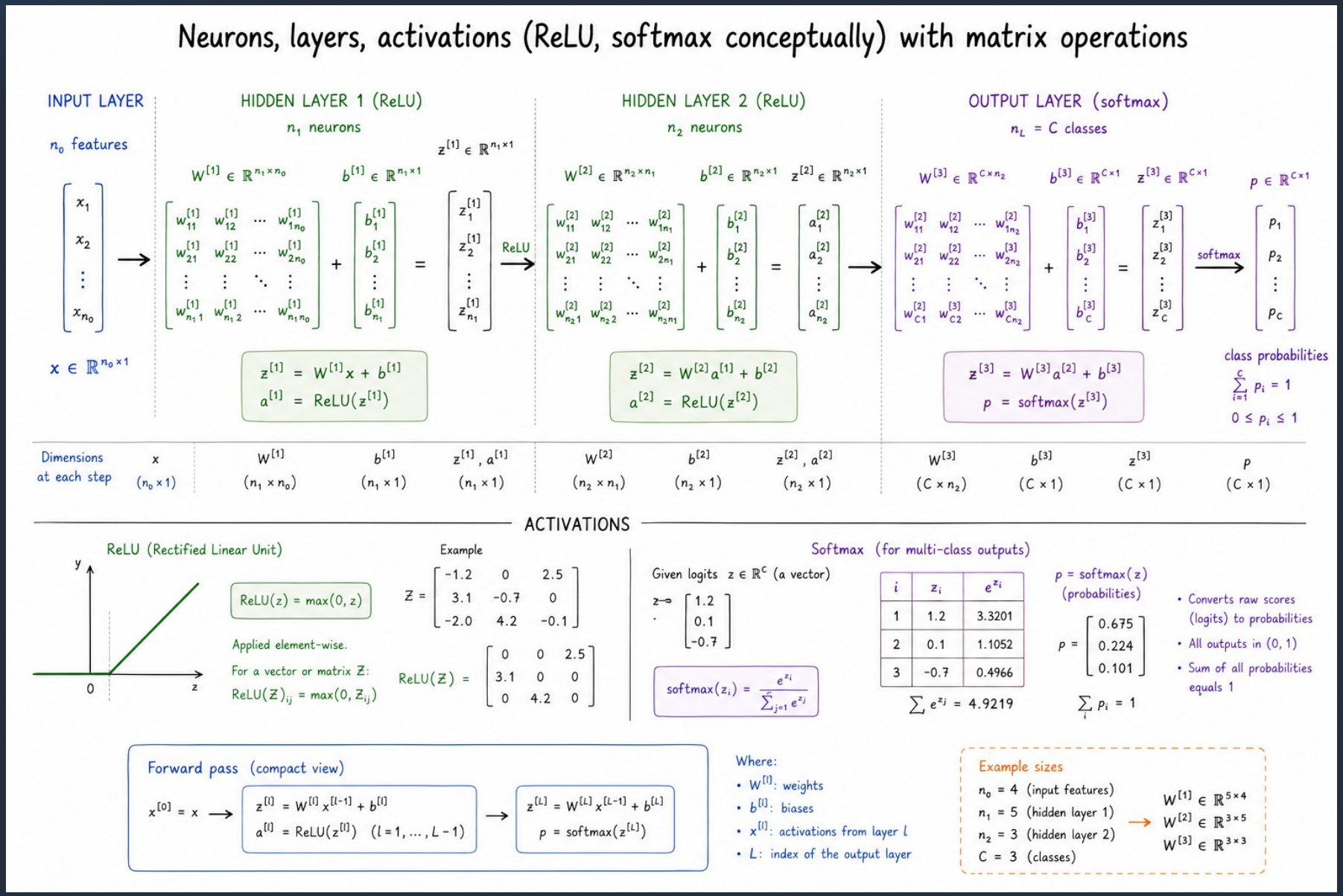


4.1 – Neurons, Layers, Activations

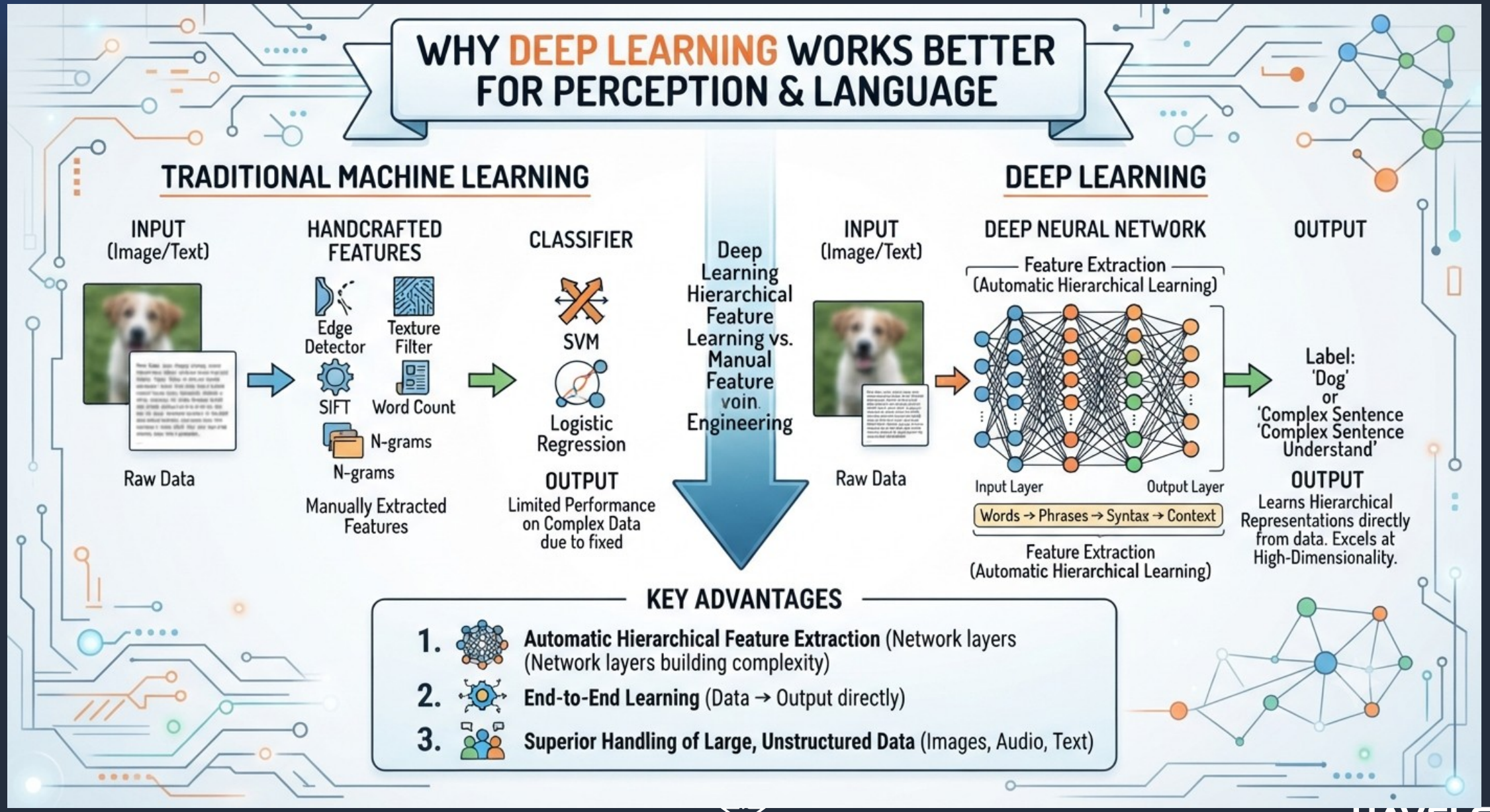
TANH USED IN SIGNAL GENERATOR NETWORKS



4.1 – Neurons, Layers, Activations



4.2 – Why Deep Learning Works Better For Perception/Language



4.3 – Classical DL Techniques

1. *Fully Connected*
2. *CNN*
3. *RNN*
4. *LSTM*
5. *GCN*
6. *RBM*
7. *VAE*
8. *GAN*
9. *KAN*
10. *Transformers*
11. *Diffusion*

4.3.1 – Classical DL Techniques – Fully Connected

UNDERSTANDING A FULLY CONNECTED NEURAL NETWORK:
ARCHITECTURE, PROPERTIES, & MATRIX OPERATIONS

PROPERTIES

1 ALL-TO-ALL CONNECTIONS

Every neuron in one layer connects to every neuron in the next.

2 LAYER TYPES

- Input
- Hidden (features)
- Output (predictions)

3 3. PARAMETERS

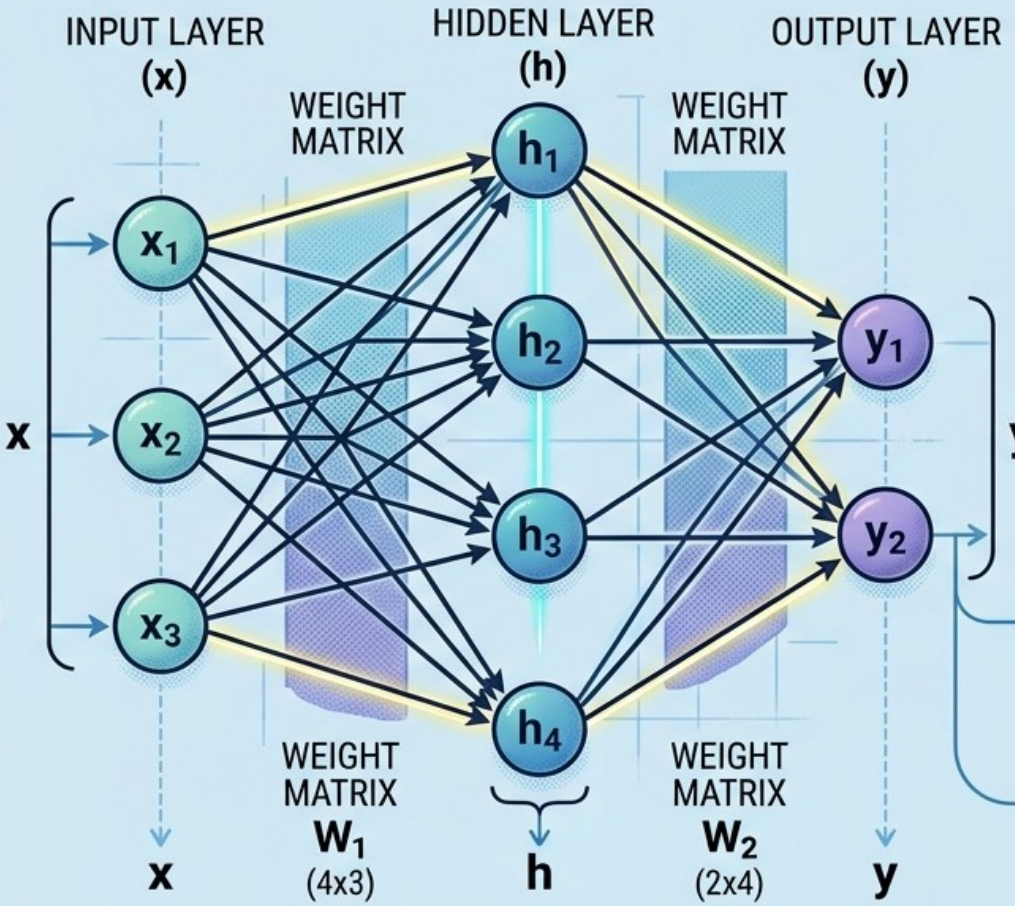
- Weights (strength of connection)
- Biases (activation threshold)

4 4. NO SPATIAL STRUCTURE

Does not preserve input spatial relationships.

5 5. DENSE CONNECTIONS

Computationally intensive for large inputs.



MATRIX OPERATIONS
(FORWARD PASS)

$$\mathbf{h} = f_1(\mathbf{x} \cdot \mathbf{W}_1^T + b_1)$$

$$\mathbf{y} = f_2(\mathbf{h} \cdot \mathbf{W}_2^T + b_2)$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \times \begin{bmatrix} x_1 & x_2 & x_3 \\ \square & \square & \square \\ \square & \square & \square \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix} = \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \\ b_{14} \end{bmatrix}$$

$\mathbf{b_1}$

(1x3) (3x4) (1x4) (1x4)

$$\mathbf{y} = f_2(\mathbf{h} \cdot \mathbf{W}_2^T + b_2)$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \times \begin{bmatrix} x_1 & x_2 & x_3 \\ \square & \square & \square \\ \square & \square & \square \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{14} \end{bmatrix} = \begin{bmatrix} \square & \square \\ \square & \square \\ \square & \square \end{bmatrix}$$

$\mathbf{b_1}$

(1x3) (3x4) (1x4) (1x4)

ACTIVATION FUNCTION

$$g(z) = f(z)$$

ReLU or Sigmoid

4.3.2 – Classical DL Techniques – CNN

UNDERSTANDING A CONVOLUTIONAL NEURAL NETWORK (CNN): ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS.

CNN PROPERTIES & CONCEPTS

1 RECEPTIVE FIELD & LOCAL CONNECTIVITY
Visual filter over on input (auot to obtain innit).

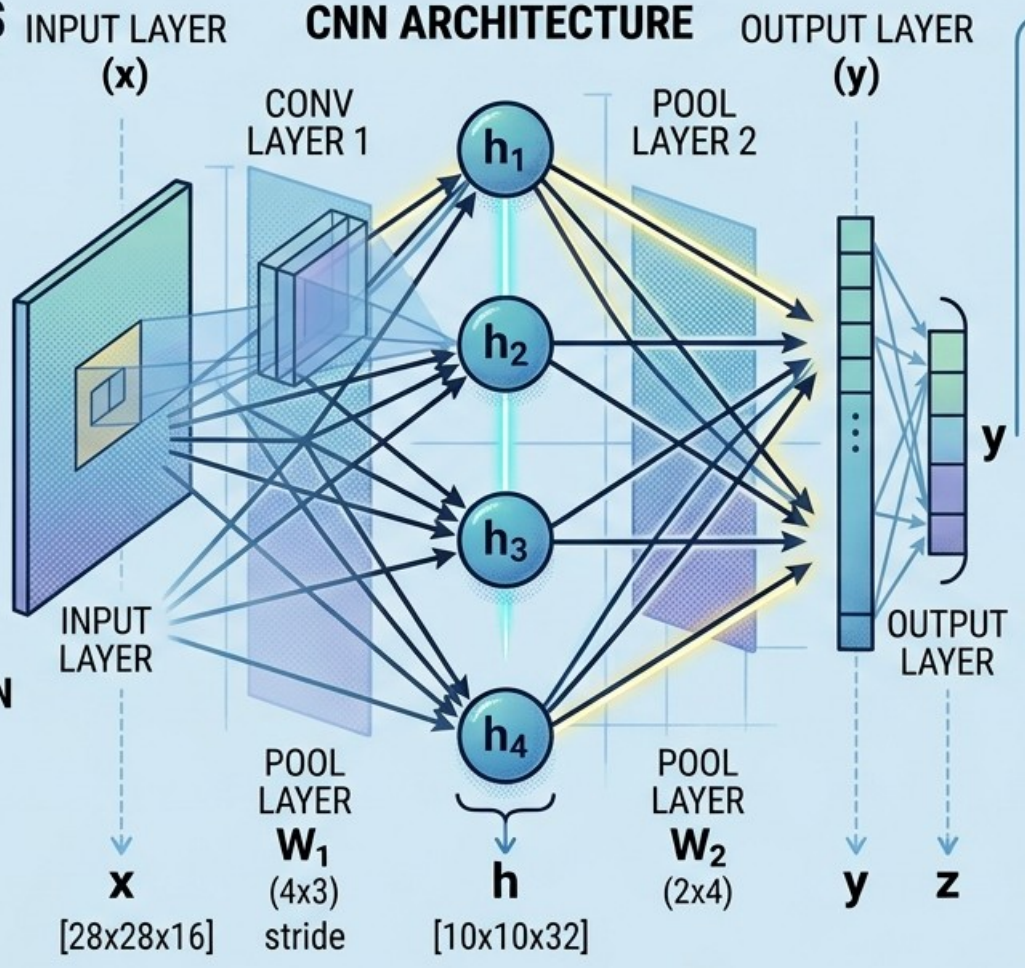
2 WEIGHT SHARING
- Filters with shared
- Filters with d shared
- Filters shared parameters

3 3. TRANSLATION INVARIANCE
- Weights (strength of connection)
- Biases (activation threshold)

4 4. DIMENSIONALITY REDUCTION
Does not preserve input (Pooling/Stride)

5 5. NON-LINEARITY
Computationasive (ReLU, etc.)

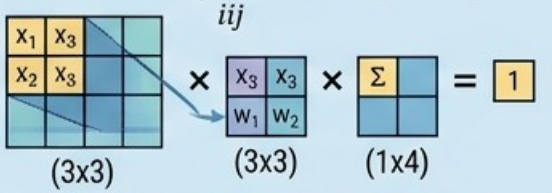
CNN ARCHITECTURE



MATRIX OPERATIONS (FOR CONVOLUTION)

CONVOLUTION → FEATURE MAP

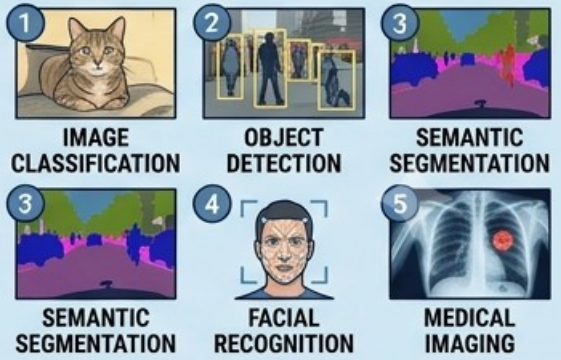
$$y_{ij} = f\left(\sum_{iij} x_{sub} \cdot w\right) + b$$



POOLING OPERATIONS

MAX-POOLING → max → max value

APPLICATION AREAS



4.3.3 – Classical DL Techniques – RNN

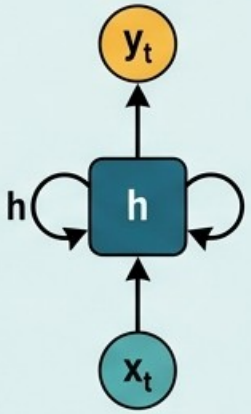
UNDERSTANDING A RECURRENT NEURAL NETWORK (RNN):
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

RNN PROPERTIES

1. **SEQUENTIAL DATA PROCESSING**
   Operates on data over time.
2. **INTERNAL MEMORY**
   h_t stores information from previous steps
3. **VARIABLE-LENGTH SEQUENCES**
   Can handle inputs of varying lengths.
4. **PARAMETER SHARING**
   The same **U, V, W, b** are used for all steps.
5. **TYPES** (examples)

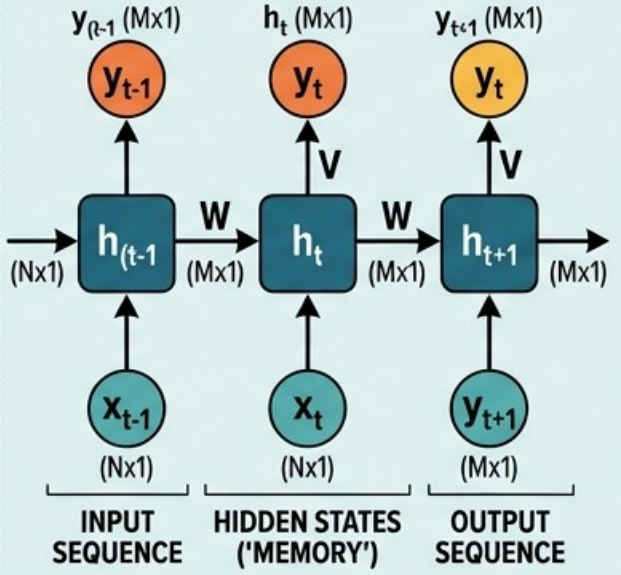
RNN ARCHITECTURE (FOLDED & UNFOLDED)

Folded View



$h_t = f(U \cdot x_t + W \cdot h_{(t-1)} + b)$

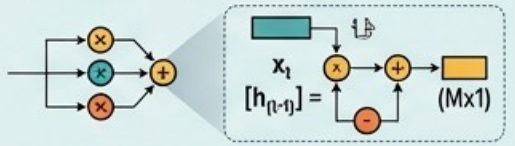
Unfolded View (3 time steps)



MATRIX OPERATIONS (FORWARD PASS)

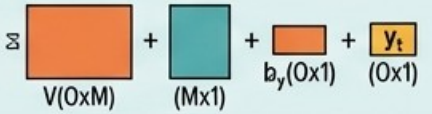
1. **Hidden State Update**

$$[h_t] = f([U]\cdot[x_t] + [W]\cdot[h_{(t-1)}] + [b_h])$$



2. **Output Calculation**

$$[y_t] = g([V]\cdot[h_t] + [b_y])$$



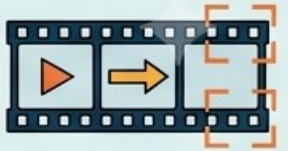
RNN APPLICATION AREAS

1. MACHINE TRANSLATION

2. SPEECH RECOGNITION

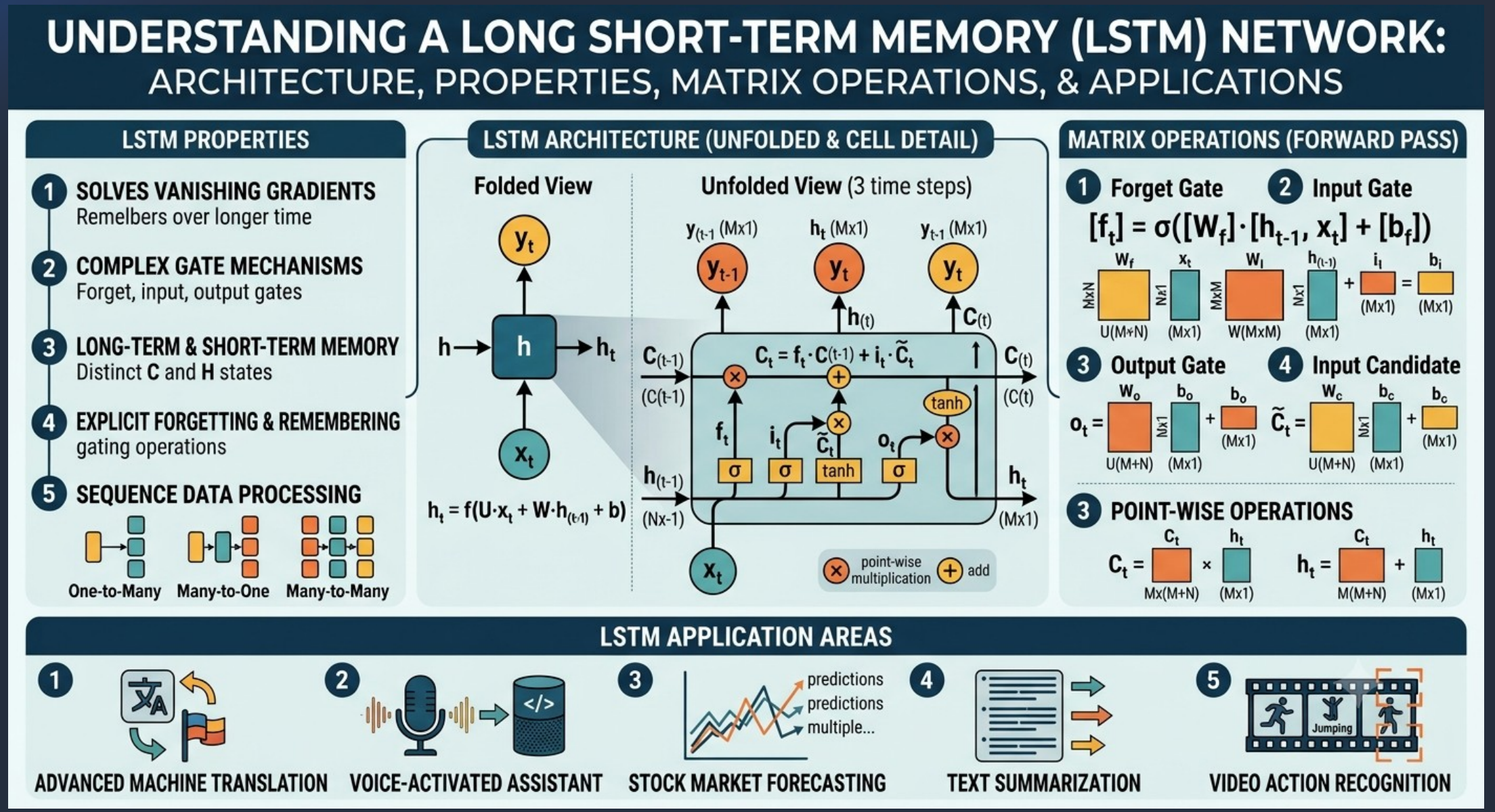
3. TIME-SERIES FORECASTING

4. SENTIMENT ANALYSIS

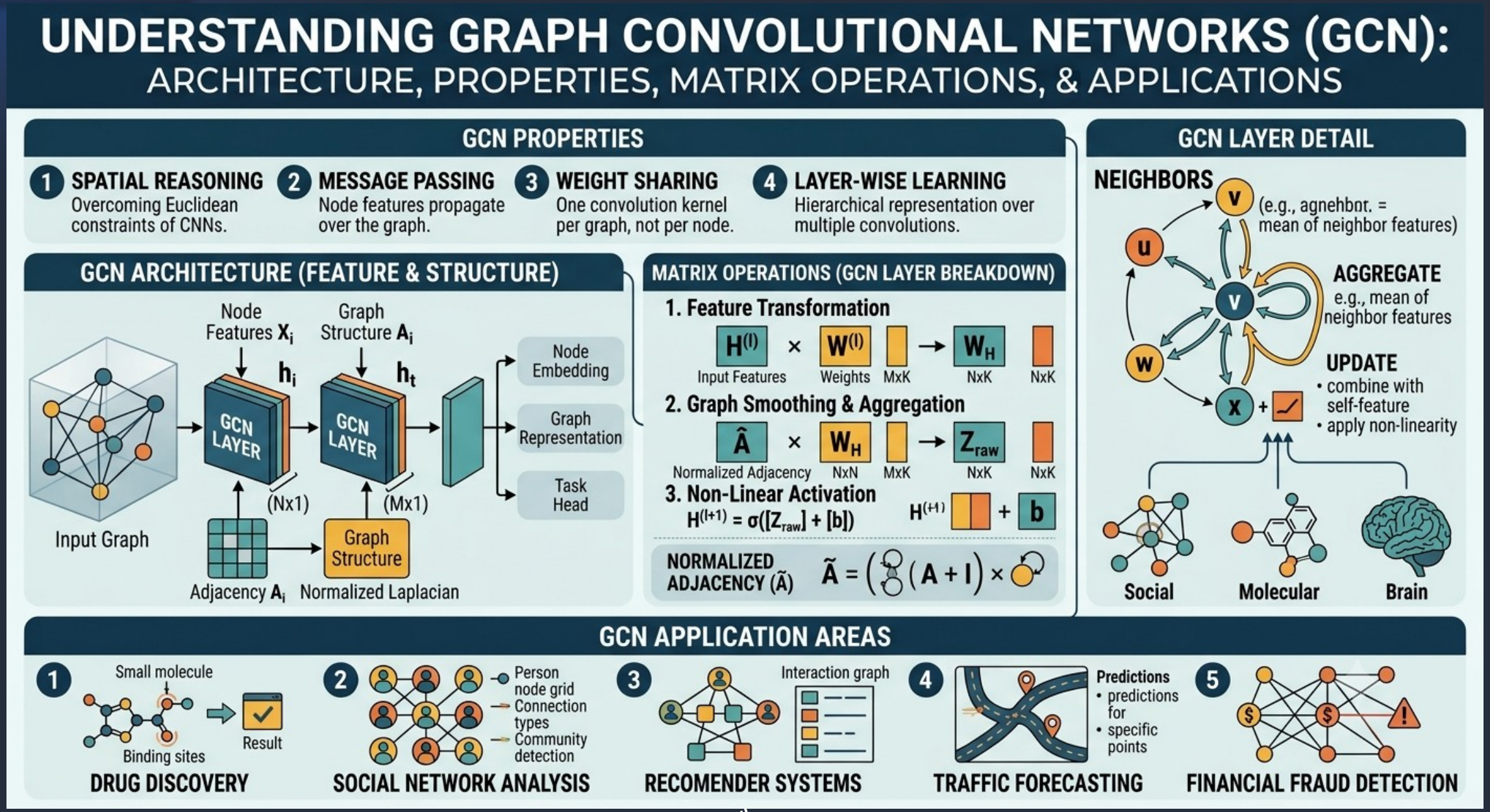


5. VIDEO ANALYSIS

4.3.4 – Classical DL Techniques – LSTM



4.3.5 – Classical DL Techniques – GCN



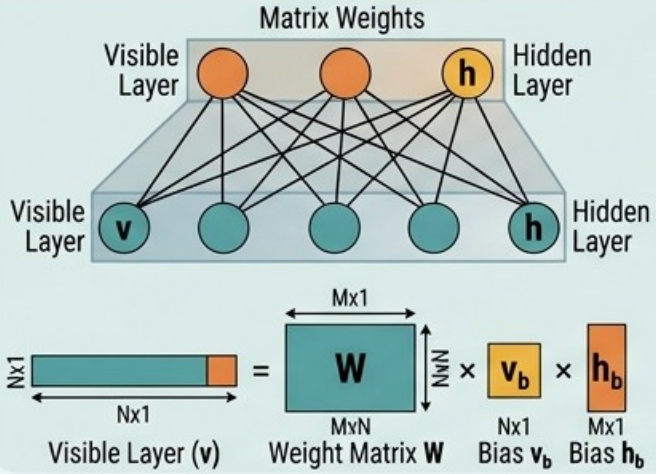
4.3.6 – Classical DL Techniques – RBM

UNDERSTANDING RESTRICTED BOLTZMANN MACHINES (RBM):
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

RBM PROPERTIES

1. **UNSUPERVISED LEARNING** — Training data is unlabeled (e.g., unlabeled image grid)
2. **STOCHASTIC NETWORK** — Visible and hidden units are random variables
3. **NO INTRA-LAYER CONNECTIONS** — Connections only between visible and hidden layers
4. **ENERGY-BASED MODEL** — Lower energy states are more probable
5. **CONTRASTIVE DIVERGENCE** — The standard learning algorithm

RBM ARCHITECTURE



NODE DETAIL

$$P(h_j \mid v) = \sigma\left(\sum_i v_i W_{ij} + h_{bj}\right)$$

NODE DETAIL

$$P(h_j \mid v) = \sigma\left(\sum_i v_i W_{ij} + h_{bj}\right)$$

MATRIX OPERATIONS (FORWARD & BACKWARD PASS)

1. Visible to Hidden Pass

$$[v]^T\ (1\text{xN}) \times \underbrace{[W]^T}_{\text{NxM}} + \underbrace{[h_b]^T}_{\text{1xM}} \rightarrow [p(h)]^T\ (1\text{xM})$$

Sigmoid applied element-wise

2. Hidden to Visible Pass

$$[h]^T\ (1\text{xM}) \times \underbrace{W}_{\text{MxN}} + \underbrace{[v_b]^T}_{\text{1xN}} \rightarrow [p(v)]^T\ (1\text{xN})$$

Sigmoid applied element-wise

3. Weight Update (CD-1)

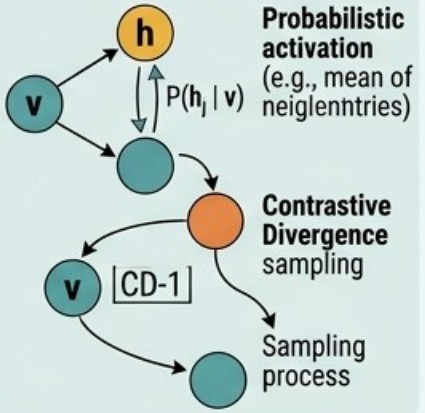
$$\Delta W = \eta * \langle v^0 h^0 \rangle_{data} - \langle v^1 h^1 \rangle_{model})$$

Data Model

4. Energy Function

$$E(v, h) = -\sum_i v_i v_{bi} - \sum_j h_j h_{bj} - \sum_i \sum_j v_i h_j W_{ij}$$

RBM STOCHASTICITY & LEARNING DETAIL



RBM APPLICATION AREAS

1. **DIMENSIONALITY REDUCTION** — High-dim input data → Compress to low-dim features
2. **FEATURE LEARNING** — Learning higher-level features from raw data
3. **RECOMMANDER SYSTEMS** — Matrix factorization
4. **GENERATIVE MODELLING** — Generating new data samples (e.g., new samples)
5. **PRE-TRAINING FOR DEEP NETWORKS** — Initialization for deep belief networks

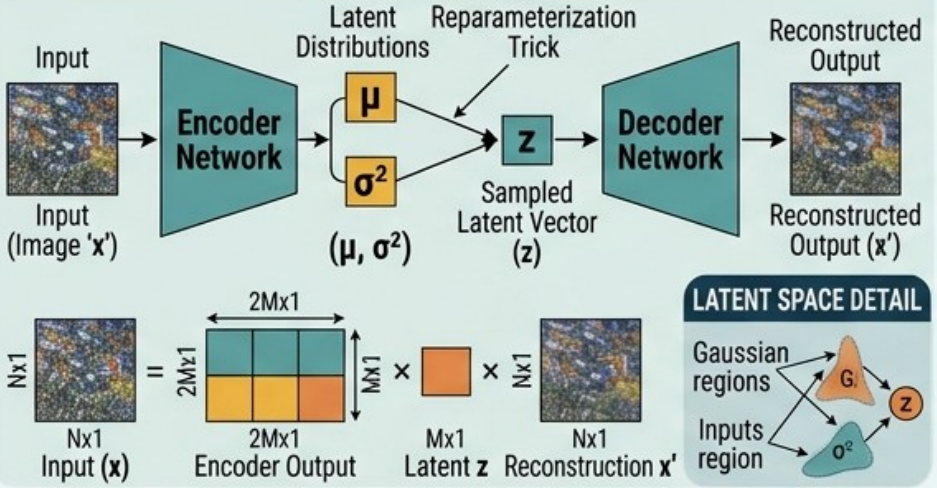
4.3.7 – Classical DL Techniques – VAE

UNDERSTANDING VARIATIONAL AUTOENCODERS (VAE):
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

VAE PROPERTIES

1. **PROBABILISTIC LATENT SPACE** — Encodes inputs as probability distributions (e.g., Gaussian bell)
2. **GENERATIVE MODELLING** — Can generate new data samples from latent space
3. **REGULARIZED LATENT SPACE** — The latent distribution is constrained be close to a prior (e.g., Normal N(0,I))
4. **UNSUPERVISED LEARNING** — Trained on unlabeled data to reconstruct and generate
5. **REPARAMETERIZATION TRICK** — Enables backpropagation through a stochastic sampling process

VAE ARCHITECTURE



MATRIX OPERATIONS (FORWARD PASS)

1. Encoder Forward Pass

$$[x]^T\ (1\times N) \times \underbrace{[W_{enc}]^T}_{N\times 2M} + \underbrace{[b_{enc}]}_{1\times 2M} \rightarrow [p(\mu, \log\sigma^2)]^T\ (1\times 2M)$$

2. Reparameterization Trick $[z] = [\mu] + [\sigma] \odot [\varepsilon]$

$$[z]^T\ (1\times M) \times \underbrace{[W_{dec}]^T}_{M\times N} + \underbrace{[b_{dec}]}_{1\times N} \rightarrow [p(x'|z)]^T\ (1\times N)$$

Sigmoid or Linear activation

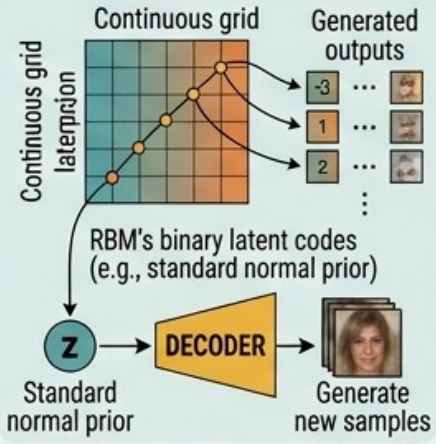
3. Decoder Forward Pass

4. Loss Function

Reconstruction Loss (e.g., MSE or Cross-Entropy) + KL Divergence Loss

$$KL(N(\mu,\sigma)||N(0,1)) = -0.5 \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

LATENT SPACE EXPLORATION & GENERATION



VAE APPLICATION AREAS

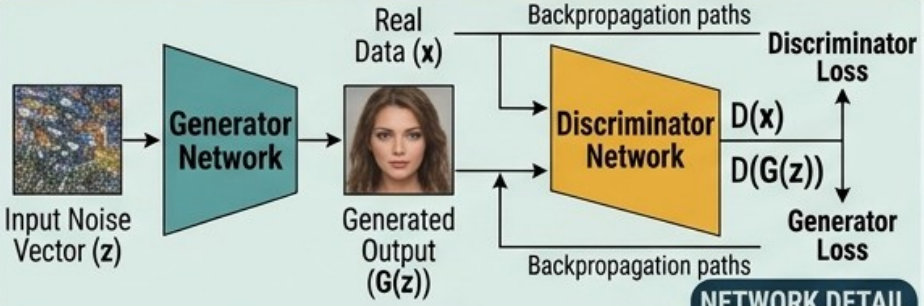
4.3.8 – Classical DL Techniques – GAN

UNDERSTANDING GENERATIVE ADVERSARIAL NETWORKS (GANs):
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

GAN PROPERTIES

1. **ADVERSARIAL TRAINING** — Competition between Generator and Discriminator
2. **GENERATIVE MODEL** — Learns to create new data samples
3. **UNSUPERVISED LEARNING** — Trained on real, unlabeled data
4. **LATENT SPACE EXPLORATION** — Traverses noise space to manipulate output features
5. **MINIMAX GAME** — Generator tries to minimize D(G(z)), Discriminator tries to maximize it

GAN ARCHITECTURE (GENERATION FLOW & LOSS)



NETWORK DETAIL

Generator: $\mathbf{z}(M\times1) \rightarrow$ Dense Layer($M\times K$) $\rightarrow [W_{g1}]^T \rightarrow z' \cdots \rightarrow G(z)$ $(N\times1)$

Discriminator: Input $(N\times1) \rightarrow$ Dense Layer($N\times L$) $\rightarrow [W_{d1}]^T \rightarrow$ Input' $\cdots \rightarrow D(x)$ (1×1 sigmoid output)

MATRIX OPERATIONS (FORWARD PASS)

1. Generator Forward Pass

$$[z]^T\ (1\times M) \times \underbrace{[W_{g1}]^T}_{M\times K} + \underbrace{[b_{g1}]}_{1\times K} \rightarrow [p(G(z))]^T\ (1\times N\times1)$$

Sigmoid or ReLU activationn

2. Discriminator Forward Pass

Real $[x]^T\ (1\times N) \times \underbrace{[W_{d1}]^T}_{N\times L} + \underbrace{[b_{d1}]}_{1\times L} \rightarrow [p(D(x))]^T\ (1\times1)$ — 1x1 sigmoid

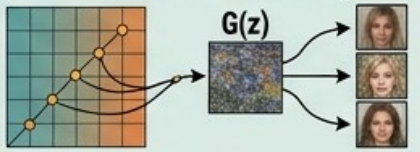
For Gener. $[G(z)]^T\ (1\times N) \times \underbrace{[W_{d1}]^T}_{N\times L} + \underbrace{[b_{d1}]}_{1\times L} \rightarrow [p(D(G(z)))]^T\ (1\times1)$ — 1x1 sigmoid

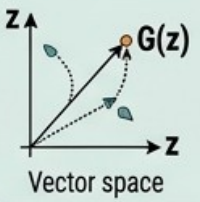
3. Minimax Loss Function

$$\mathcal{L}_{GAN} = \min_G \max_D \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

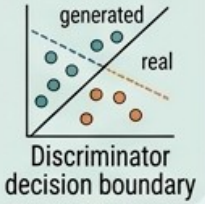
GAN STOCHASTICITY & ADVERSARIAL DETAIL

Close-up view on a small part of latent space to illustrate how small changes in **z**



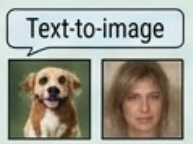


Vector space



Discriminator decision boundary

GAN APPLICATION AREAS

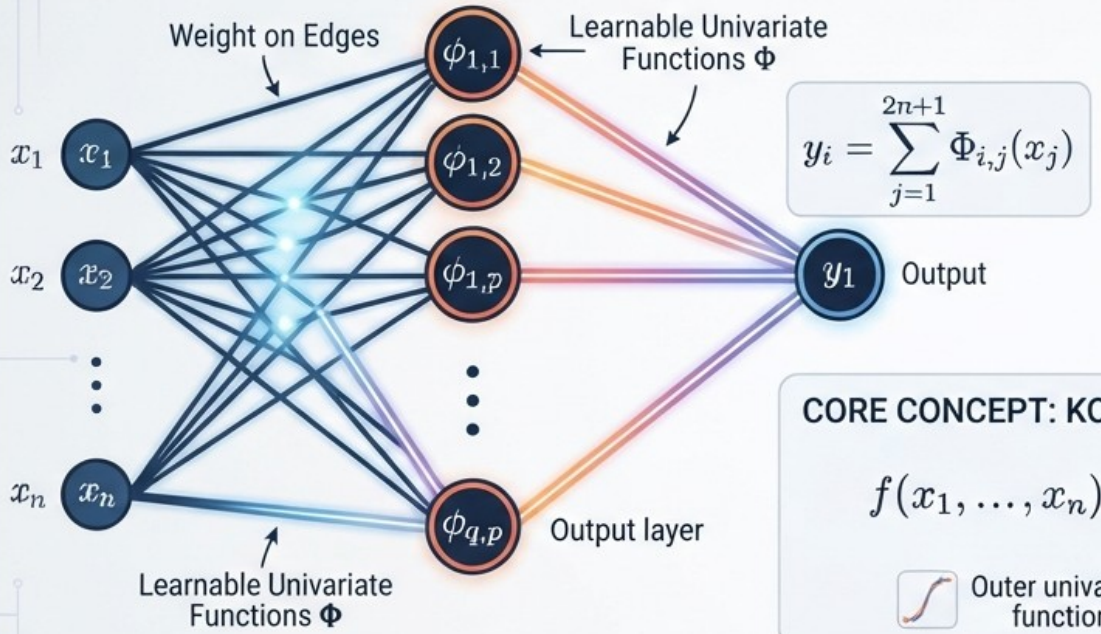
1.  Creating high-quality unique faces, landscapes, or objects (e.g., stylized generated images) — **IMAGE GENERATION & SYNTHESIS**
2.  Style transfer (e.g., converting a daytime photo to to nighttime, painting style) — **IMAGE-TO-IMAGE TRANSLATION**
3.  Creating high-resolution details from low-resolution inputs — **IMAGE SUPER-RESOLUTION**
4. 

Creating images from descriptive text prompts — **TEXT-TO-IMAGE GENERATION**
5.  Generating rare data samples for other models (e.g., rare disease scans) — **DATA AUGMENTATION**

4.3.9 – Classical DL Techniques – KAN

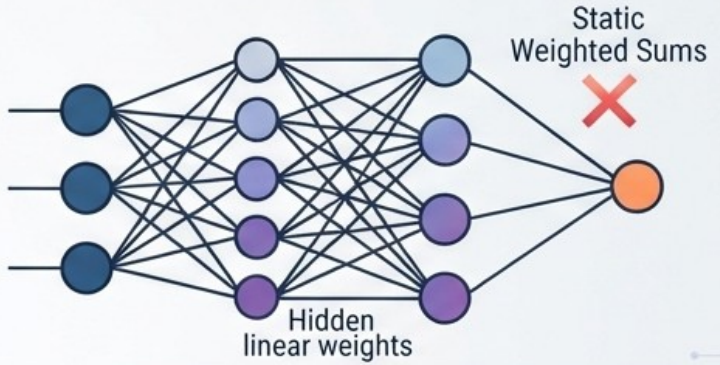
KOLMOGOROV-ARNOLD NETWORKS (KAN): FUNCTION DECOMPOSITION ARCHITECTURE

KAN ARCHITECTURE (1 Layer)



$$y_i = \sum_{j=1}^{2n+1} \Phi_{i,j}(x_j)$$

MLP ARCHITECTURE (1 Layer)



CORE CONCEPT: KOLMOGOROV-ARNOLD REPRESENTATION THEOREM

$$f(x_1, \ldots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

← number of input variables

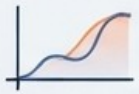
Outer univariate function

Inner univariate functions

Required number of basis functions in the hidden layer

KEY KAN CHARACTERISTICS

FUNCTION-BASED

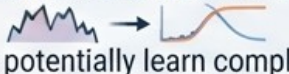


Learns arbitrary univariate functions on edges (e.g., B-splines), not fixed weights.

NO LINEAR WEIGHTS

Weights are replaced by non-linear, adaptive functions.

GREATER EXPRESSIVITY



Can potentially learn complex functions with fewer parameters than MLPs in certain tasks.

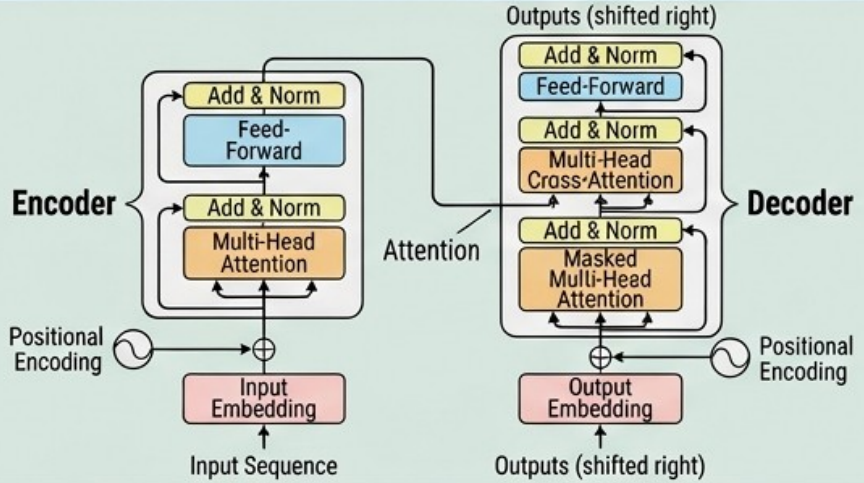
4.3.10 – Classical DL Techniques – Transformer

UNDERSTANDING TRANSFORMER NETWORKS:
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

TRANSFORMER PROPERTIES

1. **SELF-ATTENTION MECHANISM** — Relates different positions of a single sequence
2. **PARALLEL PROCESSING** — Decouples computation from sequential data, unlike RNNs
3. **GLOBAL RECEPTIVE FIELD** — Every token attends to every other token
4. **POSITIONAL ENCODING** — Introduces order information
5. **MULTI-HEAD ATTENTION** — Learns different types of relationships simultaneously

TRANSFORMER ARCHITECTURE (ENCODER-DECODER FLOW)



MATRIX OPERATIONS (MULTI-HEAD SELF-ATTENTION)

1. Linear Transformation

$$[X]^T (T \times D) \times [W_{QK}/K/V]^T (D \times D') \rightarrow [Q/K/V]^T (T \times D')$$

T(tokens)xD'(model dim per head)

2. Attention Score Calculation 3. Value Aggregation

$$[A] = \text{softmax}\left(\frac{[Q] \times [K]^T}{\sqrt{d_k}}\right) \quad [\text{Attention}_{\text{Head}}] = [A] \times [V]$$

4. Head Concatenation & Final Linear

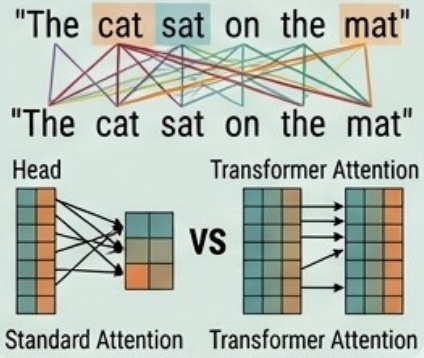
$$[\text{MultiHead}(Q,K,V)] = \text{Concat}(\text{head}_1, \text{head}_2 \ldots) \times [W_0]$$

FFN (Feed-Forward Network)

$$[X']^T (T \times D) \times [W_1]^T (D \times D_H) \rightarrow \text{ReLU} \times [W_2]^T (D_H \times D) \rightarrow \text{FFN}(X')$$

ATTENTION VISUALIZATION VISUALIZATION & DETAIL

Close-up-view on a lasst sequence to how multi-head ('heads):



TRANSFORMER APPLICATION AREAS

1. **MACHINE TRANSLATION** — Source text → Target text
2. **TEXT SUMMARIZATION** — Document to short, concise bullet points
3. **QUESTION ANSWERING** — Input question to an extracted/ generated answer
4. **IMAGE GENERATION & SYNTHESIS** — Creating high-quality unique faces, but how transformers learn global pixel relationships
5. **COMPUTER VISION** — Computer (Vision Transformer)

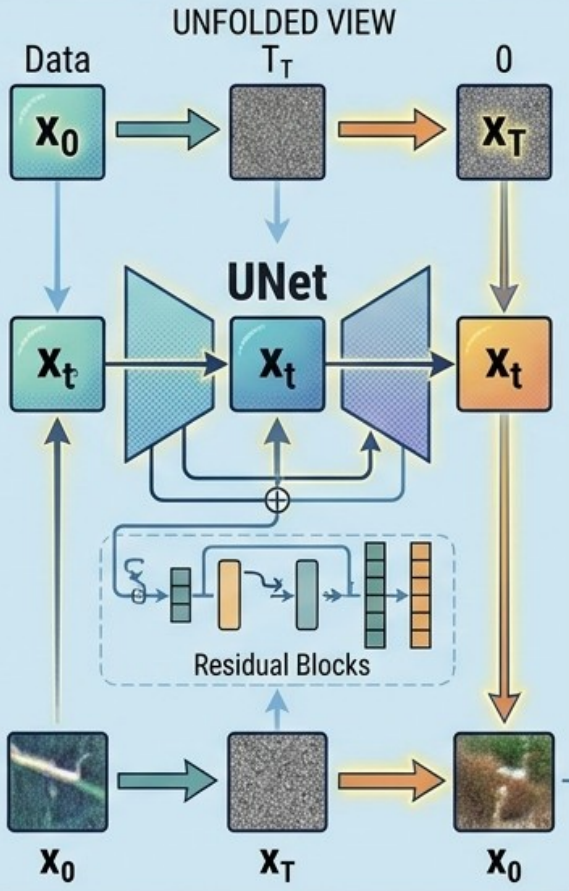
4.3.11 – Classical DL Techniques – Diffusion

UNDERSTANDING DIFFUSION MODELS:
ARCHITECTURE, PROPERTIES, MATRIX OPERATIONS, & APPLICATIONS

DIFFUSION MODEL PROPERTIES

1. **NOISE-TO-DATA GENERATION**
   - reverses noise process
2. **STOCHASTIC DENOISING**
   - probabilistic steps
3. **PROGRESSIVE REFINEMENT**
   - gradual detail creation
4. **LATENT SPACE SAMPLING**
   - operates on latent vectors
5. **DENSE CONNECTIONS**
   - e.g., inside UNet blocks

DIFFUSION PROCESS & ARCHITECTURE



MATRIX OPERATIONS (REVERSE PROCESS - DENOISING)

1. NOISE ESTIMATION

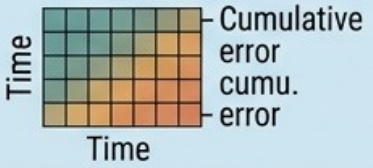
$$[x_t]^T\ (NxD) \times \underbrace{UNet(W, b)}_{(TxD) \longrightarrow (TxD)} \rightarrow [\varepsilon_\theta]^T\ (NxD)$$

2. LATENT STATE UPDATE

$$[x_{t-1}] = f\left([x_t], [\varepsilon_\theta], [\sigma_t], [z_t]\right)$$

$$= \underset{(TxD)}{\square} \times \underset{(TxD)}{\square} \left(\underset{(Tx4)}{\square} + \underset{(1x4)}{\square} \right)$$

3. NOISE ACCUMULATION Visualization



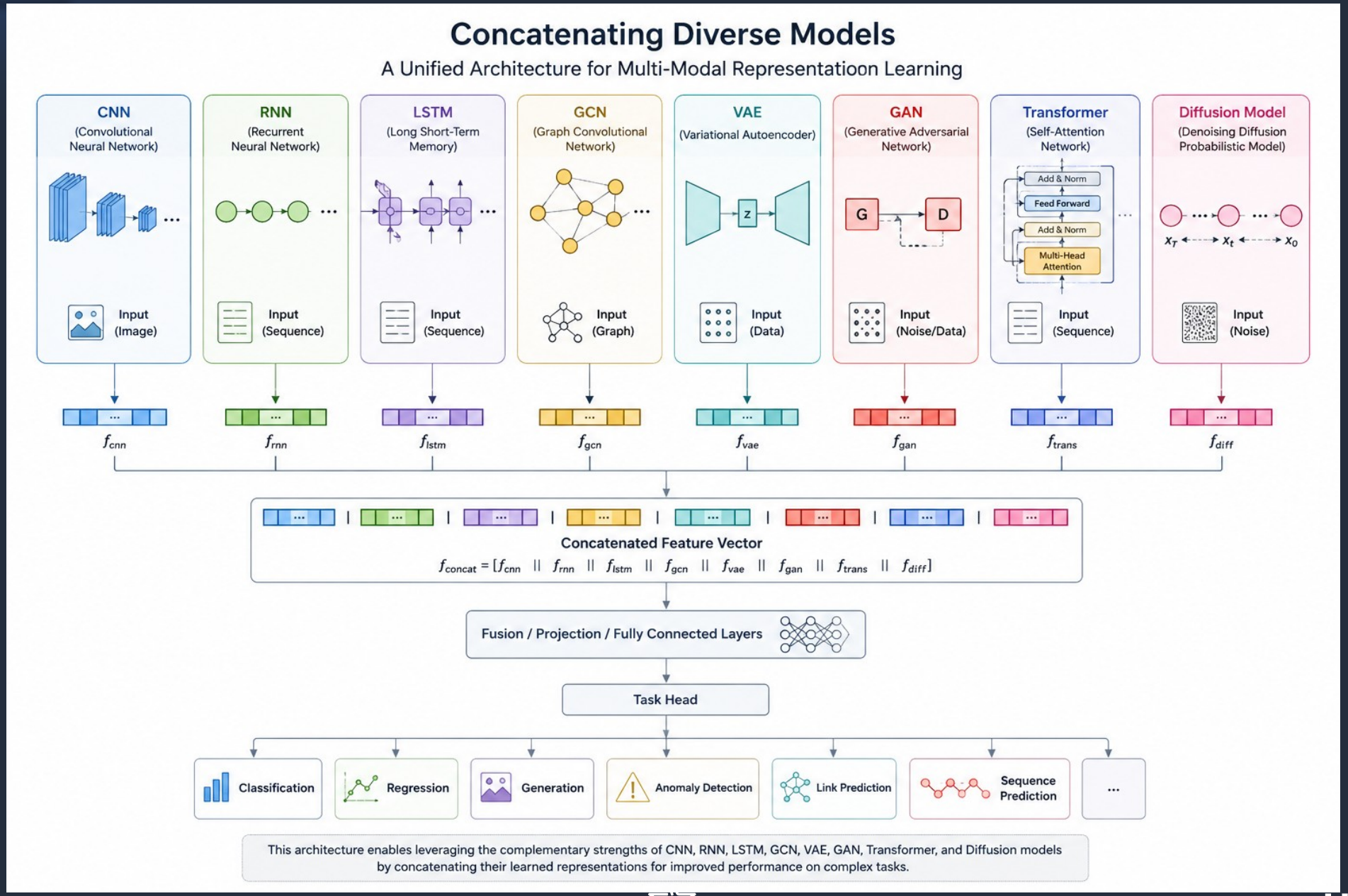
4. ACTIVATION FUNCTION

ReLU or Sigmoid

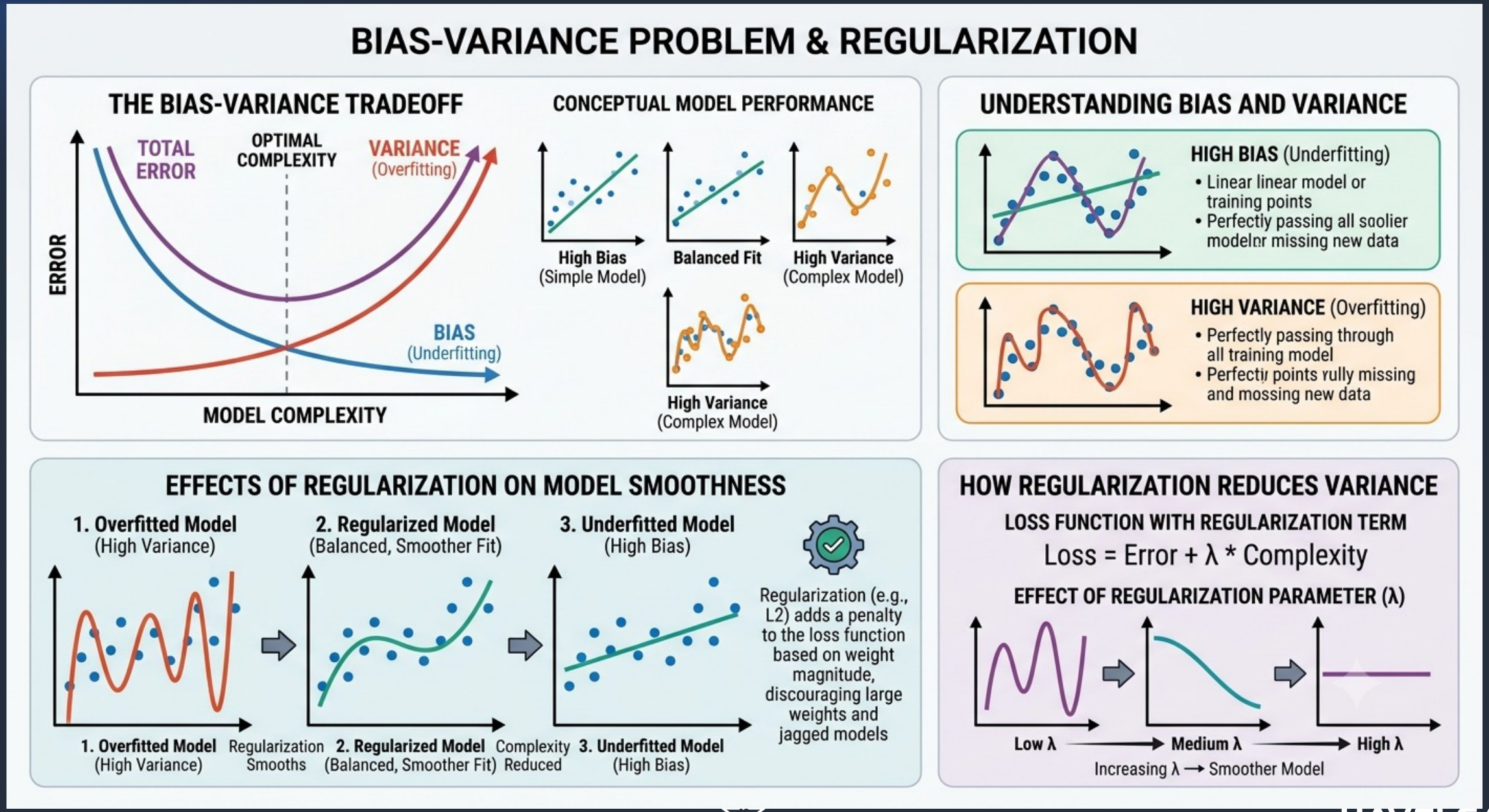
DIFFUSION MODEL APPLICATIONS

1. **IMAGE SYNTHESIS & GENERATION**
2. **IMAGE-TO-IMAGE TRANSLATION**
3. **IMAGE SUPER-RESOLUTION**
4. **TEXT-TO-IMAGE GENERATION**
   - TEXT
5. **DATA DENOISING & REPAIR**

4.3.12 – Classical DL Techniques – Concatenating Models



4.4 Bias/Variance Problem, Regularization ans Smooth Learning



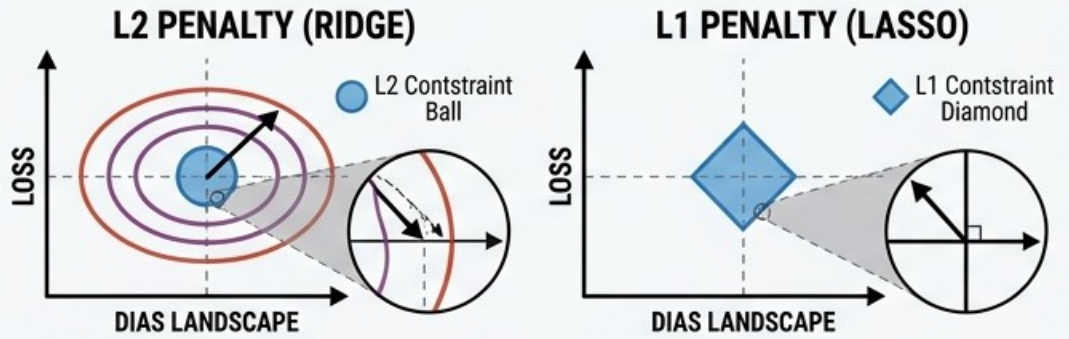
4.5 – Preventing Overfitting (Memorization)

1. *Early Stopping*
2. *Dropout*
3. *Weight Regularization*
4. *Data Augmentation (**Very important)*
5. *Simplify Model*
6. *Batch Normalization*

4.5 – Preventing Overfitting (Memorization)

ADVANCED REGULARIZATION TECHNIQUES & WEIGHT CONSTRAINT METHODS

WEIGHT DECAY & PENALTY CONSTRAINTS

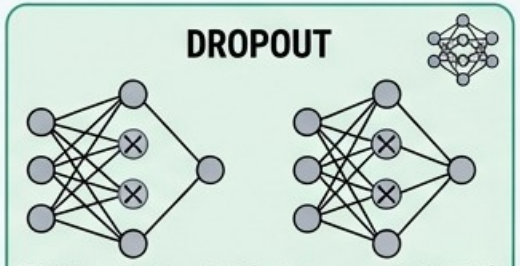


L2 (Ridge): Penalty $\lambda\sum w_i^2$ shrinks weights towards zero, reduces model variance.

L1 (Lasso): Penalty $\lambda\sum|w_i|$ induces sparsity (zero weights), effective for feature selection.

MODEL ENSEMBLING & NOISE TECHNIQUES

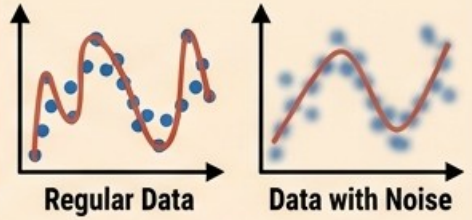
DROPOUT



Training (Active Units) **Inference** (All Units)

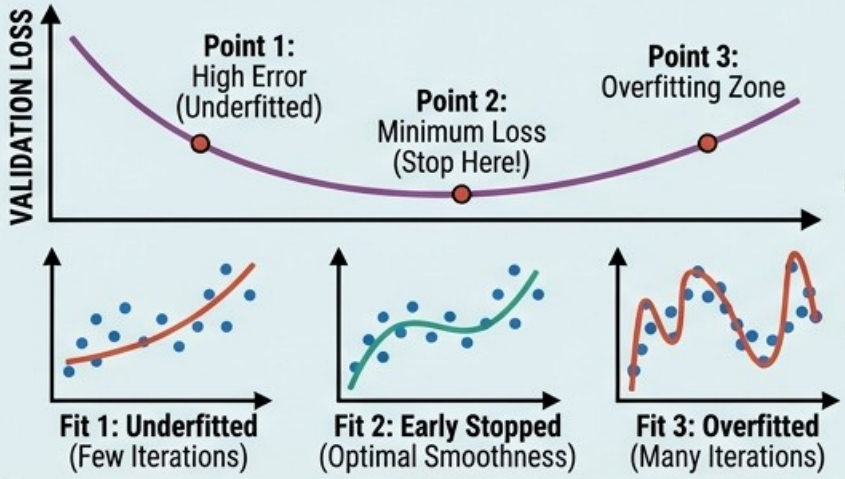
Randomly drops units during training to prevent co-adaptation, works as an ensemble method.

NOISE INJECTION



Adds noise to inputs or weights to improve robustness and generalization.

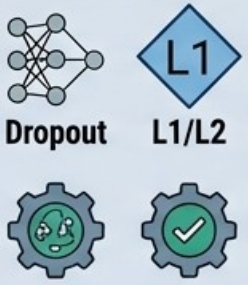
ITERATIVE REGULARIZATION & EARLY STOPPING



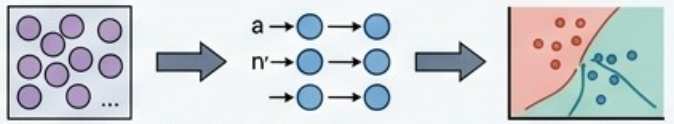
EARLY STOPPING: Monitors validation error to stop training before overfitting occurs, implicitly regularizes.

ADVANCED REGULARIZATION STRATEGIES

COMBINED TECHNIQUES

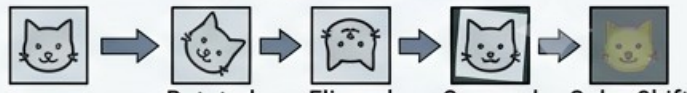


BATCH NORMALIZATION & GENERALIZATION



Batch Norm: Stable training, improves generalization, and reduces the need for heavy regularization.

DATA AUGMENTATION



Data Augmentation: Artificially expands dataset to increase diversity, reduces model's reliance on specific patterns.

4.6 – Data Augmentation

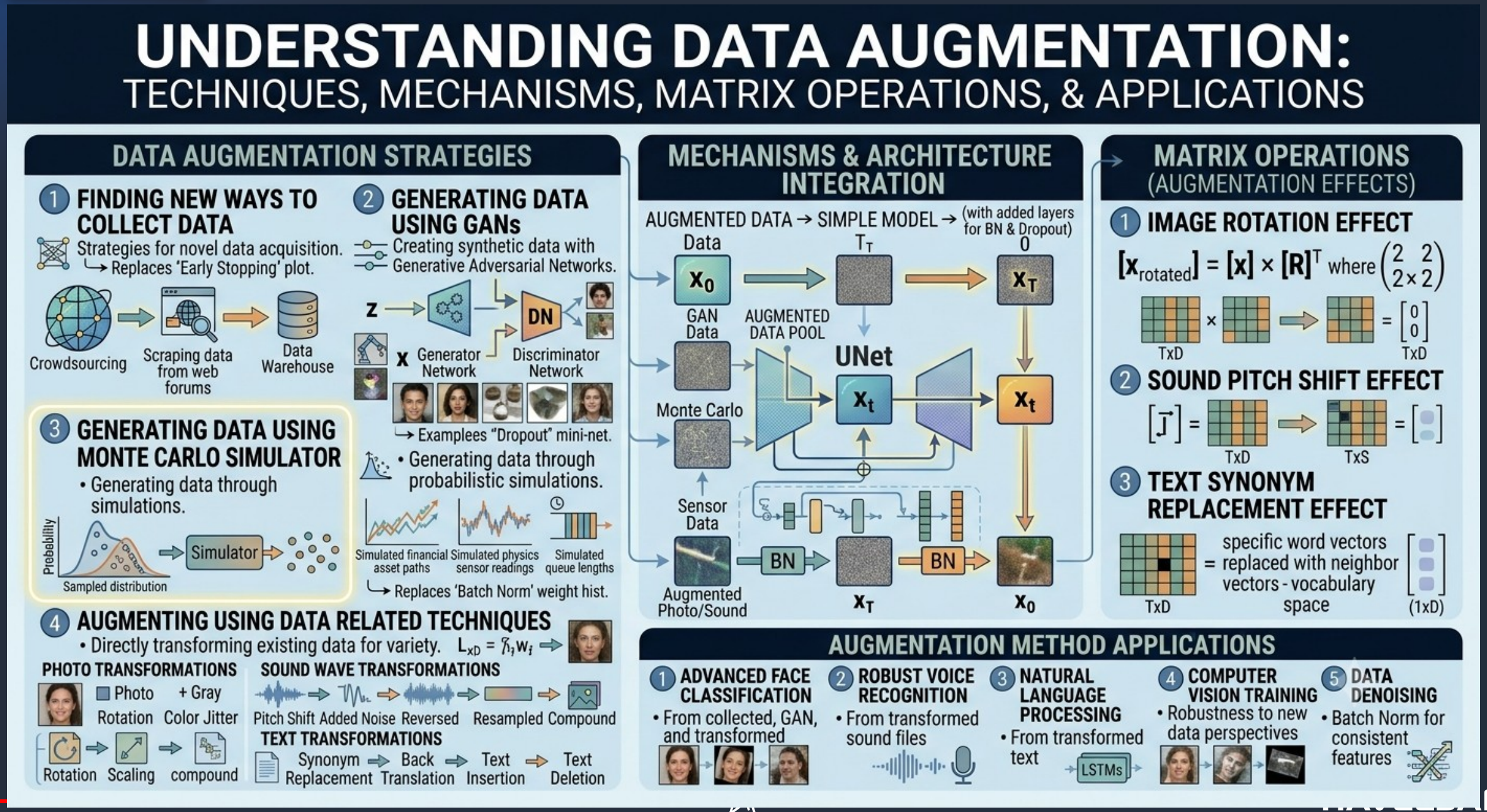
1. Find new ways to collect data : Automate the data collection phase if possible.

2. Generate Data Using GANs

3. Generate Data Using Monte Carlo Simulator (or Deterministic Algorithms / Unity)

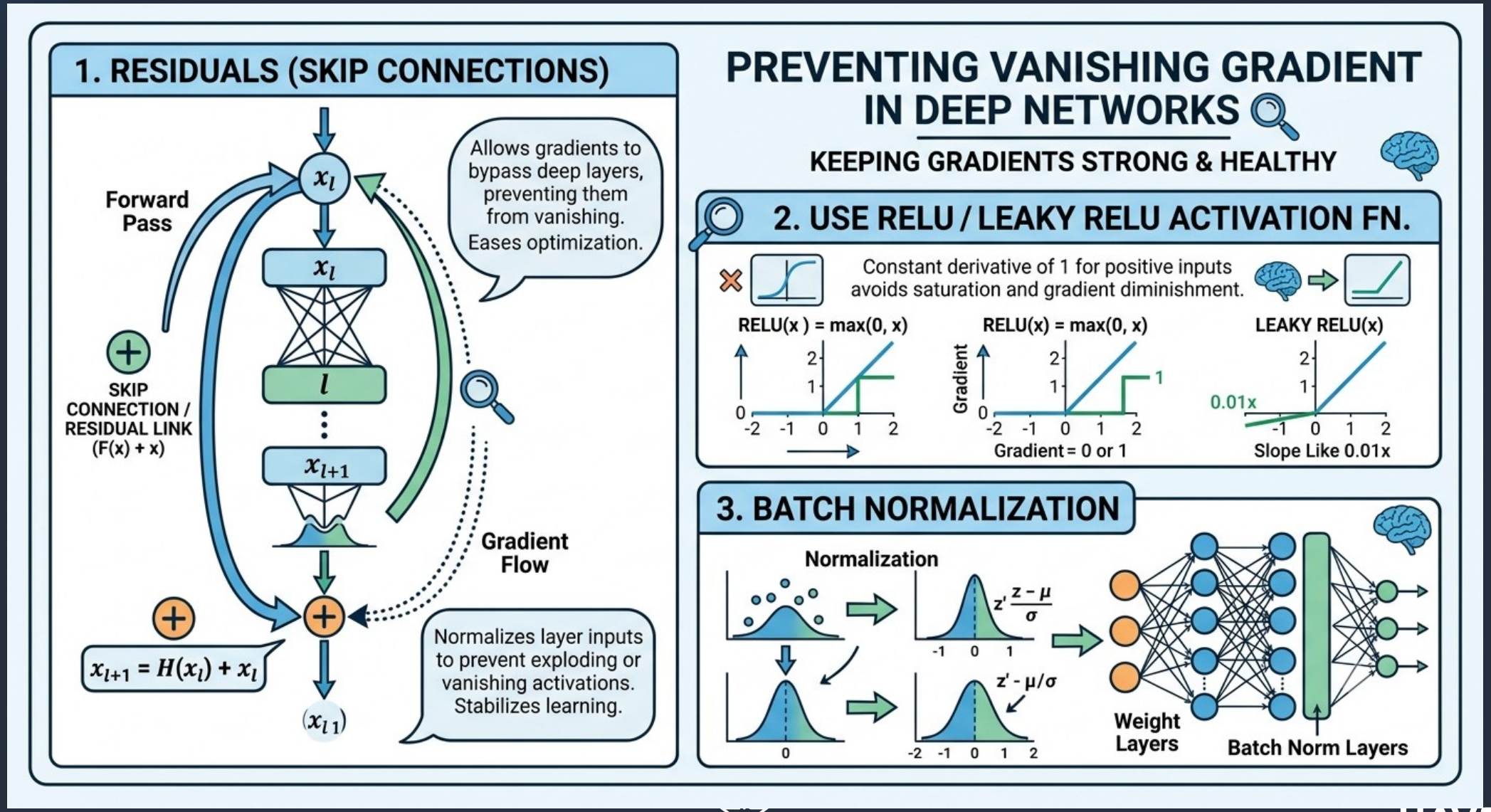
4. Generate Data By Transforming (Noising, Apply Affine Transformations, Frequency Deviations, Image Manipulation Methods, Sound Manipulation Methods, ...)

4.6 – Data Augmentation



4.7 – Preventing Vanishing Gradient

1. *Residuals (Skip Connections), Use RELU/LaekyRELU Activation Fn., Batch Normalization*



4.8 Cross Entropy, Learning Rate Momentum

Cross Entropy, Learning Rate, Momentum

1) Cross Entropy

Measures the difference between the predicted probabilities and the true distribution.

Example (3 classes)

Class	True (y)	Predicted (p)
Cat	1	0.70
Dog	0	0.20
Bird	0	0.10

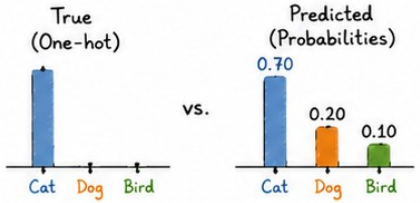
Cross Entropy Loss:

$$L = -\sum_i y_i \log(p_i)$$

For this example:

$$L = -\log(0.70) = 0.357$$

(lower is better)



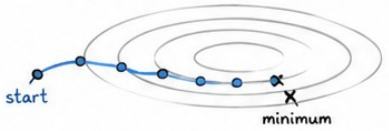
2) Learning Rate (η)

Controls the step size when updating parameters.

Update rule (Gradient Descent):

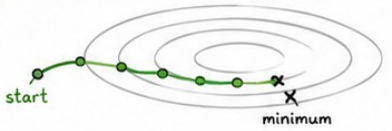
$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Small η: slow but stable



Large η: fast but risky

Good η: efficient convergence



3) Momentum (μ)

Helps accelerate updates in consistent directions and dampens oscillations.

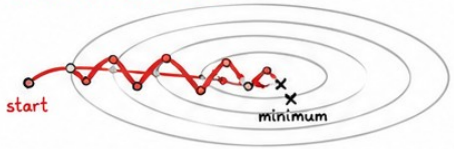
Update with Momentum:

$$v_{t+1} = \mu v_t - \eta \nabla L(\theta_t)$$

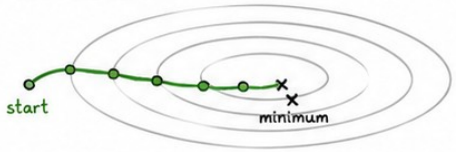
$$\theta_{t+1} = \theta_t + v_{t+1}$$

Intuition:
Momentum = accumulating velocity in the gradient direction.

Without Momentum

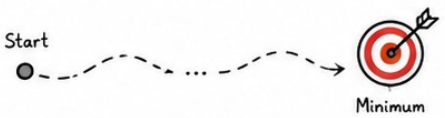


With Momentum



Summary

- Cross Entropy: measures how far predictions are from the truth.
- Learning Rate: determines how big each update step is.
- Momentum: builds velocity to go faster and smoother.



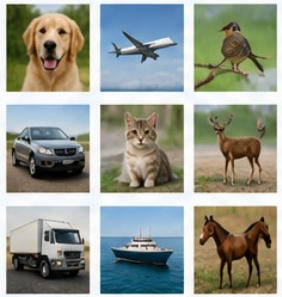
4.9 Transfer Learning

TRANSFER LEARNING

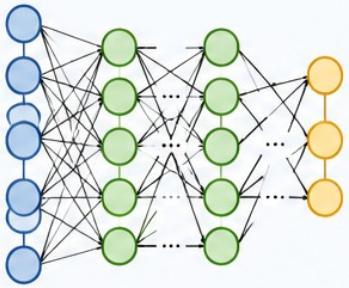
1. Pre-train on Source Task
(Large Dataset)

2. Fine-tune on Target Task
(Smaller Dataset)

Source Dataset
(ImageNet)



Model

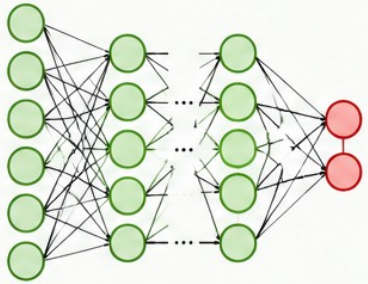


Transfer learned knowledge

(Keep learned weights)

Target Dataset
(Cats vs Dogs)

Model (Fine-tuned)



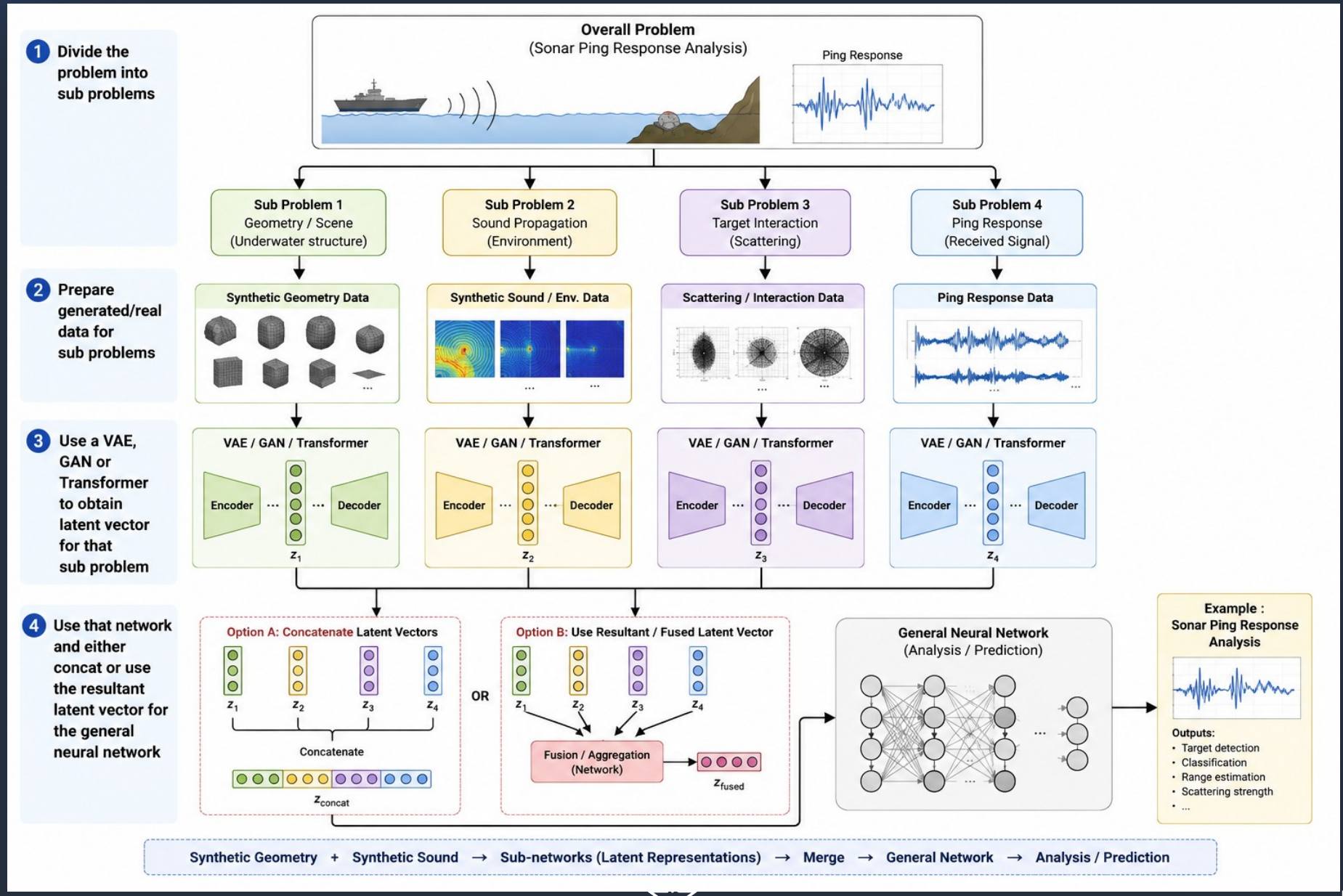
The model learns general features
(edges, textures, shapes, etc.)

The model adapts to the target task
with a smaller dataset

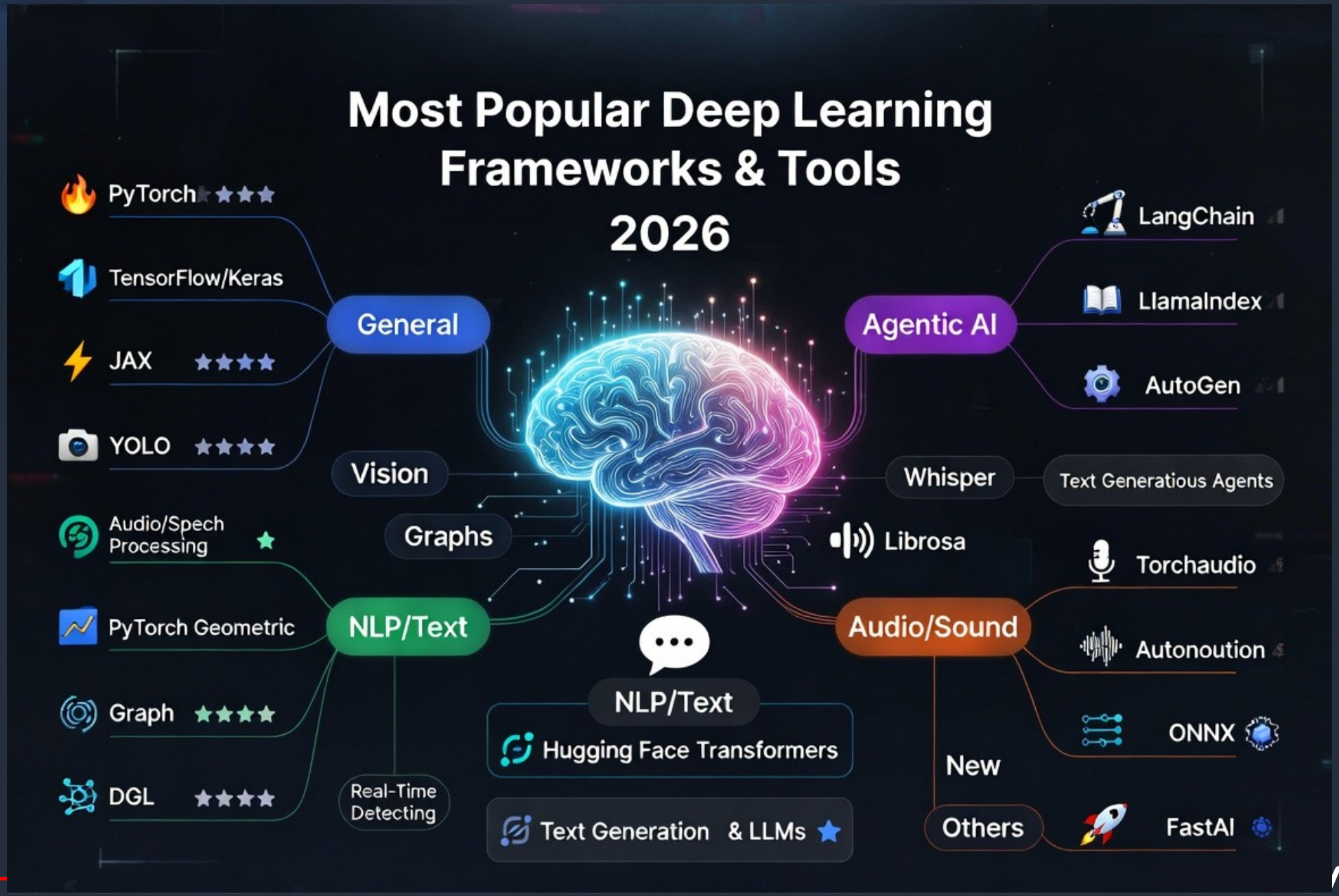
Key Idea

Leverage knowledge learned from a large source task
and apply it to a related target task.

4.10 Solving Problems Using Neural Networks



Popular Frameworks



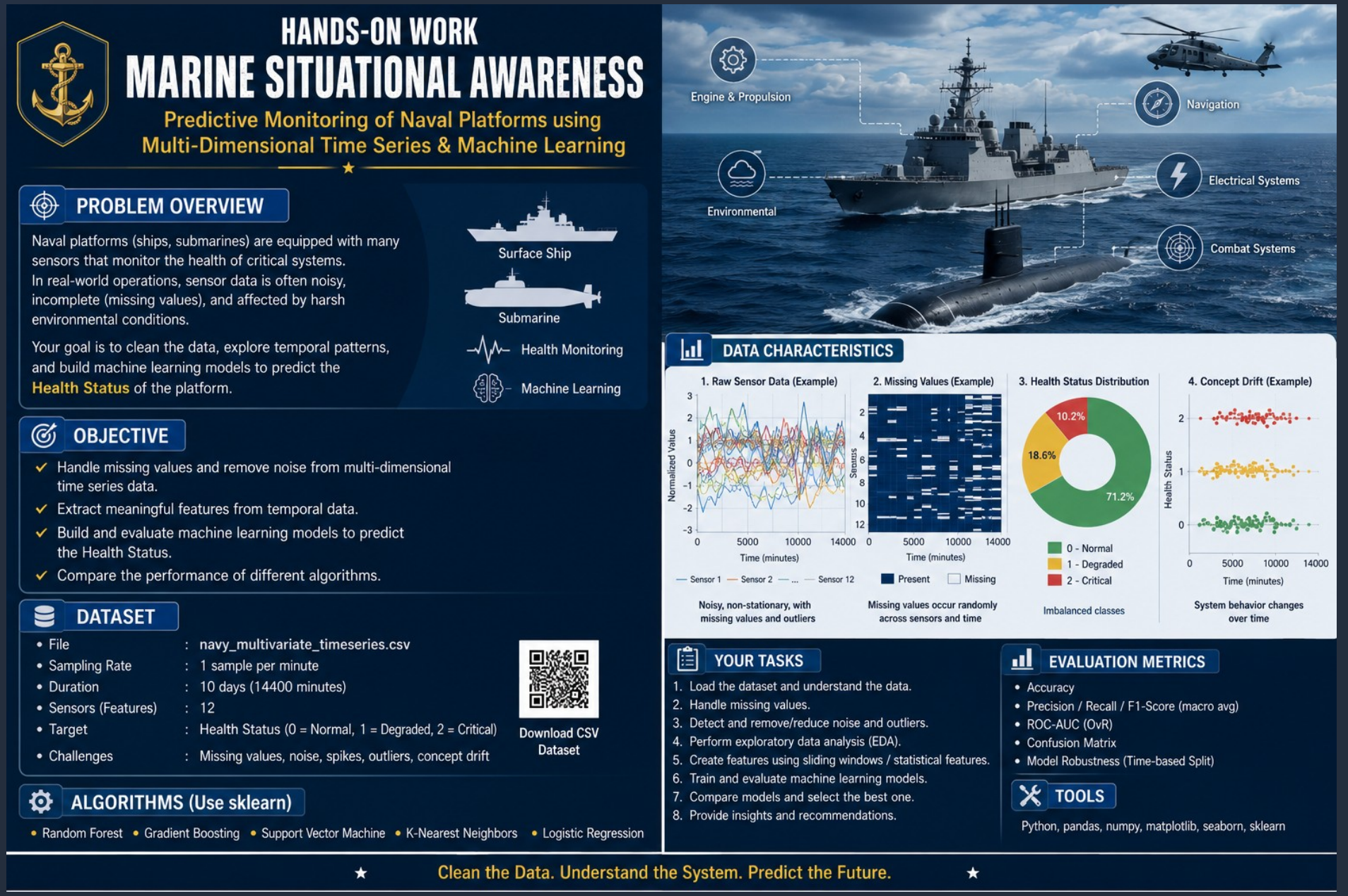
Homework-1.0

Deliver :

1. Data Visualization using Matplotlib defore and after data cleaning.

2. Enumerate meaningful features (columns)

3. Prediction results with evaluation metrics.



Homework-1.1

Problem : Predict the noise level of a room by measuring the light level

Tasks :

1. Collect Data : Write a python program to record 3-second video every hour (webcam+microphone). Leave the program to record for 3 days.

2. Clean the Data : Delete the corrupted files using a python program.
3.
4. Extract sound and images from video using a python program.

5. Use a CNN network that inputs image and predicts sound decibel.

Deliver :

1. Data Visualization using Matplotlib before and after data cleaning.

2. Python programs.

3. Prediction results with evaluation metrics.

TEŞEKKÜRLER

www.havelsan.com